

Formal Architecture and Machinery of the Normative Formal Core and Appendices G, I, J, K, and L

Stuart Swan, Platform Architect, Siemens EDA

31 August 2026

CORE/G Shared semantics + contract	I Local replay	J Safety glue	K Liveness	L Witness selection
---	--------------------------	-------------------------	----------------------	-------------------------------

Based on: Catapult HLS User View Scheduling Rules, dated 31 August 2026

The latest version of the scheduling rules document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The scheduling rules document is the normative source: it defines the scheduling contract, formal models, assumptions, theorem statements, proofs, and evidence obligations. This formal architecture guide adds value by reorganizing that material into a coherent map of how the Core and Appendices G, I, J, K, and L depend on one another, what information crosses each proof boundary, and how design choices connect to the safety, witness-selection, and deadlock arguments. It therefore serves as a navigation and conceptual-integration aid rather than an alternate specification; when its simplified explanation and the scheduling rules differ, the scheduling rules remain authoritative.

Contents

1	Executive orientation
2	Architecture at a glance
3	Design choices that determine proof applicability
4	Core vocabulary and proof boundaries
5	Normative Formal Core and Appendix G - shared semantics and observable contract
6	Appendix I - per-process replay and preparation
7	Appendix J - local-to-global safety construction
8	Appendix K - conditional deadlock preservation
9	Appendix L - witness selection and snooping
10	How the appendices work together in a verification flow
11	Assumptions and evidence matrix
12	Common misunderstandings
13	Quick-reference index

1. Executive orientation

PURPOSE

Provide a single mental model for the Normative Formal Core and five appendices that otherwise appear as separate layers of formal detail.

Engineer takeaway: Treat the architecture as a proof stack: the Core fixes the shared operational semantics; G states the observable contract; I generates local evidence; J constructs the global safety witness; K proves a separate conditional internal-deadlock result; and L selects source witnesses when latency-sensitive choices matter.

The formal architecture is best understood as two related, but deliberately separate, proof pipelines built on one qualified synchronous cycle model.

- **Safety pipeline (Core → G → I → J).** Shows that a covered reachable RTL prefix has a matching reachable source prefix at the chosen observable boundary. Core.17 supplies the canonical Σ Match occurrence/value relation and Appendix G expands it with E1-E5. Core.SelectLatitude makes finite source scheduling latitude constitutive: otherwise legal L1 issue or L3 boundary actions need not be selected maximally in a finite source prefix, and this legality does not import Core.IssueFair or B2/B3 into Theorem J.1. On the normal compositional route, source-side Core.QSync/Core.QSync-Ref supplies the finite L2-settled snapshot required by J.RegPhaseNF/J.UnitExt; when cycle-sensitive source/environment evidence requires a later source cycle entry before the next nonempty Horizon bucket, J.GWR/J.UnitExt may use a finite invariant-preserving unit-empty CompleteCycle bridge without creating empty rank buckets or equating source delay to the raw RTL Horizon gap. A directly validated J.GWR package may instead carry equivalent checked per-Horizon settling and bridge evidence. For the standard QSync route, J.CycleReach_post plus J.QSyncTraceInd first derive cycle-aligned J.TraceCertCov and J.1-Safe-QSyncCycle without importing the K-only Offer/KObs/drain obligations. Core.RTLCycleCarrier and J.CycleIdealCover then show that every reachable finite outer observable history is an ideal of one of those covered cycle-aligned histories, and J.1-Safe-QSync transfers the conclusion to all reachable finite RTL prefixes for properties satisfying J.IdealPrefixClosed. An already-proved J.QSyncCertInd projects to the safety induction. The generic J result remains finite-prefix reachability over Reach_post/Reach_ref and is observable trace compatibility rather than cycle-by-cycle or full-state equivalence.
- **Liveness/deadlock pipeline (Core → G → I → J → K).** Starts from the J-selected source/RTL package and its common typed Core.18 KObs record, reconstructs the settled message handshake and exact ReleaseOwnerSets_E/WaitOwners_E process dependencies, and then reconstructs the finite already-admitted in-flight transition system used by generalized drain closure. The architecture deliberately has three levels: (1) KObs and Core.FlightStateRec describe current reconstructible state; (2) Core.InFlightStepRec/Q.2 describe a finite current-state action/effect relation; and (3) Core.InFlightCycleExt/J.InFlightPathSound separately prove that reconstructed finite paths are operationally realizable. K.Dead itself remains a literal message-rooted predicate: for a candidate D, generalized DrainClosedNow means there is no qualified finite in-flight path that changes the process-indexed WaitRoots_D family. The action relation has no D/ownership filter, so already-admitted work outside D is explored when it can change D's roots. The deterministic Policy-S reduction remains packaged by K.SerialRoutePremises and A5-S/K.Resp. When no lowering is needed (G1), Sys_K=Sys_ref. When synchronization or aligned non-NB-CF guards require lowering (G2), Core.7b-Core.7e supply a separately typed synchronization carrier/SyncFire and lowering actions, while K.Low.ModalQual plus K.Low.InFlightPathCorr/InFlightEscapeCorr/DeadRootBridge/DrainClosedNowCorr transport the generalized modal drain result without widening MsgOf or the literal WFG. Universal RTL preservation still consumes Core.KCutClosure_post plus K.CertCov, with the standard QSync route deriving both the canonical KCut domain and package totality.
- **Witness-selection adapter (L).** Supplies the WC witness needed by I and J when failed non-blocking polls or other latency-sensitive epsilon choices affect later observable behavior. It selects an admissible source execution aligned with the free-running RTL; it does not redefine the source model or stall the RTL.

KEY ARCHITECTURAL FACT: Safety and liveness are not one theorem

Appendix J establishes a global observable-safety correspondence from local evidence. Appendix K does not strengthen that correspondence into liveness automatically. It adds a different argument with different assumptions: wait-for-graph reconstruction, drain discipline, fairness/progress, cutpoint coverage, and the scheduler-robust source-liveness premise A5-L.

1.1 What the formal framework is trying to achieve

The user-facing goal is a verification flow in which most functional verification and debug can remain at the pre-HLS SystemC level, while the RTL is checked against a precise implementation contract. The formal appendices make that promise conditional and auditable: they identify exactly which source rules, implementation properties, witness choices, boundary abstractions, and liveness assumptions must hold.

How to interpret the qualification detail. The formal architecture names many conditions that can look like new work when they are listed explicitly. The underlying engineering conditions were largely already present in conventional verification practice, both for HLS-generated RTL and for manually written or modified RTL. Reset behavior, boundary and capacity correctness, request persistence, cycle-local settling, arbitration and progress behavior, environment assumptions, and the mapping from intended interface semantics to implementation all have to be valid if source- or specification-level conclusions are to be trusted. The framework makes those dependencies explicit, separates their ownership, and requires evidence for the subset consumed by the selected claim. It can therefore add assurance and explicit accounting without implying that every named condition is a new design-user task.

The central idea is to compare only architecturally meaningful observations: committed message transfers, synchronization operations, and anchored signal reads/writes at selected boundaries. Internal behavior outside Σ is not all represented by the same formal object. Source evaluation and other proof-hidden micro-steps may be ϵ -steps; same-cycle settling is classified separately as δ -settling; a real clock cycle with hidden registered progress but no selected Σ event is a silent τ -cycle; and a B6 idle tick performs no ϵ work. Section 4 distinguishes these cases.

1.2 What the framework does not claim

- It does not claim identical cycle timing between pre-HLS and post-HLS models. Core.SelectLatitude deliberately permits finite non-maximal source selection at L1 issue and L3 boundary-action opportunities; J.HCanon discards incidental source-versus-RTL clock-bucket placement unless the selected environment or other cycle-sensitive evidence makes that spacing semantically relevant.
- It does not claim equality of all internal state, requests, or pipeline registers.
- It does not automatically prove that a particular HLS run or RTL instance conforms to E3, E4, E5/Core.SyncFence, the Core.IC request-lifecycle/attribution invariant, B-faithfulness, Core.RegSeq, Core.ReachPrep, Core.ElabProj, Core.MemFaithful, Core.MemSyncOrder, Core.MemProfile where used, the Core.QSync structural conditions including QS3 realization/adequacy and, for the standard RTL K-facing domain, QS2a origin recoverability, or the other implementation obligations. Core.IC, Core.SyncFence, and Core.SelectLatitude are constitutive legality/selection rules of the abstract Sys_ref; corresponding RTL behavior and any concrete executable source/reference-model conformance remain evidence-backed rather than obtained by definition. In the standard QSync scope, Core.KCutClosure_post and canonical KCut identification are theorem-derived from accepted structural settling evidence rather than supplied as arbitrary normalization assumptions.
- It does not infer global deadlock freedom from the source-order no-reverse rule alone.
- The standard compositional QSync route, and the standard K-facing/universal G-K results, do not cover unqualified non-QSync settling semantics. Pure combinational feedback that is not broken by explicit registered or finite-capacity state, non-deterministic or order-dependent same-cycle settling, and multi-clock instances without a separately specified semantic layer are outside that standard scope. Plain finite-prefix J.1 may instead use a directly validated J.GWR package carrying the required checked per-Horizon L2-settling evidence. A non-QSync design is not thereby asserted incorrect; broader K-facing or universal use requires an explicitly defined and separately qualified theorem extension.

- It does not treat an arbitrary finite or unqualified Policy-S simulation as a complete liveness proof. The primary route requires one selected complete instrumented run plus Definition K.SerialRoutePremises for the serial-reduction applicability conditions: SDet; A1-A4/B5 and the constructive Core.FairPick form of Core.IssueFair/B2/B3; A6-A11/K.BF; K.CV with its K.OpKey/K.CVPred/K.CV-WF provenance certificate; nonwithdrawal, K ϵ , K.Drain and Core.CycleClosure; K.EnvMF/JointFreeze and K.CompareAdvanceLegal; reset-first interruption coverage; fixed-instance binding; and NB-Align/K.Low where triggered. K.RespCov and K.CompleteS are supplied separately to K.2b, and A5-S supplies K.RespCov/K.CompleteS/K.RespClean for K.2c/K.1. For the standard universal RTL conclusion, audited QSync plus J.QSyncCertInd derive both the canonical KCut domain and K.CertCov; a single observed RTL or Policy-S run does not establish that universal package totality. Generalized drain adds a separate current-action/path qualification: every K.1A application binds the applicable Sys_ref K.InFlightQual, and a lowered G2 serial route additionally binds K.Low.ModalQual with separate Sys_K and Sys_ref K.InFlightQual records. Sys_K Core.CycleClosure is not silently transported to Sys_ref.
- It does not make latency-sensitive polling safe merely by hiding failed polls as epsilon steps; such sites require NB-CF, isolation, direct witness construction, or Appendix L snooping.

2. Architecture at a glance

PURPOSE

Show the dependency chain from source/RTL semantics to the final safety and liveness conclusions.

Engineer takeaway: The load-bearing seam is between J and K: J exports one equal KObs record for matched cutpoints; K reasons only through that compact observation plus separate admissibility and liveness evidence.

NORMATIVE FORMAL CORE + APPENDIX G - SHARED SYNCHRONOUS SEMANTICS AND CONTRACT

Core: Sys_ref, Core.8 abstract E4 Commit/Commit(q), Core.9 operational Issue, Core.10 process-owned BoundaryReq, one-global-clock scope, Core.RegSeq/Core.ReachPrep/Core.IC, Core.SelectLatitude, Core.PolicyL/PolicyS and Core.IssueFair, Core.DeclaredTerminal/Core.TerminalIdle, Core.Phase/Core.Settle/Core.QSync, Core.CycleState/Core.CycleResult/Core.RTLCycleCarrier/Core.CycleClosure, Core.FairPick/Core.FairCycleDovetail, Core.SyncFence, Core.7b-Core.7e synchronization carrier/SyncFire/EpochGateAdvance, Core.MemFaithful/Core.MemSyncOrder/Core.MemProfile and J.Mem, BoundaryEvalPoint/Core.KCut/Core.QSync-KCutCanonical, Core.ElabProj/Core.WFG with ReleaseOwnerSets_E/WaitOwners_E, Core.16/Core.Drain/Core.SettleKBridge, Core.InFlightStep/Core.InFlightDrainWF/Core.InFlightCycleExt, Core.17 Σ Match, Core.18 typed KObs plus FlightStateRec/InFlightStepRec reconstruction, and the downstream KCut-closure interface. Message BoundaryReq/MsgOf/WFG remain message-typed; synchronization is a parallel typed carrier used by in-flight/lowering correspondence. G: observable alphabet, source rules R1-R5, and compatibility obligations E1-E5.



APPENDIX I - LOCAL PROOF

Logical replay checkpoint, operational preparation/issue evidence, dynamic-operation identity, deferred NB holes

APPENDIX L - WITNESS ADAPTER

When needed, select epsilon outcomes from free-running RTL to discharge WC and align latency-sensitive choices



APPENDIX J - GLOBAL SAFETY CONSTRUCTION

LocalGWR -> generated D1-D4 semantic dependencies + D5 Horizon stratification -> J.HorizonOrderSafe/DepRank -> validated finite-ideal order -> registered phase-normal replay -> Core.SelectLatitude-backed optional unit-empty cycle bridges between nonempty Horizons -> J.GWR-Comp -> J.0 -> Theorem J.1. Empty RTL Horizons are not synthetic J.CanonicalRank buckets; any required source-cycle bridge is finite, invariant-preserving, and evidence-determined rather than inferred from the raw RTL cycle-number gap. For standard QSync safety, J.QSyncTraceInd -> J.TraceCertCov-QSyncCycle -> J.1-Safe-QSyncCycle establishes cycle-aligned packages; Core.RTLCycleCarrier -> J.CycleIdealCover -> J.1-Safe-QSync extends J.IdealPrefixClosed properties to all reachable finite outer observable prefixes. The richer J.QSyncCertInd used by K projects to the same safety induction.



SAFETY OUTPUT

Post -> Ref matching and J.1-Safe transfer of J.IdealPrefixClosed observable safety properties. On CycleReach_post, QSyncTraceInd supplies cycle-aligned packages without K-only Offer/KObs/drain evidence; CycleIdealCover then gives the all-finite-prefix QSync safety conclusion without per-prefix packages.

KOBS BRIDGE

Common typed Core.18 record <E,w,H,O>. J constructs KObsPost_J and proves equality for the exact selected package, then reconstructs process/channel/frontier/WFG facts plus the current synchronization carrier and current in-flight action relation. KObs does not contain an activation certificate, historical Issue times, or future action opportunities; J.InFlightPathSound is a separate level-3 qualification.

**APPENDIX K - CONDITIONAL LIVENESS**

Equal KObs at matched KCuts -> J reconstruction of message roots/release families and the current in-flight transition system -> K.WaitRoots/InFlightEscape/DrainClosedNow -> release-aware K.Dead -> one certified KCut -> A5-L contradiction. Drain closure is no longer the absence of one enabled residual offer: it is absence of any qualified finite Core.InFlightStepRec path that changes WaitRoots_D. D scopes only the observed roots, not the action relation. K.InFlightQual binds the Q.2 action/effect schema, finite choice/rank, Core.InFlightCycleExt or equivalent continuation evidence, and J.InFlightPathSound. In G1 Sys_K=Sys_ref. In G2, synchronization remains outside MsgOf/WFG; Core.SyncCarrierWF/SyncFire and lowering-only guard/epoch actions feed K.Low.ModalQual. LowAdminNormalize and the lowering-aware LowWaitRoots view support the global stuttering K.Low.InFlightPathCorr; InFlightEscapeCorr + DeadRootBridge yield DrainClosedNowCorr for literal source Dead_K candidates, and generalized K.Low.A5 transports A5L_Sys_K to source A5-L. The primary Policy-S serial reduction otherwise retains K.SerialRoutePremises, K.RespCov/K.CompleteS/A5-S, and the QSync universal package-totality route. q->K is closed at the rules-architecture level; remaining work is mechanization and concrete Q.2/InFlightCycleExt/ModalQual instance qualification.

APPENDIX M.7b PACKAGING OVERLAY - standard QSync certificate profile

Four logical interfaces package the M.7a ledger without replacing it: (1) instance and QSync qualification; (2) J correspondence/refinement; (3) K applicability/provenance; and (4) Policy-S response. The names are packaging interfaces, not new theorem predicates or required physical files. Every artifact remains bound through K.CertHdr/K.CertPkg. Standard J.IdealPrefixClosed QSync safety uses bundles 1-2; the primary universal Policy-S K route uses all four; an alternate A5-L discharge replaces bundle 4 with the corresponding K.A5Cert route payload.

2.1 The main interfaces between layers

Interface	What crosses it
Core -> G/I	The Core supplies Sys_ref, operational Commit/Issue/BoundaryReq and request lifecycle, Core.ReachPrep/Core.SelectLatitude, one-clock QSync/phase/cycle semantics, Core.SyncFence, Core.7b-Core.7e typed synchronization carrier and action semantics, memory profiles, Core.ElabProj, settled handshake and ReleaseOwnerSets/WaitOwners/WFG, Core.16/Core.Drain/Core.SettleKBridge, generalized in-flight machinery Core.InFlightStep/Core.InFlightDrainWF/Core.InFlightCycleExt, Core.17 ΣMatch, and Core.18 KObs plus FlightStateRec/InFlightStepRec reconstruction. The message frontier/WFG remains message-typed; the synchronization carrier is parallel proof infrastructure rather than a widened MsgOf. G constrains the shared model through R1-R5/E1-E5; I consumes the Core semantics and G/E3/E5 discipline.
I -> J	I supplies process-local J.PCert evidence: a logical replay checkpoint plus separate operational preparation/issue facts, exact dynamic identity, typed footprints, and deferred non-blocking holes. These local facts intentionally do not claim global channel enablement or one common source execution.

Interface	What crosses it
L -> I/J	L supplies a witness selector proving WC when epsilon-modeled choices can affect later observable control, values, request attribution, or frontier identity.
J -> K	For one cutpoint, J supplies the reachable source witness, exact NormRef/NormPost KCut endpoints, and common KObs equality after HCanon/OfferReach/OfferConf/KObsConf. J.KObs-Proc/Chan/Offer/WFG reconstruct the message-facing state. J.KObs-SyncCarrier and J.KObs-SyncInFlight reconstruct the typed synchronization carrier and local SyncFire action/effect when selected; J.KObs-InFlight reconstructs the current Sys_ref FlightStateRec and Q.2 action relation. Future opportunity is deliberately not inferred from KObs: J.InFlightPathSound, discharged with Q.InFlightActionEffectSound and Core.InFlightCycleExt/CycleClosure-equivalent evidence, separately proves that finite reconstructed paths are realizable. K.DrainReach/Core.SettleKBridge still qualify the exact reached source prefix; K.InFlightQual qualifies the generalized future in-flight relation. For universal standard-QSync coverage, J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync derive K.CertCov. K then factors WaitRoots, generalized drain closure, release-aware Dead_K, lowering transport, waivers, and diagnostics from these interfaces rather than comparing arbitrary RTL/source microstates.
Policy-S -> K	The design-user activity remains one selected complete instrumented Policy-S execution. K.SerialRoutePremises binds the serial-reduction applicability conditions and A5-S supplies the bounded-response package. The generalized-drain theorem adds model-specific K.InFlightQual. In G1, Sys_K=Sys_ref and the serial route's existing Core.CycleClosure can serve the continuation component for that same model once Q.2 legality/profile closure are proved. In G2, the serial result is first established on lowered Sys_K and the application additionally binds K.Low.ModalQual: exact lowering rules/action schemas, finite administrative normalization, LowStateCorr/RootCorr, MQ4 bidirectional action projection/lifting with finite-administrative-suffix root reflection, and separate Sys_K/Sys_ref K.InFlightQual. K.Low.A5 then transports A5L_Sys_K to the source A5-L consumed by K.1A. No Sys_K CycleClosure or fairness premise is silently transported to Sys_ref.

2.2 Generic theorem machinery versus design-specific evidence

The framework deliberately separates proof algorithms that should be mechanized once from evidence that must be supplied or checked for each design, theorem instance, or RTL build. Appendix T of the scheduling-rules document complements this distinction by identifying the scheduling-specific semantic and proof obligations that remain beyond generic transition-system, simulation, induction, and temporal-logic libraries. It is a conceptual gap map, not an additional theorem or certificate layer: the normative definitions and theorem interfaces remain in the Core and Appendices I-K/K-P, with evidence and qualification interfaces in Appendix M.7/M.7b. Appendix S is likewise a selected foundational mechanization-spine/import-anchor inventory, not an exhaustive side-condition list; Appendix M.7a remains the definitive consolidated per-instance discharge ledger.

Category	Examples
----------	----------

Category	Examples
Reusable formal machinery	Core operational/cycle/QSync semantics; Core.17/Core.18 observation interfaces; local-to-global Appendix-J construction and QSync package induction; Core.ElabProj and release-support reconstruction; Core.7b-Core.7e typed synchronization carrier/SyncFire/EpochGateAdvance; Core.FlightStateRec/Core.InFlightStepRec, Core.InFlightDrainWF/Core.InFlightCycleExt; Q.2 finite action/effect schemas with dangerous-direction effect soundness and finite choice; J.KObs-InFlight/J.KObs-SyncCarrier/J.KObs-SyncInFlight and model-indexed J.InFlightPathSound; K.WaitRoots/InFlightEscape/DrainClosedHeredity/InFlightRelocate/K.InFlightQual; and the Stage-2 K.Low modal stack LowAdminNormalize/LowWaitRoots/ModalQual/InFlightPathCorr/InFlightEscapeCorr/DeadRootBridge/DrainClosedNowCorr/K.Low.A5. The existing serial-dominance K.CV/OpKey/CVPred/SegWork/response machinery and QSync certificate-totality route remain reusable formal machinery. Mechanization, reusable checker implementations, concrete Q.2 template proofs, alternative-route InFlightCycleExt discharges, and real lowering-package MQ4/ModalQual qualification remain implementation work.
Per-design source evidence	Source well-formedness, unique signal anchors including the final-Write-at-shared-anchor canonicalization, Core.RegSeq classification, ReachPrep coverage where eager preparation is admitted, source semantics using Core.9 operational Issue, Core.10 operation-specific BoundaryReq, and the constitutive Core.IC/Core.SyncFence legality rules, and - when a concrete executable reference model is the theorem carrier - conformance to Core.SelectLatitude rather than a hidden eager/maximal-progress rule. Also include source-side QSync structural conformance or direct per-Horizon settling evidence, WC/NB-CF/snooping coverage, Core.MemFaithful for hidden transformed mapped-memory activity, Core.MemSyncOrder where array accesses cross synchronization, J.Mem complete dependency-level shared-memory lowering when cross-process value flow exists, Core.MemProfile when a qualified transformation-invariant logical memory boundary is used, barrier well-formedness, and value/control provenance. Qualified Sys_B^elab templates may discharge reusable settling facts, but whole-source composition/adequacy remains instance-owned.
Per-RTL/tool evidence	E3/E5 scheduling conformance including Core.9 first-request timing, fixed Core.10 process-owned BoundaryReq mapping/owner decode, Core.IC live-owner/nonwithdrawal/back-to-back attribution, and Core.SyncFence; Core.8 boundary-commit mapping from abstract E4 events to mapped valid/ready/payload observations; E4 channel realization; exact capacities/boundaries; B-faithfulness; Core.ElabProj; Core.MemFaithful mapped-memory refinement and Core.MemProfile binding where used; reset behavior; ClockScope; and RTL-side QSync frame/finite-acyclic deterministic-settling/boundary-completeness plus QS3 realization/adequacy. For Sys_B^elab, the checked elaboration includes explicit B_E(d)/occ_d storage channels; $\Pi_{elab}/\pi_{\Sigma,E}$; signed A_E; finite ElabToken(E)/$\Pi_{token,E}$ with functional event-to-token attribution; the complete stateless ready/valid/data network; and Q.1 ReleaseOwnerSets_E family-level soundness/completeness plus derived WaitOwners_E. At epsilon-quiescent cycle-boundary states Q.1(b) checks A_E against independently reconstructed outer Sys_B history; $0 \leq A_E \leq B(c)$ follows from outer E4. Local couplers are not modeled processes. Standard K-facing scope also includes QS2a K-origin recoverability for every reachable RTL KCut. Universal QSync use needs checked J.QSyncCertInd; safety-only use may use J.QSyncTraceInd.

Category	Examples
Per-liveness-instance evidence	The fixed capacities/elaboration/stimulus/environment/witness, simulator semantics, Policy-S run identity, and K.SerialRoutePremises binding, plus the model-specific generalized-drain qualification when Appendix K is used. Every selected proof model binds a concrete Q.2 action/effect schema, Q.InFlightChoiceFinite/Core.InFlightDrainWF, Core.InFlightCycleExt or sufficient CycleClosure-equivalent continuation evidence, and J.InFlightPathSound through K.InFlightQual. G2 lowering additionally binds Core.SyncCarrierWF/SyncFire/guard-epoch schemas and K.Low.ModalQual MQ1-MQ6 with separate Sys_K/Sys_ref K.InFlightQual. Separately supply the response evidence K.RespCov/K.CompleteS/A5-S as applicable, exact KCut correspondence packages and K.DrainReach evidence, QSync/QSyncCertInd provenance for universal KCut/package coverage, and the selected A5-L route. Worked Q.2 artifacts illustrate the schema but do not qualify a concrete design/template by themselves.

3. Design choices that determine proof applicability

PURPOSE	Connect common HLS design patterns to the proof layer and evidence route they affect. Engineer takeaway: The architect does not construct the formal certificates, but the design must expose communication, timing, reset, and state-transfer behavior clearly enough for the tool and verification flow to construct them. Appendix F of the scheduling-rules document is the architect-facing checklist; this section explains why those choices matter to the proof architecture.
----------------	---

This guide remains explanatory. Appendix F and the normative sections it references remain authoritative. The purpose here is to connect recognizable design patterns to Core, G, I, J, K, and L without requiring an architect to begin from certificate or theorem names.

3.1 The architect/flow responsibility boundary

ARCHITECT / FLOW BOUNDARY
The architect chooses interfaces, reset topology, memory communication, buffering, and protocol structure that admit the required correspondence. The tool and verification flow construct and check the detailed replay, channel, attribution, footprint, reset, coverage, and liveness evidence. A design may synthesize and function correctly while remaining outside a selected theorem instance when proof-relevant behavior is implicit or modeled at the wrong boundary.

The useful question is therefore not whether the architect can write a certificate. It is whether the chosen architecture exposes enough explicit structure for a qualified flow to determine what happened, assign each request or value transfer to the correct dynamic operation and reset epoch, and compare the same abstract channel and observation boundaries in source and RTL. The G-K theorem stack is a single-global-clock formalism: a selected theorem instance must stay within one synchronous clock domain unless a separate multi-clock semantic layer is supplied; CDC structures are therefore outside the ordinary instance or must be represented by separately verified boundaries. Core.IC and Core.SyncFence illustrate this boundary directly: they define legal behavior of the prescribed source transition system, while the RTL/tool flow must prove the mapped request lifecycle, process-facing completion, and synchronization-fence behavior through the corresponding conformance evidence.

3.2 Design choices affecting the safety and correspondence pipeline

These choices affect the Core $\rightarrow$ G $\rightarrow$ I $\rightarrow$ J pipeline and the Post $\rightarrow$ Ref safety result. They are not merely coding-style preferences: each determines whether the source and RTL provide a common observable contract and whether the local replay records can be composed into one reachable source witness.

Qualified synchronous settling discipline. (Core.RegSeq, Core.Settle, Core.QSync, Appendix Q)

- **Design implication.** Treat the selected theorem instance as ordinary synchronous hardware: one global clock; registered process/pipeline/channel state; sequential request/payload outputs determined from registered state and operands fixed for the cycle; and a complete same-cycle combinational/handshake network that is finite, deterministic, and acyclic after cutting at explicit state boundaries. The accepted transition relation must actually be realized by that qualified settling network. For RTL in the standard K-facing scope, every reachable post KCut must additionally carry QS2a origin recoverability: a unique same-cycle registered/input frame and designated reachable K-origin connected to that KCut only by qualified settling before the next registered update.
- **Why the proof cares.** The L2 frame freezes registered state and selected cycle-t exogenous inputs while delta-settling computes the common boundary snapshot. Core.QSync-Ref makes source SettlePoint existence/uniqueness a theorem. Core.QSync-Post supplies the RTL KCut/closure interface, and QS2a lets Core.QSync-KCutCanonical prove that every reachable standard-scope RTL KCut is one of those canonical same-cycle endpoints. This canonical domain is what the J.QSyncCertInd/K.CertCov-QSync package-totality route ranges over.
- **Supported alternatives.** Pure combinational feedback and unqualified non-QSync settling semantics are outside the standard compositional QSync and K-facing/universal G-K scope. Plain finite-prefix J.1 may still use a directly validated J.GWR package carrying the required checked per-Horizon L2-settling evidence. A separately defined and qualified extension may target the downstream Core.KCutClosure interface for broader non-QSync use. Tool reports are evidence only to the extent they are trusted or independently checked; accepting an unverified settling claim enlarges the declared trusted base.

Source selection latitude and finite deferral. (Core.SelectLatitude)

- **Design implication.** The selected Sys_ref semantics must not impose hidden maximal progress at L1 issue or L3 boundary-action selection. Under both Policy L and Policy S, an otherwise legal action may remain unselected for a finite opportunity while all ordinary legality, reset, environment, arbitration, atomicity, and Core.16 restrictions continue to apply. A concrete executable reference model used as the theorem carrier must be shown to admit this latitude.
- **Why the proof cares.** Post $\rightarrow$ Ref witness construction may need to postpone a reached-but-unissued request or an enabled boundary action in order to realize the selected RTL-derived observable sequence. This is finite-prefix legality, not fairness: Core.IssueFair and B2 are the separate complete-execution eventual-selection requirements. Nonselection does not imply that Core.16 disabled the action, and Core.SelectLatitude never licenses an action that Core.16 forbids.
- **Supported alternatives.** Use the normative abstract Sys_ref or a translation-validated/symbolic source carrier that exposes the same non-maximal selection semantics. Do not repair a finite-prefix J construction by importing B2/B3. If legal source delays can change the canonical Σ_{DUT} history, a one-run J.RefProjCover discharge needs the corresponding latency-coverage argument; pure unit-empty delay that leaves the canonical history unchanged is already observationally absorbed.

Observation boundaries and signal timing. (Appendix F: safety items 1-3)

- **Design implication.** Give every sequential signal read and write one unambiguous synchronization anchor. Keep direct inputs stable for the applicable reset epoch or update them through the supported synchronization handshake. Environment and testbench decisions used by the proof should depend on prior observable causal history, not on an unmodeled absolute cycle count.
- **Why the proof cares.** Appendix G defines the selected signal observations, Appendix I replays the process state at those anchors, and Appendix J composes complete observable units. An ambiguous branch-dependent anchor or a hidden latency-triggered environment decision does not identify one stable observation for the construction to match.
- **Supported alternatives.** Move the signal operation to a unique boundary, use hls_direct_input_sync, expose the timing decision through an explicit protocol, isolate a cycle-accurate or latency-sensitive block, or verify the timing-dependent behavior directly at RTL rather than assuming it transfers from an arbitrary source run.

Non-blocking calls and invisible polling behavior. (Appendix F: safety items 4-5)

- **Design implication.** A failed PopNB or PushNB may be hidden from the committed observable trace, but it is not harmless when its result or cadence determines arbitration, retries, timeout counters, payloads, endpoint choice, or the next visible action. An unbounded loop of failed polls is also not a stable wait in the ordinary replay fragment.
- **Why the proof cares.** Appendix I may carry a deferred non-blocking decision only past work proved independent through NoDependentUse. Appendix J later makes the shared snapshot decision. When the outcome selects later observable behavior, WC must identify a realizable source witness; Appendix L is the common implementation-alignment route.
- **Supported alternatives.** Prefer blocking communication when the architecture permits it. Otherwise prove NB-CF, use a valid Appendix L WC/L.Dir witness, directly construct the witness, or isolate and explicitly model the polling logic as a timing-sensitive protocol block.

Mapped memory, shared state, and the selected proof boundary. (Appendix F: safety item 6 and deadlock item 9)

- **Design implication.** Every mapped-memory instance in proof scope has a declared untransformed logical mapping semantics and selected logical boundary. Caching, merging, splitting, or reordering may occur below that boundary only when Core.MemFaithful shows that the transformed realization preserves the selected Core.17/E1-E5 boundary behavior relative to the untransformed reference. An event already selected at a message or anchored-signal boundary cannot be hidden merely because it arose from memory mapping. Core.MemSyncOrder independently preserves synchronization fences; in particular, one merged storage commit may not represent accesses on opposite sides of the same synchronization boundary.
- **Why the proof cares.** J.Mem is triggered by every shared-memory dependency through which one process's write can affect another process's control flow, payload/value expressions, or the identity or order of its later operations. The audit is dependency-based rather than per physical RAM access, but it is complete: the dependency must be lowered through covered proof-visible operations or the separate memory-state extension, and no other qualifying unlowered memory-carried path may remain. If a transaction-changing transformation would alter the event stream used for that lowering, the standard reusable route is Core.MemProfile, whose logical boundary lies above the transformation and satisfies Core.MemFaithful/J.Mem/Core.MemSyncOrder.
- **Supported alternatives.** For J, use a qualified Core.MemProfile boundary, the separately supplied memory-state extension, or place the unsupported transformed shared-memory case outside Theorem J.1. For K, hidden memory realization must be WFG-inert or the qualified profile must expose a transformation-invariant K-faithful logical boundary that represents the relevant capacity, backpressure, and release dependencies. A persistently backpressuring realization with no such boundary cannot compose transaction-changing transformations with the Appendix-K proof; disable those transformations or exclude that memory from the K theorem instance. Multi-client profiles also inherit A6 explicit-arbiter/point-to-point ownership, RW reset disposition, and Appendix Q.1 when the exposed boundary is Sys_B^elab.

Channel realization and coupled structures. (Appendix F: safety items 7-8)

- **Design implication.** Model the channel behavior actually provided at the selected proof boundary. A positive-capacity boundary must satisfy the Core.12 non-fall-through E4 semantics; internal bypass/fall-through may be hidden only when the chosen abstraction absorbs it without changing the selected boundary semantics. For coupled multi-input pipelines, bundles, joins, shared stages, couplers, or epoch gates, use the explicit Sys_B^elab structure rather than treating outer capacities as independent slack.
- **Why the proof cares.** E4 channel reasoning and J/K reconstruction are sound only when enablement, occupancy, transfer ordering, and release dependencies at the selected boundary match the implementation. Hidden bypass or coupled readiness can otherwise produce the wrong replay state, frontier, ReleaseOwnerSets_E/WaitOwners_E projection, or wait-for graph.
- **Supported alternatives.** Move the proof boundary to one that restores the declared E4 semantics; model a bypass as an explicit rendezvous path; or use a qualified Sys_B^elab template with explicit finite B_E(d)/occ_d storage/rendezvous channels, the complete finite stateless ready/valid/data network, a checked coordinated Core.ElabProj package (Π_{elab} , $\pi_{\Sigma,E}$, $\Pi_{token,E}$, signed A_E), and the required process release-support projection. Stateless coupler logic remains realization detail rather than a modeled process vertex. When Appendix K generalized drain can traverse proof-internal staged/join/synchronization/gate state, the same qualified template also declares the applicable Q.2 finite action/effect schema and its finite-state/rank evidence. The three Q.2 worked artifacts are architecture-validation examples, not automatic qualifications of a concrete template library.

Reset topology and cross-epoch transaction disposition. (Appendix F: safety item 9)

- **Design implication.** Reset both endpoints and the state of a live channel consistently, or define an explicit flush, abort, cancellation, or end-of-epoch protocol. Outstanding requests and in-flight data cannot be silently retained, discarded, or reattributed across a reset boundary.
- **Why the proof cares.** Appendix J represents reset as an atomic observable unit with a named scope, fresh dynamic identities, request clearing, reset-state correspondence, and framing outside the scope. Appendix K adds a second question: whether a reset can occur during the serial comparison without invalidating the pending operation used by the reduction.
- **Supported alternatives.** Use a channel-consistent reset scope, make cross-epoch disposition an explicit observable protocol, or supply a separately verified reset-state extension. Reset scopes are stated in terms of the logical source endpoint owners and the modeled process identities associated with elaborated dependencies; a stateless coupler is not invented as a reset-scope process. For the Policy-S deadlock route, either provide candidate-specific `K.DomResetScope` plus reset-first `K.ResetEpoch/K.TerminalDisposition` gate records, or use the normative uniform instance-level U1-U3 discharge: one conservative dominance scope covering all potentially relevant processes/boundaries, no permitted reset intersecting it, and no relevant pending frontier at any reachable `DeclaredTerminal` state. If neither route can be certified, use another A5-L route.

3.3 Additional design choices affecting the deadlock pipeline

The following choices affect the additional Core $\rightarrow$ G $\rightarrow$ I $\rightarrow$ J $\rightarrow$ K liveness pipeline. They are not extra premises for every use of the Appendix J safety theorem. They matter when the claim is that the defined reachable closed internal message-passing wait-cycle cutpoint cannot occur.

Unambiguous wait ownership and persistent requests. (*Appendix F: deadlock items 1-3*)

- **Design implication.** Each source-level logical message channel should have one producer process owning its Push operations and one consumer process owning its Pop operations; shared buses and multi-client resources are represented by explicit arbiters and point-to-point process-facing channels. In `Sys_B^elab`, an evaluated explicit boundary may instead meet a stateless coupler on one side. That coupler is not a process owner or WFG vertex: the qualified elaboration declares the modeled-process release supports for the settled blocking condition. Appendix K's deadlock observation boundary must include every WFG-relevant explicit abstract message-passing boundary; RTL micro-staging already absorbed into the selected B(c) remains internal. Multi-lane or burst behavior must be explicit. Once an ordinary blocking request is issued, Core.IC requires the same attributed dynamic instance (and stable Push payload) to remain pending until process-facing commit or an affecting RW2 reset; inserted latency is permitted under continued-request attribution. Independent nonterminal cancellation, withdrawal, retry, or reattribution is outside the present G-K theorem model.
- **Why the proof cares.** For each disabled frontier item, Core.WFG receives `ReleaseOwnerSets_E`: a finite antichain of minimal nonempty modeled-process release supports. Processes inside one support are conjunctive requirements and different supports are alternatives. `WaitOwners_E` is only their union for the conservative directed WFG; `K.Dead` keeps the unflattened family for exact release closure. Q.1(j) owns reconstructibility: the settled handshake and state-instantiated release family must be reproducible from E and the reconstructed current state, with `J.KObsConf` separately validating the same equations/signals/schema against the RTL boundary mapping. Q.1(k) is the release-family correctness firewall relative to the declared template semantics: support soundness forbids spurious rescue alternatives, while support completeness forbids missed genuine rescue alternatives. Both directions matter because `K.Dead` consumes the exact family for release closure while Core.WFG consumes its `WaitOwners` union for connectivity. At an ordinary primitive point-to-point boundary the family reduces to $\{\{Q\}\}$ for the unique complementary process owner. Hidden arbitration, an unqualified coupler dependency, or a speculative request that can disappear would invalidate the stable release/dependency and attribution facts used by the wait-cycle theorem.
- **Supported alternatives.** Insert an explicit arbiter process, expose lanes or beats in the abstract interface, or use a qualified `Sys_B^elab` template whose Appendix-Q contract supplies the Q.1(j) reconstructible stateless handshake/release schema and the Q.1(k) sound/complete `ReleaseOwnerSets_E` process supports. Do not create fictitious process vertices for local coupler logic or treat a shared/coupled resource as an ordinary scalar point-to-point dependency without that projection. For the primary Policy-S serial route, Q.1(j)/(k) qualification alone is not sufficient for every coupling-sensitive topology; apply the Q.1(l) requirement below when `K.2b` consumes such a boundary.

- **Serial-route coupling qualification.** When K.2b consumes a coupling-sensitive boundary, Q.1(l) additionally requires a finite handshake-support description and CouplerPersistence: once the earlier support facts and persistent request roots needed for the target transfer are established, every permitted pre-target action in the fixed frozen continuation must preserve those support facts or preserve ChannelEnabled at the next applicable settled opportunity. Q.1(d)-(k) well-formedness alone does not establish this for mutual-stall, shared-capacity, or arbitrated/non-monotone handshake topologies. If the selected template cannot prove CouplerPersistence, that candidate is outside K.PSF/CF-S and must use another A5-L discharge route.

Drain-only behavior after a blocking frontier is reached. (Appendix F: deadlock item 4)

- **Design implication.** Previously accepted work may finish through a finite set of qualified already-admitted proof-model actions, including ordinary already-issued message commits and explicit staged/bundle/join movement. Under the synchronization-qualified profile, an already-pending ready synchronization may complete through SyncFire; lowered Sys_K may also retire explicit guard/epoch administrative state. None of those actions may launch/issue new source-later work behind the blocked frontier. Do not model a peer that still has to execute source control to reach a synchronization as hidden drain work; it remains a release owner.
- **Why the proof cares.** Core.16 activates at the L2 SettlePoint and Core.Drain/Core.SettleKBridge control the actually reached non-replenishing cleanup to later KCuts. Generalized DrainClosedNow at a K-facing observation is a different, current-state modal question: from the reconstructed FlightStateRec, does any finite qualified Core.InFlightStepRec path change WaitRoots_D? The action relation is deliberately independent of D, so outside-D staging or synchronization work is explored when it can change a D root. Q.2 supplies exact current-state actions/effects and finite choice; Core.InFlightDrainWF supplies a decreasing activation-local measure; Core.InFlightCycleExt/J.InFlightPathSound supply real same-/cross-cycle opportunity and path realization. Capacity contributes to finiteness but does not by itself prove a decreasing drain measure or future path existence.
- **Supported alternatives.** Use automatic-flush pipelining or an equivalent proved no-new-later-work discipline together with a qualified Q.2 action/effect model for every proof-internal transfer that can alter a K root. If the action identity/effect depends on hidden register, token, credit, arbitration, or participant state, expose that state at the qualified proof boundary or prove it irrelevant. A custom controller that keeps launching source-later work, or a template for which the required action/effect or path-opportunity qualification cannot be established, requires a different liveness argument.

Scheduler robustness and the actual capacity configuration. (Appendix F: deadlock items 5-6)

- **Design implication.** Do not make deadlock freedom depend on favorable overlap, same-cycle issue, or a preferred arbitration choice when claiming scheduler-robust A5-L. Check the selected design against its actual back-annotated capacities and explicit elaborated boundaries, not an idealized unbounded network or a different rendezvous/buffered configuration.
- **Why the proof cares.** Policy-S intentionally serializes process-facing blocking issue. Definition K.CV is the authoritative design-level causal/value-determinism and provenance premise. K.OpKey and the K.CVPred/K.CV-WF certificate structure discharge its occurrence/provenance machinery; those lower-level definitions do not presuppose an ambient K.CV discharge. The serial-route comparison-availability closure must make every required predecessor fact available on the liberal side without changing DomEpoch, releasing the frozen component, or introducing an unmatched dominance-domain endpoint transfer; recursively materialized non-transfer predecessors carry an explicit Core.16 reach-legality disposition. K.CompareAdvanceLegal separately covers the main P0 preparation/launch and R Issue/assertion compatibility. Ordinary or comparability-certified singleton frontiers use K.CompareAdvanceLegal-Ord; explicit Sys_B^elab multi-frontiers require checked elaboration/order evidence. K.2b then proves that an A5-L counterexample would force a finite response failure on the selected conservative run.
- **Supported alternatives.** Rewrite or buffer the protocol, add explicit synchronization, change the selected capacity/elaboration and rebind the theorem instance, or use a scheduler-specific structural, invariant, exploration, or tool-certificate proof of A5-L.

Reset, environment, termination, and other excluded stalls. (Appendix F: deadlock items 7-8)

- **Design implication.** Treat dynamic reset during the serial comparison through a certificate-supplied K.DomResetScope and the K.ResetEpoch gate; disjoint RW5 resets may remain legal. The dominance scope follows source-owned endpoint operations and the modeled process identities named by the selected elaboration's process-dependency/release-support interface; stateless couplers are not scope processes. Treat modeled EOF/done/abort/end-of-test through the

nested K.TerminalDisposition gate, not by an external host watchdog. Distinguish environment-only stalls, environment-facing co-frontiers, and coupling-sensitive multi-owner/multi-alternative release structures from the internal wait-cycle predicate.

- **Why the proof cares.** Appendix K excludes a specific closed internal message-passing wait structure; it does not convert every form of nonprogress into the same graph predicate. An affecting reset can clear the exact pending operation, a declared terminal disposition can end the selected execution with it pending, an external event may release the component, and starvation or perpetual under-supply can persist without satisfying Dead_K. The Policy-S proof therefore evaluates Reset first, Terminal second, and enters the frozen infinite-continuation argument only on both Continue branches.
- **Supported alternatives.** Use the independently certified ResetEpoch.DirectFail or TerminalDisposition.DirectFail branch where applicable; use the uniform U1-U3 instance-level Continue discharge when one conservative scope can exclude all relevant resets/terminal pending frontiers; or represent terminal/environment protocols through ordinary observable actions. Under StandardEnv, candidates satisfying K.JointFreeze-Ord's ordinary/single-release-support premises need no explicit JointFreeze certificate, while every other admitted candidate - including a single frontier with a non-singleton or multi-alternative ReleaseOwnerSets_E family - requires K.JointFreeze. Under a timed/reactive or otherwise nondefault environment, the serial route requires exactly one selected internal frontier per process together with the single-singleton-release-support K.JointFreeze-Ord premises; K.EnvMF-Uniform is the standard instance-level sufficient discharge when those structural facts hold at every reachable K-facing cutpoint. Otherwise discharge A5-L by a separate proof.

3.4 How to use Appendix F with this guide

The parenthetical tags on the nine Appendix-F-tagged themes give the complete item-level mapping to the nine safety items and nine deadlock items in Appendix F, preserving Appendix F as the checklist. Use Appendix F first as the architect-facing design-pattern screen. For each applicable item, identify the proof layer and evidence family described above, select the intended theorem scope (safety only or safety plus internal-deadlock preservation), and ensure that the tool/verification flow records the corresponding discharge in Appendix M.7a. This guide explains the dependency; it does not replace the normative constraint or the per-instance evidence record.

4. Core vocabulary and proof boundaries

PURPOSE	Define the small set of objects that repeatedly connect the Normative Formal Core and Appendices G, I, J, K, and L. Engineer takeaway: keep four distinctions visible: observable events versus generic epsilon micro-steps; delta-settling versus silent tau-cycles versus B6 idle ticks; committed history versus current residual offers; and safety correspondence versus liveness assumptions.
---------	---

Term	Engineer-oriented meaning
Sigma (Σ)	The selected alphabet of observable events: committed Push/Pop transfers, synchronization events, and selected signal Read/Write events. The proof boundary determines which internal or DUT-boundary signals/channels are included.
Epsilon (ε)	The generic proof-hidden micro-step relation. It includes source evaluation, failed non-blocking choices, and other internal behavior according to the model. It is not a synonym for a silent clock cycle. Delta-settling is a specific same-cycle settling classification; B5 may charge a silent tau-cycle to epsilon work without identifying the two.

Term	Engineer-oriented meaning
δ-settling (delta)	Cycle-local same-cycle settling. On a phase-bearing source model, δ_{ref} is exactly the Core.Settle relation $\rightarrow_{\epsilon, L2}$; on RTL_B in the standard QSync scope, δ_{post} is the same-cycle settling relation of Core.QSync. Delta steps preserve cycle index and registered state. Other epsilon steps may also preserve the cycle but are not delta-settling.
Silent τ-cycle (tau)	A real clock cycle in which no selected Σ event commits while hidden registered/internal progress may occur. Tau is explanatory cycle-level terminology, not a transition label and not an epsilon step. B5 may prove that an internally advancing silent cycle consumes bounded epsilon work.
B6 idle tick	A real clock tick after the complete system has reached the B5/global fixed point. It performs no epsilon-prefix/suffix work and leaves non-clock state unchanged; it is treated as epsilon-stutter only by the observable trace-matching convention.
Observable unit	A singleton Σ event or an explicitly atomic same-cycle group such as a rendezvous Push/Pop, paired synchronization transaction, R2C fixed-point unit, signal-anchor group, or an explicitly selected proof-internal RESET unit. RESET may be carried by the Core.17 observable-unit machinery without being added to Σ .
Commit	Core.8 defines a boundary commit as the selected proof model's legal E4 channel-commit transition, producing the committed Push/Pop unit and abstract channel-state update. For RTL_B, boundary mapping/B-faithfulness relates that abstract event to the clock cycle in which mapped valid and ready are simultaneously satisfied and the payload is observed. Commit(q) is the designated process-facing completion/accept/release occurrence for source operation q; CommitCycle(q) is its cycle index. Commit/release is distinct from L4/Core.RegSeq returned-result availability.
Issue	Core.9: Issue(q) is the first cycle in which the calling process begins requesting the transfer for q at its selected explicit boundary endpoint. It is the cycle index of that operational issue event; witness/scheduler records may carry the value but cannot choose a different timestamp. Core.10 separately identifies the operation-specific process-owned BoundaryReq. Issue is not generally reconstructible from committed labels alone and remains outside KObs history fields.
ReturnedAtCycle	The cycle boundary at which a message-call result becomes process-visible. For a successful blocking or non-blocking call it is the transfer-commit cycle; for a failed PushNB/PopNB it is the cycle in which the failure status returns even though no message commits. It is an operational timestamp, not a KObs field.
Sys_ref	The prescribed source-reference transition system: Sys_B in the ordinary back-annotated case, or Sys_B ^{elab} when explicit staged, bundle, join, guard, coupler, or epoch-gate boundaries are required. Raw rendezvous Sys remains Appendix G's conceptual presentation; the G-K proof stack uses Sys_ref, with B(c)=0 recovering rendezvous semantics.
Horizon	The selected cycle/decision horizon used by J to group construction actions and enforce registered result availability and L1/L2/L3 phase ordering.
Finite ideal	A finite down-closed set of observable units: if it contains a unit, it also contains every unit required to precede it. Appendix J constructs the source witness through a chain of these legal partial-order prefixes.

Term	Engineer-oriented meaning
KObs	The compact Core.18c cutpoint record $\langle E, w, H, O \rangle$. It contains the elaboration/witness binding, canonical committed history, and current residual message offers. It deliberately does not contain historical Issue times, a Core.16 activation certificate, an in-flight-path witness, or future action-opportunity facts. Core.FlightStateRec and the J.KObs reconstruction lemmas derive the current proof-model state/action relation from KObs plus the fixed qualified schemas.
Residual offer	A typed current outstanding source-level request at an explicit abstract boundary, including process, frontier item, dynamic call, direction, and reset epoch.
ReleaseOwnerSets / WaitOwners / WFG	For each disabled message frontier item x of P , the selected elaboration supplies $\text{ReleaseOwnerSets_E}(x, \sigma)$: a finite antichain of minimal nonempty modeled-process release supports. Members of one support are conjunctive; different supports are alternatives. WaitOwners_E is their union, and Core.WFG adds $P \rightarrow Q$ for each internal Q in that union. $K.\text{Dead}$ does not flatten the family: it uses exact release closure over $\text{ReleaseOwnerSets_E}$ plus induced-WFG connectivity. At an ordinary primitive point-to-point boundary the family is exactly $\{\{Q\}\}$ for the unique complementary process owner. Stateless couplers, FIFO stages, and other elaboration-local nodes are not WFG process vertices.
SettlePoint / BoundaryEvalPoint	SettlePoint is the first fixed point reached by the complete source L2 epsilon-settling relation after L1 issue closes and requests freeze; Definition Core.Settle itself is conditional on that endpoint existing. BoundaryEvalPoint is the settled explicit-boundary request/enablement interface. In the standard scope Core.QSync-Ref proves source SettlePoint existence and uniqueness.
Core.QSync	The qualified synchronous same-cycle model. QS1-QS5 require one global clock, registered/frame separation with fixed cycle inputs and stable sequential drives, deterministic local evaluation, a finite acyclic dependency graph, explicit-boundary completeness, and QS3 realization/adequacy tying allowed same-cycle transitions to the settling network. For RTL_B in the standard K-facing scope, QS2a additionally requires unique recovery of the registered/input frame and reachable K-origin for every reachable post KCut. Determinism is unique settled result for fixed entry/drives, not one global Sys_ref execution.
Core.KCut	The K-facing comparison domain: a reachable state that is model-specific epsilon-quiescent and whose explicit $\text{BoundaryReq}/\text{ChannelEnabled}/\text{ChannelDisabled}$ network is at its selected fixed point. Source SettlePoints are KCuts; RTL has its own KCut domain and no source L0-L4 phase tag.
Core.KCutClosure_post	The downstream RTL KCut-existence/non-vacuity interface. In the standard QSync scope Core.QSync-Post derives it, while QS2a plus Core.QSync-KCutCanonical identify every reachable post KCut with the unique canonical endpoint of its recovered same-cycle K-origin and supply the canonical NormPost witness. $K.\text{CertCov}$ remains a distinct package-totality interface, but the preferred standard route derives it from $J.\text{QSyncCertInd}/J.\text{CertExist-QSync}/K.\text{CertCov-QSync}$.
Core.SyncFence	Constitutive Sys_ref synchronization-completion legality: a synchronization may commit only after every earlier blocking Push/Pop in $\text{pref_S}(s)$ has process-facing committed, possibly in the same cycle. On RTL, the same rule is a conformance obligation; equal-cycle “before interval advance” accounting is certificate attribution, not observable physical sub-

Term	Engineer-oriented meaning
	cycle ordering.
Core.MemSyncOrder	Local schedule/mapping-layer qualification for array or memory accesses source-ordered with synchronization calls: mapped storage commits before s occur no later than s ; commits after s occur strictly later. It is recorded in M.7a but does not add memory events to Σ or become an Appendix I/Theorem J.1 replay premise; J.Mem remains separate for cross-process shared-memory value flow.
Core.ReachPrep / Core.IC	ReachPrep permits finite dependency-independent L1 preparation/reach of a source-later operation before an earlier blocking result returns, without erasing source order or implying Issue. Core.9 makes Issue the actual first request cycle. Core.10 makes BoundaryReq the fixed-mapping, operation-specific process-owned request rather than raw channel/coupler availability. Core.IC then gives the constitutive blocking lifecycle: every asserted blocking BoundaryReq has exactly one live owner and remains nonwithdrawably attributed until process-facing commit or affecting RW2 RESET; for a back-to-back next same-direction blocking $q1$, continued process-owned request ownership begins in $t+1$ with $\text{Issue}(q1)=t+1$, with no ownerless interval. DeclaredTerminal may end execution while a request remains pending. RTL lifecycle/attribution and owner decoding are separately checked M.7a conformance evidence.
Core.CycleState / Core.CycleResult / Core.CycleClosure	Core.CycleState is the model-indexed cycle-entry theorem state carrying the registered, environment, clock, reset-unit, and cycle-valued monitor state at the cycle origin. Core.CycleResult is the typed real-cycle outcome carrying the full next state, committed observable units, and scoped RESET units. Core.RTLCycleCarrier derives a narrow post-side fact from this carrier: a reachable prefix that has already entered a legal RTL cycle and contains a complete selected unit can be carried through that same cycle to the successor K-origin; this is not generic RTL progress from an arbitrary K-origin. Core.CycleClosure separately qualifies legal nonterminal source cycles and the partial-cycle extension used by Core.CycleChoiceExt; when K.2b.E builds a fair continuation, Core.FairPick/FairCycleDovetail supply the constructive selector/dovetail bridge.
Core.IssueFair / Core.DeclaredTerminal / Core.TerminalIdle	IssueFair is weak fairness for a reached operation that remains IssueEnabled at every applicable L1 opportunity; merely intermittent legality does not trigger the antecedent, and it is separate from B2/B3 boundary progress. Core.DeclaredTerminal is the modeled machine/test termination predicate; a host watchdog or simulator stop is not terminal unless it is part of the selected theorem-instance semantics. Core.TerminalIdle is the stronger B6-eligible fixed/no-future-enabling condition for machine plus environment; it may be established by a stable declared terminal state or by a permanently idle nonterminating service model. An ordinary temporarily silent cycle is a Core.SilentCycle.
Core.SelectLatitude	Constitutive finite-prefix source selection rule: under Policy L or Policy S, otherwise legal L1 issue or L3 boundary actions need not be selected maximally at a particular opportunity. Nonselection is ordinary legal source-cycle behavior, does not imply Core.16 activation, and cannot override Core.16 or another legality rule. Core.IssueFair and B2 are separate complete-execution eventual-selection obligations; a unit-empty legal cycle may be a Core.SilentCycle with ordinary full-state next-cycle evolution.
Core.MemFaithful / Core.MemProfile / J.Mem	Core.MemFaithful constrains the existing Σ -boundary selection freedom for mapped memory against the declared untransformed logical mapping semantics: hidden cache/merge/split/reorder behavior must preserve the selected outer events/order/values/atomicity and, for K, the represented blocking/WFG facts. J.Mem separately requires complete lowering of every qualifying cross-process memory

Term	Engineer-oriented meaning
	dependency. Core.MemProfile packages the qualified reusable logical boundary above transaction-changing behavior, including MemFaithful, J.Mem, MemSyncOrder, K disposition, A6 arbitration/ownership, Q.1 where elaborated, and RW reset treatment.
FlightStateRec / InFlightStepRec	KObs-pure reconstruction of the finite current proof-model state and one-step already-admitted action/effect relation. Actions are admitted because their work is represented now in requests, explicit storage/token state, or a ready synchronization carrier; the relation has no candidate-D or ownership filter. Operational future opportunity is not part of this relation.
WaitRoots / InFlightEscape / DrainClosedNow	WaitRoots_D is a process-indexed family of current frontier, enablement/disablement, and exact release-support data for P in D. InFlightEscape holds when a finite qualified InFlightStepRec path changes that family. DrainClosedNow is its negation. This replaces the former offer-only snapshot intuition; D selects which roots are observed, not which actions may occur.
SyncCarrier / SyncFire / EpochGateAdvance	Core.7b-Core.7e give synchronization a separate typed carrier: endpoint/transaction identity, participants, pending/ready/blocked state, exact internal SyncReleaseOwnerSets, and the real source SyncFire action. A lowering-only EpochGateAdvance changes explicit Sys_K gate state and projects to source stutter. None of these definitions widens MsgOf, message BoundaryReq, or literal Dead_K/WFG.
LowAdminNormalize / LowWaitRoots / K.Low.ModalQual	Stage-2 lowering correspondence infrastructure. LowAdminNormalize finitely retires enabled lowering-only guard/epoch residue; LowWaitRoots is an extended source root view used only inside K.Low; ModalQual binds the local lowering cases, normalization/rank, root correspondence, bidirectional action projection/lifting with finite-suffix root reflection, separate Sys_K/Sys_ref K.InFlightQual, and certificate/schema versions.

DETERMINISM TAXONOMY - do not conflate these notions

Notion	Logical status	Engineer-oriented meaning
Same-cycle settling determinism	Model-class restriction	R2C/Core.QSync: fixed registered state, fixed cycle-t inputs, and fixed explicit request/issue drives yield one settled boundary valuation.
Conditional synchronous-step determinism	Functional cycle view after explicit choices are fixed	Once the currently legal witness, issue/scheduling, arbitration, and environment choices are fixed, the cycle can be viewed functionally. This does not remove Policy-L latitude or WC-sensitive source nondeterminism.
B1 observable RTL trace determinism	Theorem-instance assumption	Under fixed stimulus and initial state, RTL_B has one observable Sigma-projected committed history; internal epsilon behavior may still differ.
I.DR / I.DR'	Derived theorem	Replay-checkpoint equality proved from the Appendix I premises. It is not an independent assumption to enter in the M.7a discharge record.
K.CV	Appendix K premise	Authoritative Appendix-K design-level causal/value determinism and provenance premise. K.CV binds the

		common OpKey universe, the K.CVPred/K.CV-WF typed fact-provenance certificate, serial-route comparison-availability, and observation determinism/isolation conditions under the fixed theorem instance. K.CVPred/K.CV-WF supply the low-level CV2 certificate structure and do not assume an ambient K.CV discharge. The broad Appendix-G causal graph and rendezvous mutual enablement are not automatically provenance predecessors.
SDet	Selected-instance fixing/premise	Fixes the Policy-S simulator, scheduler, arbitration, normalization, and related choices so the selected instance has no second incompatible committed history.
Normalization selection/congruence	Selected proof data plus theorem	Records which replay normalization is selected and why the replay quotient is preserved. Core.QSync canonicalizes only K-facing same-cycle settling; Appendix I/J replay normalization remains separate.

For theorem premises, Appendix M.7a supplies the evidence axis: inapplicable to the selected theorem scope, satisfied by a syntactic/design restriction, discharged by tool/RTL evidence, or discharged by a source/RTL witness construction. A required condition may not be marked inapplicable; derived theorems such as I.DR do not become independent premises merely because their proof artifacts are recorded.

CUTPOINT BOUNDARY: KObs is not full-state equivalence

KObs is the typed Core.18 cutpoint record $\langle E, w, H, O \rangle$. Core defines the source record, its pure reconstruction functions, and the cross-model equality notion; J.OfferReach/OfferConf/KObsConf establish one realized common record for the selected source/RTL package. J.KObs-Proc/Chan/Offers/WFG reconstruct process replay, explicit channel state, message frontier/request roots, settled handshake, ReleaseOwnerSets_E/WaitOwners_E, and Core.WFG. Stage 2 adds J.KObs-SyncCarrier/J.KObs-SyncInFlight and J.KObs-InFlight, which reconstruct the current synchronization carrier and finite Q.2 in-flight action/effect relation without adding fields to KObs. Core.FlightStateRec explicitly excludes activation certificates and future opportunity claims. J.InFlightPathSound is a separate model-indexed operational realization theorem: it consumes Q.2 effect soundness plus Core.InFlightCycleExt/CycleClosure-equivalent evidence. Equal KObs is therefore typed current-observation equality, not full-state equivalence, historical Issue equality, or a proof that every reconstructed future edge is realizable.

4.1 Safety direction: why Post -> Ref is the normal signoff argument

For ordinary safety signoff, the useful deductive direction is: every covered reachable RTL prefix has a matching reachable Sys_ref prefix with the same canonical observable history and Core.17 Σ Match events/values. Therefore, if all relevant Sys_ref histories satisfy a J.HCanon-invariant safety property meeting Definition J.IdealPrefixClosed, the covered RTL histories do too. Here “prefix-closed” means downward closure over finite required-order ideals, not merely deleting a suffix from one chosen linearization of independent events. When one complete source run is used as the verification representative, J.RefProjCover must cover the relevant source observations by finite canonical ideals/down-closed observable prefixes of that run. In the standard QSync route, J.CycleReach_post/J.QSyncTraceInd/J.TraceCertCov-QSyncCycle first establish the cycle-aligned Post -> Ref packages; Core.RTLCycleCarrier and J.CycleIdealCover then cover every reachable finite outer observable history by a reachable cycle-aligned history, and J.1-Safe-QSync uses J.IdealPrefixClosed to obtain the all-finite-prefix safety conclusion without creating a separate J.GWR/J.TraceCertCov package for each intra-cycle ideal. Assertion-style no-bad-event invariants, per-channel FIFO/value legality, and explicitly downward-closed protocol invariants are natural uses. A cross-interface scoreboard whose check depends on an otherwise droppable explanatory event may fail J.IdealPrefixClosed and should instead use direct J.TraceCertCov on the required arbitrary-prefix domain (or refine the retained observation/order).

The reverse direction is intentionally a selected-pair compatibility result, not a constructive-existence theorem. J.RTLConsistentRefPrefix is typed over a reachable source prefix already equipped with a selected reachable RTL prefix and compatible annotations. Their complete selected observable-unit projections must match under Core.17 - neither side may contain an extra unmatched selected unit - and any cutpoint use must carry the matching J.OfferReach/J.OfferConf/J.KObsConf package. Because the RTL witness is already supplied, the Ref -> Post clause does not depend on a generic fair-extension theorem. Appendix L can select such a source witness when latency-sensitive choices require RTL-aligned epsilon outcomes.

5. Normative Formal Core and Appendix G - shared semantics and observable contract

PURPOSE

Define the shared source-reference semantics and the observable scheduling contract that the post-HLS implementation must preserve.

Engineer takeaway: The Core is the normative semantic skeleton for G-K; Appendix G constrains that skeleton through R1-R5 and E1-E5. Neither layer by itself constructs the matching source execution - Appendices I and J do that work.

5.1 Source-side rules R1-R5

Rule	Role in the architecture
R1 - synchronization order	Synchronization calls of each process remain in dynamic source order and commit in strictly increasing cycles.
R2 - signal anchoring	For a sequential process, a signal Read is logically anchored to the closest preceding synchronization and a signal Write to the closest succeeding synchronization. This anchor also determines the signal event's formal ordering for frontier reasoning: lexical placement around intervening message operations does not by itself impose message order. If several source Writes by the same process to the same signal share one E2 anchor, the last executed source Write supplies the canonical visible value and earlier same-anchor Writes are epsilon-internal.

Rule	Role in the architecture
R2C - combinational fixed point	Combinational observations are defined from the cycle-entry valuation and the unique settled result of the qualified component/network. Internal epsilon/stutter re-evaluation is permitted so long as it converges to that unique result. R2C fixed-point units are matched by component identity and local emitted-occurrence ordinal rather than by a global cycle bucket; Sys_ref and RTL_B need not share an internal combinational model or absolute cycle numbering. ComponentParticipatesInCycle defines each side's observation domain. A cycle-locked component emits exactly one complete fixed-point unit in every participating cycle and none outside it; per-side participation/completeness and the implementation/certificate justification are part of the R2C conformance evidence.
R3 - sync isolation	A synchronization boundary separates the message operations in the immediately preceding and succeeding source intervals, and its Core.SyncFence completion clause requires every earlier blocking Push/Pop in pref_S(s) to process-facing commit no later than the synchronization commit. Same-cycle completion is permitted.
R4 - safe issue order	Dynamic message operations preserve prefix-closed source issue order. Core.9 Issue is the actual first process-owned request cycle, not a witness-selected annotation. For distinct blocking operations on the same scalar explicit boundary/direction, Core.IC(4) gives strict issue order; the ordinary same-interface special case is discharged from the fixed Core.10 process-owned BoundaryReq mapping, Core.IC live-owner/nonwithdrawal/back-to-back attribution, endpoint serialization, and the dynamic source sequence. If q0 commits at t and a continuing process-owned request at t+1 belongs to the next same-direction blocking q1, q1 owns it beginning at t+1 and Issue(q1)=t+1; the request may not be ownerless in between. Distinct-interface no-reverse and cross-interface prefix closure remain substantive implementation checks.
R5 - channel capacity semantics	The source reference uses rendezvous or finite-capacity channel semantics at the selected explicit abstract boundaries; elaboration may introduce proof-internal channels but does not reorder ordinary source calls.

DEFINITION: Core.RegSeq - registered sequential-interface semantics

For a sequential process, request and Push-data outputs in cycle t depend only on registered state and operands fixed before L2 settle. If ReturnedAtCycle(q,t) holds - for a success or failed non-blocking return - any dependent evaluation, control, endpoint/address, payload, or issue has a later Horizon. ReturnedAtCycle is defined normatively in Appendix C; Core.RegSeq states the dependent-use rule.

- **Why it matters.** It eliminates same-Horizon decision-to-dependent-request edges that would be inconsistent with clocked hardware.
- **Where it is used.** Appendix I local preparation, J.FoundationRank/J.DepRank, J.RegPhaseNF, and the Appendix K serial-reduction assumptions.
- **DV implication.** Checked source/tool evidence must establish the registered-cycle rule. Zero-time dependent NB cascades must be identified and then rejected, excluded, isolated, remodeled atomically, or aligned through Appendix L; silent scope exclusion is forbidden.

CLARIFICATION: Core.OrdFrontSingleton - ordinary-source frontier uniqueness

Within an ordinary non-elaborated sequential source interval, dynamic message calls remain ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and ordered unrolled instances. Therefore Front_P has at most one member. A genuine message-operation multi-frontier requires explicitly modeled incomparable Sys_B^elab frontier items with the corresponding order, boundary, projection, and attribution evidence.

5.2 Normative Core operational semantics and the cycle-level hardware model

The Normative Formal Core treats `Sys_ref` as an operational transition system with explicit process state, explicit abstract channel state, theorem-instance dynamic operations, pending requests, and epsilon micro-state. Core.8 defines a boundary commit as the selected proof model's legal E4 channel-commit transition; on `RTL_B`, boundary mapping and B-faithfulness/conformance relate that abstract event to the cycle in which the mapped valid/ready conditions and payload are observed. `Commit(q)` is separately the process-facing commit/accept/release occurrence designated as completing the source operation, and `CommitCycle(q)` is its cycle index; completion does not make returned data/status available for same-cycle dependent use, which remains an L4/Core.RegSeq fact. Core.9 now makes `Issue(q)` explicitly operational: it is the first cycle in which the calling process begins requesting `q` at its selected endpoint, and any witness/scheduler record merely carries that cycle rather than selecting a different timestamp. Core.10 makes `BoundaryReq` operation- and boundary-specific: it is the fixed-mapping process-owned request for the specific frontier item/dynamic operation, not a raw complementary ready/valid availability level or coupler signal. Core.IC is the exhaustive ordinary blocking-request lifecycle invariant. An asserted blocking `BoundaryReq` has exactly one live owner until process-facing commit or affecting RW2 RESET; if `q0` commits in cycle `t` and the same process-owned request continues in `t+1` as the next same-direction blocking operation `q1`, ownership transfers to `q1` at `t+1` and `Issue(q1)=t+1` - there is no ownerless request interval. Genuine scheduling delay remains available only before `q1`'s process-owned `BoundaryReq` begins, through `Core.SelectLatitude`. `Core.SyncFence` is source synchronization-completion legality. Core.IC and `Core.SyncFence` are constitutive `Sys_ref` semantics, while their mapped RTL behavior is named implementation-side conformance evidence. `Core.SelectLatitude` is the complementary finite-prefix selection rule: at L1 and L3, an otherwise legal action need not be selected maximally in that cycle, under either Policy L or Policy S. This nonselection is legal behavior rather than evidence of disablement/Core.16 activation; `Core.IssueFair` and B2 are separate complete-execution weak-fairness requirements and apply only under their continuous-enablement antecedents. For complete/fair source-continuation proofs, `Core.CycleState/Core.CycleResult` carry the full registered, environment, clock, reset-unit, and cycle-valued monitor state; `Core.CycleClosure C2` qualifies selected finite, not necessarily maximal, L1/L3 choices and their partial-cycle extension. `Core.RTLCycleCarrier` is a separate narrow RTL typing/completion lemma: once a reachable prefix has already entered a legal real cycle and contains a complete selected unit, the enclosing `CycleResult` carries that same execution through the already-entered cycle to its successor K-origin. It is not generic cross-cycle progress from an arbitrary K-origin. `Core.FairPick/Core.FairCycleDovetail` are the stronger constructive evidence form used only when Appendix K must build a complete fair continuation. The 31 August $q \rightarrow K$ extension adds a second, explicitly current-state layer on top of that operational model: `Core.InFlightStep/Core.InFlightDrainWF/Core.InFlightCycleExt` classify already-admitted progress, while Core.18 `FlightStateRec/InFlightStepRec` reconstruct the finite action/effect system from `KObs`. Core.7b-Core.7e type synchronization separately from message operations so `SyncReady` is only a state predicate, `SyncFire` is the real source transaction, and lowering-only `EpochGateAdvance` remains distinct. These objects support Appendix K's modal drain proof but do not alter the observable alphabet or literal message WFG.

Policy	Meaning and use
Core.PolicyL / Core.LLegal	Core.PolicyL is the permissive issue policy used by the safety witness and A5-L. Core.LLegal fixes exact step legality. Core.ReachPrep may allow a later dependency-independent operation to become reached before an earlier same-interval blocking call commits; reach is not Issue, source order remains, and <code>Post -> Ref</code> coverage must ensure <code>Sys_ref</code> is permissive enough for every preparation segment used by the accepted J construction. <code>Core.SelectLatitude</code> then permits an otherwise legal reached issue to remain unselected at a finite L1 opportunity; <code>QSync</code> does not remove either preparation or selection latitude.

Policy	Meaning and use
Core.Phase / Core.LAdm	Core.Phase defines L0-L4 and the pre-settle/settled-frontier boundary. Core.SelectLatitude applies to the selected L1 issue segment and L3 boundary-action set; they are finite and need not be maximal, and the L1 issue selection may be empty when no operational rule requires an action. Core.LAdm qualifies finite Policy-L prefixes and complete executions by adding the drain/settle-to-KCut discipline; SelectLatitude-permitted nonselection and any resulting legal CompleteCycle/SilentCycle remain Policy-L-legal. When K.2b.E constructs a complete fair continuation it additionally consumes Core.CycleClosure/Core.CycleChoiceExt and Core.IssueFair/B2/B3; a B6 suffix still requires Core.TerminalIdle.
Core.PolicyS	The conservative serial-blocking issue policy: a later same-interval blocking operation may issue only after every earlier blocking operation has process-facing committed in a strictly earlier cycle. Policy S inherits Core.SelectLatitude: the earlier legal operation need not issue at the first opportunity merely because it is legal. SDet later fixes the concrete selected Policy-S simulator/run used by the primary Appendix K route; Core.IssueFair/B2 govern eventual selection only where a complete fair execution is required.
Core.SelectLatitude	One no-maximal-progress rule shared by L1 issue selection and L3 boundary-action selection under both issue policies. It is finite-prefix operational legality, not fairness. It never licenses a Core.16-forbidden action, and absence of issue/commit does not by itself prove disablement or Core.16 activation.

DEFINITIONS: Core.Phase, Core.Settle, Core.KCut, Core.16, Core.Drain, and Core.LAdm

Core.Phase gives each source cycle a process-owned L1 preparation/issue window, L2 settling with requests and selected cycle-t inputs frozen, L3 boundary actions, and L4 registered result availability. Core.ReachPrep governs dependency-independent eager preparation within L1; reach and Issue remain distinct. Core.SelectLatitude makes the L1 and L3 selections finite but non-maximal: a legal Issue or boundary action may be deferred at a finite opportunity, subject to Core.16 and every existing legality/fixed-instance constraint, without invoking Core.IssueFair or B2. Definition Core.Settle names the first L2 fixed point if reached, and Core.QSync-Ref proves finite existence/uniqueness in the standard scope. Core.SettleIsKCut makes it a source KCut. Core.16 activates when the minimal frontier is fully disabled and forbids source-later launch broadly enough to include replenishing preparation/reach, not only message Issue. Core.Drain permits finite non-replenishing cleanup; its finiteness comes from Core.16(6)/K.Drain plus the stated progress/qualification evidence rather than capacity alone. Core.SettleKBridge connects a persisting activation to the later drain-closed KCut. For complete continuations, Core.CycleState names the cycle-entry theorem state and Core.CycleResult/Core.CycleClosure type the real-cycle result and closure; C2 expressly uses the selected finite, not necessarily maximal, L1 endpoint and does not force maximal L3 selection. Core.CycleChoiceExt extends selected phase-legal partial cycles. RTL_B has no source phase tags; Core.QSync-Post supplies its settled KCut/closure interface.

ARCHITECT MENTAL MODEL - one ordinary synchronous cycle

Cycle-level view	Formal correspondence
1. Registered state and selected cycle-t exogenous inputs are fixed.	QS1/QS2; ClockScope audit; L2/QSync frame.
2. Sequential process request/payload outputs are determined from registered state and operands already fixed for the cycle.	Core.RegSeq. Ordinary combinational logic from registers is allowed; a newly returned cycle-t result cannot feed a dependent same-cycle sequential output.
3. The complete system combinational/ready-	Delta-settling; R2C; QS3/QS4; Appendix Q for Sys_B^elab.

valid/guard/join/coupler network settles to one deterministic valuation.	Pure combinational loops are outside the standard scope; feedback crosses explicit state.
4. The settled boundary snapshot is read.	BoundaryEvalPoint; source SettlePoint; Core.KCut when the K-facing proof consumes the snapshot. These are proof views of the ordinary settled cycle state, not additional hardware states.
5. Boundary decisions and commits occur.	Core.Phase L3; E4; R3/E5; Core.SyncFence.
6. The next registered state/result availability is established.	Core.Phase L4 / next cycle. RTL_B has its own registered update rather than source L0-L4 tags.
Selection latitude (not a hardware stage)	Core.SelectLatitude. At the source proof level, the legal L1 issue segment and L3 boundary-action set may be non-maximal or empty when no other rule requires an action. A no-selected- Σ -commit result can be a real Core.SilentCycle whose registered/environment/monitor state still evolves. Core.IssueFair/B2 are the later fairness counterparts, not part of finite-prefix J legality.

The functional cycle picture is conditional on explicit choices, not a claim that Sys_ref has only one execution. WC-sensitive outcomes, Policy-L issue/scheduling choices, arbitration, and reactive-environment choices remain part of the admissible execution space. Likewise R2C/Core.QSync determinism means a unique settled result for a fixed cycle-entry valuation and fixed drives/choices; internal epsilon/stutter re-evaluation is allowed, and Sys_ref and RTL_B need not share the same internal combinational model or absolute cycle buckets.

5.3 Post-HLS compatibility obligations E1-E5

Obligation	What the RTL/tool flow must establish
E1	Preserve each process's synchronization-event order.
E2	Preserve anchored signal values/visibility for sequential processes and fixed-point signal units for combinational processes. For sequential processes, canonicalization records at most one Read and at most one Write per (process, signal) at each E2 anchor; when several source Writes share an anchor, the last executed source Write supplies the matched value and earlier same-anchor Writes are epsilon-internal.
E3	Preserve safe message Issue order and attribution. Core.9 fixes Issue as the operational first-request cycle and Core.10 fixes the process-owned BoundaryReq mapping. For the ordinary scalar same-interface blocking case, strict order is derived from qualified Core.IC persistence/back-to-back ownership, scalar endpoint serialization, and dynamic source order rather than chosen Issue annotations. Distinct-interface no-reverse ordering and the prefix-closed issued-set condition remain substantive E3 checks; Core.17/ Σ Match separately checks committed occurrence/value matching.
E4	Realize legal rendezvous or finite-capacity FIFO behavior at each selected explicit abstract channel boundary, including payload/order/occupancy behavior.
E5	Do not move a message operation across its immediate synchronization boundary: predecessor operations issue no later than the sync; successor operations issue strictly later; and every earlier blocking Push/Pop in pref_S(s) must process-facing commit no later than the sync. Same-cycle message/sync completion is allowed. On the post side, the "accounted before interval advance" convention is certificate/trace attribution, not a physical sub-cycle RTL ordering requirement.

BOUNDARY CONDITIONS: B-faithfulness/Core.ElabProj and mapped-memory Core.MemFaithful/Core.MemProfile

E4 only has meaning at a chosen abstract storage/rendezvous boundary. At an ordinary primitive Sys_B boundary, B-faithfulness checks selected transfers and B(c)/occ_c, and primitive E4 legality also determines the complementary handshake. In Sys_B^elab, operational storage/rendezvous channels use the explicit B_E(d)/occ_d state declared by E; Q.1(a)-(c) separately uses outer Sys_B B(c)/occ_c/E4 history as the refinement target. Core.ElabProj coordinates Π_{elab} , outer trace map $\pi_{\Sigma,E}$, finite token projection $\Pi_{token,E}$, signed A_E state accounting, the settled handshake network, and the process-dependency projection. At ε -quiescent cycle-boundary states A_E must agree with independently reconstructed outer history; its legal nonnegative range is then derived from outer E4 rather than certified as a separate obligation. Internal staged/bundle/join transfers with outer effects have a unique declared token attribution, while transfers with no outer effect may have none; one token may account for several outer Push/Pop effects only without invention, omission, duplication, or double counting. ReleaseOwnerSets_E/WaitOwners_E contracts stateless couplers to modeled processes. A stateless coupler may sit at a process-facing boundary but is not a process, residual offer, or stored channel state.

Mapped memory uses the same selected-boundary discipline but a separate qualification. Core.MemFaithful names the untransformed logical mapping-layer semantics as the reference and permits transformed physical memory traffic to remain below the selected boundary only when cache/merge/split/reorder behavior preserves the selected Core.17/E1-E5 events/order/values/atomicity. A selected message or anchored-signal event cannot be selectively hidden. J.Mem still requires complete lowering of qualifying cross-process memory dependencies. If transaction-changing behavior or persistent backpressure needs a reusable boundary above the physical stream, Core.MemProfile supplies the qualified logical interface; its K-facing boundary must faithfully represent capacity/backpressure/release dependencies or the transformation is outside the covered K instance.

5.4 Trace compatibility is observational, not temporal equivalence

The relation approximately-equal ($\approx$) is Core.17 Σ Match plus E1-E5. Σ Match explicitly matches the complete selected occurrence sequences and value labels - including per-channel Push/Pop ordinals, canonicalized sequential signal observations, R2C fixed-point units matched by component/local emitted-occurrence ordinal, synchronization occurrences, and any explicitly selected proof-internal RESET units carried by the observable-unit machinery (without adding RESET to Σ). Independent matched observations may occur in different cycles or be grouped differently where the ordering and atomicity rules permit. Core.SelectLatitude is the source-side operational reason a matching finite witness may defer otherwise legal issue/boundary actions without making that delay itself observable. Appendix J may insert only the finite invariant-preserving unit-empty cycle bridge required by selected cycle-sensitive evidence; J.HCanon does not equate the bridge length to the raw RTL Horizon gap. Appendix G expands this matching convention; it does not define a second competing trace relation.

Appendix G's explanatory causal-dependency graph is not a universal well-founded proof order. It may contain legal same-cycle atomic cycles. Appendix J instead generates its own construction relation: D1-D4 are semantic prerequisite edges covered by J.DepSound, while D5 is Horizon proof stratification and is justified separately by J.HorizonOrderSafe commutation evidence. Appendix K uses yet another relation under the authoritative K.CV premise: Definition K.CVPred/K.CV-WF supplies the well-founded fact-provenance certificate used by K.CV and must not inherit cycles merely because two operations are causally or temporally adjacent. K.CVPred/K.CV-WF define that certificate structure without assuming K.CV as a prior premise.

READING RULE: What to take from Appendix G

For architects: the Core fixes the operational model and G defines what implementation freedom remains invisible to a latency-insensitive environment. For DV: together they identify the observable checker boundary and conformance obligations. For formal implementers: the Core supplies the shared transition-system/KObs/WFG/drain vocabulary, while G supplies the R/E contract consumed by I and J.

6. Appendix I - per-process replay and preparation

PURPOSE

Show that each process can locally replay the selected RTL observable prefix and prepare for its next participation using a finite source-owned script.

Engineer takeaway: Appendix I is intentionally local. It identifies the right dynamic operation and reaches its request/decision point, but it does not assert that the complementary endpoint is present or that all process scripts can be merged globally.

6.1 Inputs and preconditions

Input	Why I needs it
Fixed initial state and stimulus	The process is replayed under the same global Sigma-causal environment behavior, not an arbitrary P-local stimulus.
WC witness	Every epsilon-modeled choice that can affect later observable control, values, identities, requests, or frontiers is fixed consistently.
I.A0 / E3/E5	The RTL message-issue discipline is supplied as an implementation assumption, including prefix-closed issue attribution and the E5/Core.SyncFence rule that no earlier blocking request survives a synchronization commit into the next interval.
Core.RegSeq	Dependent next-cycle requests cannot be created from same-cycle returned results.
B4 / finite pipeline depth	Internal micro-progress has a finite epsilon bound when the process is not externally stalled.
B2/B3 when progress is invoked	These are theorem-applicability fairness/progress assumptions, not consequences of source code and not generically “environment assumptions.” B2 requires eventual selection only for a boundary action that remains continuously ChannelEnabled; B3 supplies complementary channel discharge under its persistent-request antecedent. Their discharge owner may be source/scheduler semantics, HLS-generated or hand-written RTL arbitration/back-pressure, channel-library behavior, an external master/environment, or a qualified reusable class bound to the concrete theorem instance. Intermittent enablement does not satisfy the weak-fairness antecedent by design.
Source well-formedness	Signal anchors, dynamic operation identities, direct-input contracts, shared-memory lowering, and reset behavior are well formed.

6.2 The replay checkpoint: I.ReplayObs, I.CutpointOfferObs, I.ReplayObs-Congruence, I.DR, and I.DR'

I.ReplayObs is the logical P-local replay checkpoint determined by the committed observable-unit history, fixed stimulus, and witness. It contains the logical replay control position, replay-relevant values, witness-selected outcomes, replay-fixed dynamic attribution, completed-item status, candidate NextFront_P, anchored values, returned results, reset epoch, and normal-completion status. It is not the complete raw operational state: certified outcome-independent L1 preparation may advance the operational cursor or compute temporaries without changing the logical checkpoint.

I.CutpointOfferObs is the separate operational slice for current uncommitted blocking offers: Pending_P, Front_P, BoundaryReq, owning dynamic calls, boundary/direction/epoch facts, and stable Push payload obligations. The K-facing cutpoint therefore combines replay history H with residual offers O. Deterministic replay does not manufacture O from H; Appendix J constructs and audits O through J.OfferReach and J.OfferConf.

LEMMA: I.ReplayObs-Congruence

From equal logical replay checkpoints, the same next complete observable unit produces equal canonical immediate post-unit checkpoints. An exact selected finite normalization may then mix replay-inert preparation segments, which leave each checkpoint unchanged, with witness-fixed epsilon transitions that may update replay fields. The normalization convention must pair those replay-changing transitions so that equal checkpoints remain equal; the theorem does not assume per-path neutrality, global confluence, or uniqueness of all legal raw endpoints.

LEMMA: I.DR and I.DR' - deterministic replay

I.DR' establishes equality of the logical ReplayObs checkpoint at the canonical immediate post-unit state, before optional following L1 preparation or epsilon-normalization. I.DR establishes equality at the selected epsilon-quiescent cutpoints by applying paired congruence to the exact proof-relevant normalization paths. Appendix J uses the immediate form when constructing the next Horizon and records normalized witnesses explicitly when a quiescent cutpoint is required.

6.3 Deferred non-blocking decisions

A process-local proof cannot decide a non-blocking poll from process state alone because success or failure depends on the common incident-channel snapshot and possible complementary requests. Stopping the local replay at every poll would be too restrictive: a registered process may execute independent same-Horizon work that does not depend on the poll result.

DEFINITION: I.DeferredNBHole

A deferred hole records the dynamic non-blocking call, explicit boundary, snapshot key, Horizon, result destinations, reset epoch, witness-selected expected outcome, and NoDependentUse evidence. The process-local preparation script can carry the unresolved hole through independent work while forbidding any dependent use of its result.

6.4 The local preparation theorem

LEMMA: I.3 - finite process-local preparation script

Given the replay state, exact dynamic operation identity, Core.RegSeq, WC, E3/I.A0, source well-formedness, and typed footprints, I.3 constructs a finite process-owned epsilon/request path to the selected operation, its persistent issued request, or a carried deferred non-blocking decision hole. The script does not commit an unmatched observable unit and does not assume global enablement.

- **Output.** The information later packaged in J.PCert: the logical ReplayObs checkpoint, exact dynamic identity, finite per-Horizon preparation actions, separate operational issue/pending attribution, stable payload/value facts, field-sensitive footprints, and deferred-hole records.
- **Non-result.** No complementary endpoint, common channel snapshot, globally legal merge, or reachable system extension is proved here.
- **Why finite.** The source-control segment is finite and internal progress is bounded by B4/FPD when not externally stalled.

6.5 Engineer example: independent work after a poll

Suppose a process performs a PopNB, updates an independent diagnostic counter, prepares a request on another endpoint whose control and payload do not depend on the PopNB result, and only later branches on the poll status. Appendix I may carry the poll as a DeferredNBHole through the independent counter/request work. Appendix J later freezes the complete channel request snapshot at L2, decides the poll once globally, and prevents the branch from occurring until the result is available under Core.RegSeq.

ARCHITECTURAL SEAM: Why Appendix I cannot be the whole safety proof

Local scripts may each be valid when viewed against read-only summaries, yet still conflict through shared channel state, atomic rendezvous pairing, a common non-blocking snapshot, reset scope, or overlapping field accesses. Appendix J proves that the scripts coexist and derives a legal global order; it does not merely conjoin them.

7. Appendix J - local-to-global safety construction

PURPOSE

Construct one reachable `Sys_ref` execution from local process/channel/atomic certificates and prove observable compatibility with a selected RTL prefix.

Engineer takeaway: Appendix J is the main safety theorem. Its defining achievement is that global witness realizability is derived from local evidence rather than accepted as an unexplained system-level premise.

7.1 Theorem J.1: the top-level safety result

THEOREM: J.1 - execution-level / trace-set compositional compatibility

For every covered reachable RTL_B prefix, the normal compositional route uses Definition J.LocalGWR, QualifiedSynchronousModel_Sys_ref(I)/Core.QSync-Ref, Core.SelectLatitude, and Theorem J.GWR-Comp to construct a reachable Sys_ref prefix and a global witness package whose annotated projections satisfy E1-E5. The construction may use finite invariant-preserving unit-empty source cycles between nonempty Horizons when selected cycle-sensitive evidence requires a later source entry; this is source operational legality and not B2/B3 fairness. A directly validated J.GWR package is an alternate route and may carry equivalent checked per-Horizon L2-settling/bridge evidence instead of the global source QSync premise. Plain finite-prefix J.1 does not require post-side QSync unless a post KCut/NormPost interface is invoked. The reverse Ref -> Post clause applies only to source prefixes already equipped with an RTL-consistent annotation/witness package: that premise already selects `tau_post*`; theorem-wide B-faithful/E4 facts supply Proposition J.0's channel premises; J.0 performs the conditional composition; and the existential post witness is `tau_post := tau_post*`.

Four details are important for engineering use:

- **Finite reachability, not only fair executions.** The generic Appendix I/J domains are `Reach_post` and `Reach_ref` finite reachable prefixes. Theorem J.1 does not require generic `Exec_post/Exec_ref` objects or a J.FE fair-extension lemma. Finite postponement of an otherwise legal L1 issue or L3 boundary action is ordinary `Core.SelectLatitude` legality; `Core.IssueFair` and B2/B3 are not imported into this finite-prefix construction. Complete/fair source continuation is introduced only by later liveness machinery such as `Core.LAdm/Exec_L^adm` and K.2b.E.
- **Required-prefix induction.** The proof orders observable units by the constraints that must be preserved, rather than by raw RTL cycle buckets or full source order across independent endpoints.
- **Annotated carrier.** Compatibility includes dynamic identities, witness choices, Issue/Commit attribution, request facts, and channel histories. It is stronger than matching an unannotated list of labels, but weaker than full-state equivalence.
 - **Source-side settling and gap route.** On the normal J.LocalGWR -> J.GWR-Comp route, QualifiedSynchronousModel_Sys_ref(I) and Core.QSync-Ref supply the finite L2 SettlePoint needed by J.RegPhaseNF/J.UnitExt for every constructed nonempty Horizon. When the selected source witness, environment/reset contract, or other cycle-sensitive evidence requires a later source cycle-entry state before the next nonempty bucket, Core.SelectLatitude permits a finite invariant-preserving unit-empty CompleteCycle bridge. A directly validated J.GWR package may instead carry independently checked finite L2-settled segments and equivalent bridge evidence. Plain finite-prefix J.1 does not thereby require post-side QSync unless a post KCut/NormPost interface is also used.

7.2 The local certificate package

Component	What it certifies
J.PCert	Per-process logical replay checkpoint plus separate finite preparation/issue evidence, exact dynamic identity, field-sensitive footprints, and deferred non-blocking holes produced from Appendix I.

Component	What it certifies
J.CCert	Per-explicit-channel boundary identity, capacity, E4 history, occupancy/head, payload, request snapshot, enablement, reset, and B-faithfulness evidence.
J.ACert	Agreement for explicitly atomic units: rendezvous pairs, paired synchronization, E2/R2C groups, RESET, and other named multi-participant actions.
J.EnvCert / J.ResetCert	Environment and reset-unit identity, scope, action, and locality evidence.
Typed footprints	Field-sensitive ReadSet/WriteSet evidence for every construction action, including component-local elaboration actions.
LocalAgree	Equality, uniqueness, ownership, coverage, and duplicate-field agreement across local records.
Generated dependencies	Locally justified Dep_J edges and enough evidence to prove that omitted pairs commute; the package does not supply a convenient global rank.

DEFINITION: J.LocalGWR

The complete local package for one RTL prefix: PCert for every process, CCert for every explicit boundary, ACert for every atomic unit, applicable environment/reset records, complete action/deferred-hole inventory, field-sensitive footprints, Core.RegSeq and source conformance, generated dependencies, LocalAgree, and coverage. It must not assume that these records already form a realizable global replay.

7.3 How J derives a global construction order

J converts the certificate package into a finite graph of primitive construction actions. D1-D4 encode actual semantic prerequisites from process control/data/issue, shared snapshots, channel state, resets, environment, and explicit component rules; J.DepSound applies only to those semantic edges. D5 may additionally order cross-Horizon actions for proof stratification. A D5-only pair must carry J.HorizonOrderSafe evidence that the actions commute without changing legality, complete observable units, terminal post-settle semantic state, or the J.HCanon quotient.

J.FoundationRank/J.DepRank then prove acyclicity without assuming the order that the construction is meant to derive.

Definition / lemma	Role
J.DepJ	Generated dependency relation with two roles: D1-D4 are semantic prerequisites (process control/data/issue/ReturnedAtCycle, request/snapshot, channel/FIFO, reset/environment/elaboration); D5 is Horizon proof stratification and is not automatically an operational prerequisite.
J.DepSound / J.HorizonOrderSafe	J.DepSound proves only D1-D4 semantic edges are real operational prerequisites. For a D5-only edge with no semantic path, J.HorizonOrderSafe requires a commutation certificate showing the actions may be exchanged without changing legality, complete observable units, terminal post-settle semantic state, or J.HCanon.
J.DepComplete	If two non-atomic actions are left unordered, they either commute by field-sensitive framing or are the explicitly proved same-cycle buffered Push/Pop dual update. D5 may order already-commuting cross-Horizon pairs but cannot hide an actual semantic interference.
J.FoundationRank	A graph-independent lexicographic semantic rank based on reset epoch, Horizon, semantic stage, local ordinal, and component tie data.
J.DepRank	Every generated edge strictly advances the foundation rank.
J.DepAcyclic	A directed cycle would strictly increase the well-founded rank back to its start, which is impossible.

Definition / lemma	Role
J.CanonicalRank	A derived deterministic totalization of the acyclic prerequisite relation. Stable tie keys order only commuting actions.

LOAD-BEARING RESULT: J.DepComplete + J.DepRank

The flow cannot simply provide a global rank that is assumed correct. It must generate every non-commuting prerequisite from local rules, prove that unordered actions really commute, and prove each edge advances an independent semantic foundation rank. This is what makes the construction auditable and prevents a hidden global replay assumption.

7.4 Registered phase normalization

Within each Horizon, the proof must respect actual clocked interface semantics. It cannot decide a non-blocking result and then create a dependent request in the same cycle, and it cannot evaluate different polls against inconsistent partial channel snapshots.

LEMMA: J.RegPhaseNF - registered phase normal form

Every finite same-Horizon construction fragment can be permuted, without changing its canonical observable history, into: L1 process evaluation and request/attempt writes; one finite L2 settle producing the common snapshot; and L3 non-blocking decisions plus complete boundary commits. The lemma needs the local fact that the frozen L2 state admits a finite $\rightarrow_{\varepsilon, L2}$ path to a SettlePoint. On the normal compositional route Core.QSync-Ref supplies that fact uniformly; a direct J.GWR route may carry the equivalent checked per-Horizon evidence. Registered results become available only in L4/next cycle.

LEMMA: J.NBDecision

Once all L1 predecessors have executed and J.CCcert supplies the common settled snapshot, each deferred non-blocking call has exactly one legal source result. A failed poll cannot coexist with a matching enabled rendezvous request, and a successful poll cannot be invented in a full/empty state that forbids it.

7.5 From one Horizon to the complete source witness

Lemma / theorem	Function in the construction
J.Frame	A certified transition changes only its WriteSet; disjoint state and guards are preserved.
J.LocalLift	Embeds one Appendix I process-local script into a reachable global state when owned/shared locations match its certified summaries.
J.PrepareCommute	Moves independent preparation actions into the common L1 window while respecting snapshots and true dependencies.
J.UnitEnable	After L1/L2, the requests, payloads, channel state, atomic participants, and holes make each selected L3 unit genuinely enabled.

Lemma / theorem	Function in the construction
J.UnitExt	Before the next nonempty Horizon bucket, takes any finite evidence-determined invariant-preserving unit-empty CompleteCycle/SilentCycle bridge required by the selected source/environment/reset timing evidence under Core.SelectLatitude; then executes the bucket, updates replay/channel/atomic state, resolves holes, and preserves the construction invariant. Bridge cycles are not synthetic rank buckets and use no B2/B3 fairness.
J.GWR-Comp	Outer induction over nonempty Horizon buckets constructs the complete reachable Sys_ref witness chain and stable J.GWR interface, preserving the induction invariant across any Core.SelectLatitude-backed unit-empty bridge. The rank/order machinery remains defined only on actual construction actions.

J intentionally ranks only construction actions in nonempty Horizon buckets. An RTL cycle/Horizon with no construction action is not converted into a synthetic J.CanonicalRank element: Height, J.FoundationRank, J.DepRank, and J.DepAcyclic continue to range over Act_L(τ_{post}). Before a later nonempty bucket, J.UnitExt may instead take the finite evidence-determined sequence of unit-empty CompleteCycle transitions needed to reach the required source cycle-entry state. A bridge transition with no selected Σ commit is a Core.SilentCycle.

THEOREM: J.GWR-Comp - local certificates construct global witness realizability

From J.LocalGWR plus QualifiedSynchronousModel_Sys_ref(I), Core.QSync-Ref supplies a finite source L2 SettlePoint for every reachable constructed nonempty Horizon. J.GWR-Comp validates the dependency graph and construction order, phase-normalizes each nonempty Horizon, and applies Horizon-batched J.UnitExt. Where the selected source witness, environment, reset-state contract, or other cycle-sensitive evidence requires a later source cycle-entry state, J.UnitExt first uses Core.SelectLatitude to take the finite evidence-determined invariant-preserving unit-empty CompleteCycle bridge needed to reach that entry. A no- Σ -commit member is Core.SilentCycle, but the proof does not assume its full state is unchanged: the typed cycle evolution and existing certificates preserve or re-establish the replay/channel/request/witness/reset/environment invariant. Empty RTL Horizons never become J.CanonicalRank buckets, so Height/FR/DepRank/DepAcyclic remain over Act_L(τ_{post}), and bridge length is not inferred merely from the raw RTL Horizon-index gap.

Local certificates

↓

Generated dependency graph

↓

J.DepRank / J.DepAcyclic

↓

Validated order with finite-ideal prefixes

↓

Core.SelectLatitude gap bridge where required + Core.QSync-Ref + J.RegPhaseNF + Horizon-batched J.UnitExt

↓

J.GWR + J.RegPhaseReach

↓

Proposition J.0

↓

Theorem J.1

Non-circularity. LocalGWR supplies no global order, no phase-normal replay, no assumed global extension, and no fairness premise. Core.SelectLatitude is constitutive source legality rather than B2/Core.IssueFair.

Stable interface. J.GWR can also be supplied directly by a translation validator or symbolic replay tool; that direct route must independently validate the same reachability, ordering, ideal-prefix, finite per-Horizon L2-settling, and

any required invariant-preserving bridge interface.

Reset handling. RESET is an atomic observable unit; RW1-RW5 and Lemma J.RS splice a new matched reset epoch while framing unaffected components.

Finite-ideal invariant. Every flattened prefix of the validated order is down-closed: if it contains a unit, it contains all required predecessors. Unit-empty bridge cycles do not change that ideal because they add no selected J unit. Tie ordering among independent units is bookkeeping; required predecessors are not skipped and atomic units are never split.

The bridge is not assumed to leave the whole source state unchanged. Because a SilentCycle is a real cycle, registered, environment, reset, request/witness, or cycle-valued monitor state may evolve. The J proof therefore requires the typed cycle transition plus the existing local/channel/environment/reset/LocalAgree evidence to preserve or re-establish its induction invariant; the required-prefix ideal and CanonHist remain unchanged because no selected J unit is added. Bridge length is not the numerical RTL Horizon gap unless the selected cycle-sensitive evidence itself requires that alignment.

7.6 Proposition J.0 is deliberately separate

PROPOSITION: J.0 - pairwise composition lemma

Given an already selected compatible source/RTL trace pair, compatible per-process annotated projections, and per-channel E4 well-formedness, J.0 lifts E1-E5 to the system level. It does not construct the source or RTL trace, prove B-faithfulness, or establish J.GWR. On J.1's normal Post $\rightarrow$ Ref route, J.GWR-Comp constructs the reachable source/RTL pair and Theorem J.1 instantiates J.0 with that constructed pair. In the restricted Ref $\rightarrow$ Post clause, RTLConsistentRefPrefix instead supplies the already-selected pair and J.0 is the non-circular composition step.

7.7 Safety transfer for the verification flow

DEFINITION: J.TraceCertCov + COROLLARY: J.1-Safe

J.TraceCertCov_I(D) is the named Post $\rightarrow$ Ref safety-certificate coverage premise. For every RTL prefix in D it records either the normal J.LocalGWR/J.GWR-Comp package with source Core.QSync-Ref/Core.SelectLatitude or a directly validated J.GWR package with equivalent checked per-Horizon settling/bridge evidence, plus the remaining Post $\rightarrow$ Ref premises. Corollary J.1-Safe consumes that route-sensitive coverage for a J.HCanon-invariant predicate satisfying J.IdealPrefixClosed. If one complete pre-HLS run is used as representative, J.RefProjCover must cover every relevant reachable source observation as a finite canonical ideal/down-closed prefix of that run.

Core.SelectLatitude enlarges admissible source latency realizations; pure unit-empty delay that leaves CanonHist unchanged is already absorbed, but if legal selection delay can change the canonical Σ_{DUT} history then the one-run RefProjCover discharge needs a separate latency-invariance/coverage argument. For $D = \text{CycleReach_post}(I)$, the standard QSync route derives package coverage from J.QSyncTraceInd. These safety-side coverage predicates do not imply K.CertCov.

QSYNC SAFETY PROFILE: J.CycleReach_post / J.QSyncTraceInd / J.CycleIdealCover / J.1-Safe-QSync

J.CycleReach_post is the exact reachable RTL-prefix domain ending at K-origins, before same-cycle QSync settling. J.QSyncTraceState retains only the exact-prefix LocalGWR/J.GWR-Comp witness and J.HCanon needed by safety. J.QSyncTraceInd proves a base case and one-cycle preservation for every reachable successor (not just one observed path); it explicitly does not consume J.OfferReach, J.OfferConf, J.KObsConf, K.DrainReach, or K.RLAdmReach. J.TraceCertCov-QSyncCycle and J.1-Safe-QSyncCycle establish cycle-aligned package coverage/safety. Core.RTLCycleCarrier supplies the execution-carrier fact for an already-entered RTL real cycle; J.CycleIdealCover then proves every reachable finite outer canonical history is a finite ideal of a reachable cycle-aligned history, and J.1-Safe-QSync uses J.IdealPrefixClosed to transfer safety to every reachable finite RTL prefix without asserting a separate J.TraceCertCov package for each intra-cycle ideal. If the richer K-facing J.QSyncCertInd is already established, J.QSyncCertInd-Trace supplies the safety induction automatically. Cross-interface scoreboards that fail J.IdealPrefixClosed still require direct arbitrary-prefix J.TraceCertCov coverage.

7.8 The J-to-K cutpoint bridge

Trace matching alone is insufficient for deadlock reasoning. Core.18 defines the cross-model K-facing correspondence as equality of one common typed Core.18c record $\langle E, w, H, O \rangle$, not as a list of independently asserted microstate clauses. J reconstructs the source process/channel/message-frontier and release-support/WFG state from that record. Under the generalized drain profiles, it also reconstructs the current synchronization carrier and the finite Q.2 in-flight action/effect relation through J.KObs-SyncCarrier, J.KObs-SyncInFlight, and J.KObs-InFlight. The architecture then deliberately stops: equal KObs/current local applicability does not prove a future CycleResult or multi-cycle continuation. J.InFlightPathSound is the separate level-3 theorem that realizes each finite reconstructed path under the bound Core.InFlightCycleExt/CycleClosure-equivalent profile. Appendix K uses this split to define generalized DrainClosedNow without expanding KObs into a future-behavior certificate.

Bridge element	Purpose
J.HCanon	Proves that the selected source and RTL prefixes have the same canonical replay history H modulo the permitted epsilon/history quotient.
J.RegPhaseReach	Records that the exact source witness was constructed with Core.RegSeq and the Core.Phase L1/L2/L3 discipline.
J.OfferReach	Shows that $\langle E, w, H, O \rangle$ is actually realized by a reachable source KCut selected by the exact NormRef witness, not merely a well-typed tuple.
J.OfferConf	At the selected RTL KCut, bidirectionally and uniquely matches every residual offer in O to a relevant asserted RTL request and every relevant RTL request back to exactly one offer. Core.SyncFence is load-bearing for identifying unqualified residual offers with the current-interval Pending_P set.
J.KObsConf	Binds one exact source/RTL package with explicit NormRef/NormPost KCut endpoints, common I/E/w/H/O, complete request attribution, and boundary/reset/epsilon-opacity evidence. It is the downstream theorem obligation that establishes the Core.18 cross-model equality; the Core itself fixes only the common type/equality notion.
Corollary J.1	Defines KObsPost_J, proves its equality with the realized source KObs, and exports the current-state reconstruction facts. Message-frontier/WFG reconstruction remains J.KObs-Proc/Chan/Offer/WFG; synchronization and generalized drain additionally use J.KObs-SyncCarrier/J.KObs-SyncInFlight/J.KObs-InFlight. The corollary does not itself assert K.Dead or future reconstructed-path realizability.

Bridge element	Purpose
J.KObs-SyncCarrier / J.KObs-SyncInFlight / J.KObs-InFlight	Reconstruct the typed current synchronization carrier, local SyncFire action/effect, and the complete selected Sys_ref FlightStateRec/InFlightStepRec action relation from the same KObs. No message offer is synthesized for synchronization and no future path opportunity is inferred.
J.InFlightPathSound	Model-indexed finite-path realization theorem. Given Q.2 action/effect soundness and Core.InFlightCycleExt or sufficient CycleClosure-equivalent evidence, every reconstructed finite path is embedded in a legal source-proof-model operational extension with exact successors and prefix-compositional profile closure. This is separate from KObs equality and from B2/B3 fairness.

AUDIT CRITICAL: J.OfferConf request-to-offer completeness

Every relevant asserted RTL boundary request must be attributed to exactly one source residual offer at the selected explicit boundary. Omitting a hidden asserted request, duplicating attribution, or using an unrelated source cutpoint invalidates the KObs bridge. This is the implementation abstraction boundary on which Appendix K relies.

8. Appendix K - conditional deadlock preservation

PURPOSE	Preserve absence of reachable generalized-drain-closed, release-closed internal message-passing wait cycles from the selected source-reference instance to covered RTL KCuts. Engineer takeaway: K is not a generic progress theorem. Literal Dead_K remains message-rooted, but “drain-closed” is now a finite-reachability statement over a qualified reconstructed in-flight transition system rather than an offer-only snapshot test. K.InFlightQual supplies the action/effect/path qualification; profile G2 additionally uses K.Low.ModalQual to relate lowered Sys_K paths to source Sys_ref paths without widening MsgOf/WFG. Universal RTL claims still consume Core.KCutClosure_post and K.CertCov, with the standard QSync route deriving the canonical domain and package totality.
----------------	--

8.1 The deadlock predicate

Appendix K evaluates Core.KCut states, not arbitrary epsilon-quiescent states. A KCut is reachable, epsilon-quiescent for the relevant proof model, and settled at every explicit boundary used by BoundaryReq/ChannelEnabled/ChannelDisabled. Source L2 SettlePoints are KCuts by Core.SettleIsKCut. In the standard RTL K-facing QSync scope, QS2a requires every reachable post KCut to identify its unique same-cycle registered/input frame and designated reachable K-origin, with only qualified settling between origin and KCut; Core.QSync-KCutCanonical therefore proves every reachable KCut is exactly the unique Core.QSync-Post endpoint of that origin. Core.KCutClosure_post remains the downstream semantic interface. Appendix K's deadlock observation boundary includes every WFG-relevant explicit abstract message-passing boundary, while RTL micro-staging already absorbed into B(c) remains internal. Environment-facing channels and SyncChannel barriers are handled through separate scope/assumption rules. Core.WFG is the conservative process graph obtained from WaitOwners_E, but Definition K.Dead is release-aware: it requires exact family-level release closure using ReleaseOwnerSets_E together with induced-WFG connectivity and drain closure. Lemma K.ReleaseSCCReduction shows that the connectivity restriction is without loss of generality for internally supported release-closed sets when every member has a nonempty internal release family. Appendix-K A9 and A10 remain instance assumptions referring back to the Core scalar endpoint-cardinality and reset-empty channel conventions. Pure synchronization remains outside the literal message WFG/Dead_K even though Stage 2 gives it a typed SyncCarrier and in-flight action semantics for lowering correspondence. Generalized drain is evaluated per applicable proof model $M \in \{\text{Sys_ref}, \text{Sys_K}\}$ under its bound action/path schema.

Object	Meaning
Front_P	The minimal pending blocking message-operation items of process P whose endpoint requests are currently asserted. Merely being the next source call is not enough; the request must have issued. Core.OrdFrontSingleton proves $ \text{Front_P} \leq 1$ in an ordinary non-elaborated source interval; multiple members require explicit Sys_B^elab partial-order structure.
Blocked process	Front_P is nonempty and every frontier item is ChannelDisabled at the selected explicit boundary. A disabled edge for one item does not by itself make a multi-frontier process blocked.
WFG edge P -> Q	For each disabled frontier item x of P, Core.WFG contains $P \rightarrow Q$ for every internal modeled process Q in WaitOwners_E(x, sigma), the union of x's minimal ReleaseOwnerSets_E supports. This graph is intentionally conservative: one support may require several processes jointly, while different supports are alternatives. Multi-frontier processes may also have outgoing edges from several disabled items. Q.1(j) makes the settled handshake and state-instantiated family reconstructible from E plus the reconstructed current roots, and J.KObsConf separately checks RTL faithfulness to that declared schema. Q.1(k) requires family-level support soundness (no spurious rescue) and completeness (no missed rescue) relative to the qualified template before WaitOwners is formed. The blocked-process condition remains an all-items-disabled test; exact deadlock closure is checked against the unflattened release families.
DrainClosedNow(D)	Generalized form: no finite nonempty qualified Core.InFlightStepRec path from the current FlightStateRec changes WaitRoots_D. D scopes the observed process-root family only; it does not filter action ownership or causal scope. DrainCandidates_D is retained only as an ordinary primitive diagnostic/fast path, with K.PrimitiveDrainExact giving the restricted equivalence case.
Definition K.Dead / K.DeadW / K.ReleaseSCCReduction	K.Dead is the single authoritative reachable drain-closed internal wait-cycle predicate. A candidate component is nonempty, every member is blocked, every internal disabled frontier release support intersects the component (exact family-level release closure), and the induced coarse WFG is strongly connected and contains a genuine internal process-dependency cycle. For $ D =1$, this requires an actual $P \rightarrow P$ dependency; the vacuous one-vertex SCC convention is not enough. Dead_K^W adds the declared KObs-closed waiver exclusion. K.ReleaseSCCReduction shows that connectivity can be reduced to a terminal SCC without loss when the release-closed set has a nonempty internal release family at each member; in the singleton terminal-SCC case its within-set outgoing-edge argument supplies the required genuine self-loop. Optional Dead_KS remains diagnostic only; later assumptions do not redefine K.Dead.

Object	Meaning
WaitRoots_D	Process-indexed $P \mapsto \text{WaitRoot_P}$ family containing each P frontier, its enablement/disablement facts, and exact $\text{ReleaseOwnerSetsRec_E}$ data. Process indexing makes restriction to $T \subseteq D$ definitional, enabling $K.\text{InFlightEscapeMono}/K.\text{DrainClosedHeredity}$ and the repaired $K.\text{ReleaseSCCReduction}$.
FlightStateRec / InFlightStepRec / InFlightEscape	Finite current-state reconstruction used by generalized drain. The step relation contains ordinary already-issued commits and qualified proof-internal staged/bundle/join actions; the synchronization profile also admits ready SyncFire and, on lowered Sys_K , explicit lowering-admin actions such as EpochGateAdvance . Multi-step paths matter: the two-stage non-fall-through artifact has no immediate root change on its first action but still provides a later escape.

SCOPE: Reachable closed internal wait-cycle cutpoint

The operative predicate is stronger than “the system is currently slow” and different from a temporal permanent-deadlock definition. It identifies a reachable message-rooted internal wait cycle whose exact release alternatives are trapped and for which the qualified finite already-admitted transition system has no escape that changes the candidate's WaitRoots . The directed WFG remains a conservative WaitOwners union; $K.\text{Dead}$ keeps the exact $\text{ReleaseOwnerSets_E}$ family. Synchronization carriers, NB-aligned guard causes, and lowering administration may participate in the proof of drain/lowering correspondence, but they do not become literal message WFG roots. Environment-only rescue, barrier mismatch, starvation, and other excluded stalls retain their separate scope rules.

8.2 Reconstructing the wait state and generalized in-flight relation from J.KObs

The 31 August $q \rightarrow K$ revision makes the drain argument explicitly modal while keeping KObs compact. From equal KObs, J reconstructs the current message-root state and the current finite proof-model transition system. $\text{Core.FlightStateRec}$ contains only reconstructible current state; $\text{Core.InFlightStepRec}$ gives one complete already-admitted action/effect step. Neither object contains historical Issue times, activation certificates, or a promise that a locally plausible action will be selectable in a future cycle.

K then defines WaitRoots_D as a process-indexed family and InFlightEscape as existence of a finite nonempty reconstructed path that changes that family. DrainClosedNowRec is the negation. D appears only in the root comparison: the step relation is not filtered by candidate ownership, so an outside-D staging, join, message, or synchronization action is explored if its finite consequences can change a D root. $K.\text{InFlightQual}$ is what permits that finite reconstructed graph to stand for real operational progress: it binds exact Q.2 action/effect soundness, finite action choice and a decreasing in-flight measure, $\text{Core.InFlightCycleExt}/\text{CycleClosure}$ -equivalent opportunity, $J.\text{InFlightPathSound}$, and the declared precision/search class.

Two proof levels should therefore not be conflated. $\text{Core.Drain}/K.\text{DrainEvidence}/\text{Core.SettleKBridge}$ qualify the actual reached source prefix and its no-replenishment behavior. Generalized DrainClosedNow asks what finite already-admitted progress remains possible from the current K-facing state. The first does not manufacture future cycles; the second is usable operationally only after the separate $\text{InFlightQual}/\text{path-soundness}$ discharge.

Result	Function
J.KObs-Proc / Chan / Offers / WFG	Appendix J reconstructs process replay, explicit channel state, residual message offers, frontier/request roots, settled handshake, $\text{ReleaseOwnerSets_E}/\text{WaitOwners_E}$, $\text{ChannelEnabled}/\text{ChannelDisabled}$, and Core.WFG from the common KObs record. These are current message-root facts; generalized in-flight reconstruction is a separate J.KObs-InFlight layer.
K.EnvTerminalAdequate / K.KObsDerived	Appendix K first supplies the environment-terminal adequacy premise and gathers the K-facing derived functions needed before deadlock factoring.

Result	Function
K.KObs-Dead / K.KObs-Ext	K-owned factoring and equal-KObs transfer of release-aware deadlock/waiver predicates using the same reconstructed FlightStateRec/InFlightStepRec/WaitRoots relation. The generalized action/path schema identity is bound separately through K.InFlightQual/K.CertHdr; equal KObs does not manufacture that operational evidence.
K.0 / K.DeadChar	K.0 reconstructs the operational minimal pending blocking frontier from KObs-derived inputs. K.DeadChar is the source-side/lowered-KCut channel-disabled characterization on the domain where K.Dead is defined; its buffered/rendezvous cases cite Core.12/Core.13 directly rather than asserting an RTL microstate theorem.
K.2 / K.2a	Use equal reconstructed state to prove extensionality of the exact release/WFG data and generalized drain representation. K.2a compares the same FlightStateRec/InFlightStepRec/WaitRoots reachability structure, not a legacy offer-only selector.
J.KObs-SyncCarrier / SyncInFlight / KObs-InFlight / J.InFlightPathSound	Current-state reconstruction and future-path realization are separate. The KObs lemmas reconstruct the typed synchronization carrier and selected Q.2 action relation. J.InFlightPathSound then proves finite reconstructed paths operationally real under the model-specific continuation/profile evidence.
K.WaitRoots / K.InFlightEscape / K.DrainClosedHeredity / K.InFlightRelocate	WaitRoots is process-indexed; an escape is any finite qualified path changing that family. Subset projection yields escape monotonicity and drain-closure heredity, which repairs ReleaseSCCReduction. If a candidate is not drain-closed, InFlightRelocate realizes an escape, stops at the first K-facing root change, and redispaches all candidate sets; it assumes neither SCC preservation nor monotone candidate identity.
K.InFlightQual / Q.2 / Core.InFlightCycleExt	Qualification layer that makes generalized drain sound. Q.2 supplies exact current-state action/effect schemas and finite action choice; Core.InFlightDrainWF supplies the finite decreasing admitted-work measure; Core.InFlightCycleExt or sufficient CycleClosure-equivalent evidence supplies real action opportunities; J.InFlightPathSound composes exact successors. Spurious actions/effects/opportunities are the dangerous direction because they can fabricate escape and hide a real Dead_K.
K.Low.ModalQual / InFlightPathCorr / DeadRootBridge	Profile-G2 lowering stack. LowAdminNormalize retires finite lowering-only guard/epoch residue; LowWaitRoots is an extended source root view used only inside K.Low. ModalQual MQ1-MQ6 binds local lowering/action cases, normalization/rank, normalized root correspondence, bidirectional path projection/lifting with finite-suffix root reflection, separate Sys_K/Sys_ref InFlightQual, and certificate versions. InFlightEscapeCorr plus DeadRootBridge collapses the extended view back to ordinary source message roots for literal Dead_K candidates, yielding DrainClosedNowCorr and generalized K.Low.A5.

8.3 One certified cutpoint versus universal coverage

DEFINITION: K.CertifiedCutpoint

A K-ready package for one reachable RTL KCut ρ_{post} , together with at least one explicitly recorded reachable prefix τ_{post} and finite normalization witness ν_{post} satisfying $\text{NormPost}(\tau_{\text{post}}, \nu_{\text{post}}, \rho_{\text{post}})$. It binds the theorem instance and contains the J.1/KObs correspondence, exact source KCut witness, J.RegPhaseReach, K.DrainReach including Core.SettleKBridge, the A6 source-ownership/elaborated-dependency-projection facts for every evaluated KObs/WFG boundary, waiver closure, and the remaining K-facing assumptions for that one package. On the normal compositional route the package may cite K.DrainReach-Comp for the exact K.DrainReach field once the bound DR1-DR4 instance evidence is established. It does not imply K.CertCov. When generalized drain is selected, the common K.CertHdr also binds the exact Q.2 action/effect schema and uniform Sys_ref K.InFlightQual used by K.1A; G2 serial/lowering packages additionally bind K.Low.ModalQual and both model-indexed InFlightQual records upstream.

THEOREM: K.1A - certified-cutpoint preservation from A5-L

Assume one certified RTL cutpoint satisfies Dead_K^W . Equal KObs transfers the same reconstructed message-root data and generalized DeadKRec structure to the exact reachable Sys_ref witness. The K-facing package supplies J.RegPhaseReach/K.DrainReach so the exact source prefix is Core.LAdm-admissible, while the theorem application separately binds the applicable Sys_ref K.InFlightQual that makes generalized DrainClosedNow operationally sound. This contradicts source A5-L. K.1A is route-neutral: it consumes source A5-L and source K.InFlightQual, not a G1/no-lowering premise; a G2 serial route may have obtained A5-L upstream through K.Low.ModalQual/K.Low.A5.

COROLLARY: K.1A-Total - universal result under K.CertCov and KCut closure

K.1A is pointwise over an already selected certified KCut. A universal claim additionally consumes Core.KCutClosure_post and K.CertCov. In the standard QSync scope, audited structural evidence including QS2a invokes Core.QSync-Post and Core.QSync-KCutCanonical, giving a canonical NormPost endpoint for every reachable KCut. The preferred package-totality route is not an unrelated per-cutpoint assumption: J.QSyncCertInd proves a base case plus one-cycle preservation over every reachable RTL successor; J.CertExist-QSync lifts that induction to J.QSyncCertCov and K.CertCov-QSync binds the tuples through K.CertHdr to derive K.CertCov. A single observed simulation path or sampled KCut set is insufficient unless a separately proved determinism/exhaustiveness result collapses the successor set. K.CertCov remains total only on KCutReach_post(l), not every arbitrary RTL prefix or normalization witness.

STANDARD QSYNC PACKAGE-TOTALITY ROUTE

J.QSyncCertState contains the substantive exact-prefix J/K package: canonical NormPost endpoint, accepted J.LocalGWR/J.GWR-Comp witness, J.HCanon/OfferReach/OfferConf/KObsConf, theorem-derived J.RegPhaseReach, and QC5 K.DrainReach/Core.SettleKBridge. J.QSyncCertInd is anti-circular: CI0 establishes the initial tuple; CI1 must extend it through any reachable one-cycle RTL successor; CI2 requires exhaustive successor coverage and may not assume the successor K.CertifiedCutpoint or K.CertCov. On the standard normal route, CI1 preserves applicability of the independently established DR1-DR4 K.DrainReach-Comp evidence, rebuilds the successor J.GWR-Comp/I.A0 attribution, history, and QSync settling facts, and then re-applies K.DrainReach-Comp to obtain successor QC5; there is no independent candidate-specific DR5 history input. J.CertExist-QSync plus K.CertCov-QSync then supplies K.CertCov on demand over the canonical KCut domain. The same J.QSyncCertInd proof projects to the cycle-aligned J safety proof.

8.4 The central source-liveness premise A5-L

PREMISE: A5-L - scheduler-robust liberal source liveness

A5-L abbreviates A5L_Sys_ref: no finite Policy-L prefix admissible under Core.LAdm for the selected Sys_ref instance reaches a source K-facing cutpoint satisfying Dead_K^W . Drain-closedness is part of Dead_K itself. Under generalized drain, the theorem application also binds the applicable Sys_ref K.InFlightQual; proving A5-L alone does not manufacture path-opportunity evidence. On the primary G2 serial route, the bounded-response reduction first establishes A5L_Sys_K on the normalized lowered model and generalized K.Low.A5 transports it to this source A5-L. Because Policy-S-shaped serial prefixes are included among admissible Policy-L prefixes, a reachable serial wait cycle still makes A5-L false.

A5-L is intentionally global. Internal wait cycles depend on interactions among processes, capacities, environment behavior, and the selected issue policy. Unlike the J safety construction, it cannot generally be reduced to independent per-process liveness theorems.

8.5 The primary Policy-S discharge route

The primary user-facing activity is still one selected complete instrumented Policy-S execution, and Definition K.SerialRoutePremises remains the authoritative K.2b applicability record. The generalized-drain revision does not replace that reduction; it makes the drain predicate and lowering transport precise around it. In G1, $\text{Sys_K} = \text{Sys_ref}$ and the serial route's bound Core.CycleClosure can serve the continuation-existence component of K.InFlightQual for that same model once the Q.2 action legality/profile-closure facts are supplied. In G2, the Policy-S reduction is evaluated on lowered Sys_K and must carry the Sys_K generalized-drain qualification. K.Low.ModalQual additionally binds the separate Sys_ref K.InFlightQual and the lowering stuttering/reflection/lifting package; generalized K.Low.A5 then transports A5L_Sys_K to source A5-L. $\text{K.SerialRoutePremises}$ deliberately does not contain A5-L itself. K.RespCov and K.CompleteS remain separate direct-K.2b inputs; A5-S supplies $\text{K.RespCov}/\text{K.CompleteS}/\text{K.RespClean}$ for $\text{K.2c}/\text{K.1}$. The existing $\text{K.CV}/\text{OpKey}/\text{CVPred}$, $\text{EnvMF}/\text{JointFreeze}$, $\text{CompareAdvanceLegal}$, reset-first interruption, and constructive $\text{FairPick}/\text{FairCycleDovetail}$ machinery remains otherwise unchanged.

Lowering is therefore a proof transport, not a redefinition of deadlock. Literal Dead_K is message-rooted in both models. A source synchronization carrier is separately typed by $\text{Core.7b}/\text{Core.7c}$; SyncFire is the real source transaction, while guard/epoch retirement actions introduced only by lowering project to source stutter. LowWaitRoots exists only to state the cross-model modal correspondence. The bridge back to the ordinary source predicate is $\text{K.Low.DeadRootBridge}$, which is candidate-scoped to a literal source message-blocked Dead_K candidate.

Element	Role
K.Low / K.Low.A5 / Sys_K	Witness-indexed lowering remains the route for synchronization waits and aligned non-NB-CF guard dependencies, but Stage 2 replaces the old local drain-equality idea with one global modal correspondence. $\text{Core.7b}-\text{Core.7e}$ type the source synchronization carrier and distinguish real SyncFire from lowering-only gate progress. $\text{K.Low.AdminStepRec}/\text{LowAdminNormalize}$ handle finite guard/epoch residue; LowWaitRoots is the extended source proof view; K.Low.ModalQual MQ1-MQ6 binds local lowering/action cases, finite normalization/rank, root correspondence, bidirectional path projection/lifting including finite-admin-suffix root reflection, separate $\text{Sys_K}/\text{Sys_ref}$ K.InFlightQual , and schema/checker versions. $\text{K.Low.InFlightPathCorr} \rightarrow \text{InFlightEscapeCorr}$ plus $\text{K.Low.DeadRootBridge} \rightarrow \text{DrainClosedNowCorr}$ supply the generalized K.Low.A5 transport. Literal $\text{MsgOf}/\text{WFG}/\text{Dead_K}$ remain message-only.
K.EnvMF	For $\text{StandardEnv}/\text{B1}$ -available, a candidate satisfying K.JointFreeze-Ord 's ordinary/single-release-support premises gets the no-release fact theorem-wise. Every other admitted candidate - including a single frontier item with a non-singleton or multi-alternative $\text{ReleaseOwnerSets_E}$ family and every explicit multi-frontier - supplies $\text{K.JointFreeze JF1/JF2}$. Timed/reactive or otherwise nondefault environments require exactly one selected internal frontier per process plus the K.JointFreeze-Ord single-singleton-release-support premises. K.EnvMF-Uniform is the standard instance-level sufficient discharge of that timed/nondefault branch; in the common ordinary primitive case its structural part reduces to $\text{Core.OrdFrontSingleton}$ plus the $\text{Core.WFG} \{\{Q\}\}$ release-support fact and Dead_K membership.
K.JointFreeze / K.CompareAdvanceLegal	K.JointFreeze-Ord is theorem-derived for the qualifying ordinary/comparability-certified single-frontier/single-singleton-release-support case; K.JointFreeze is required for other admitted StandardEnv structures. Generalized drain narrows one part of JF2: at the activation cutpoint, the candidate's DrainClosedNowRec already excludes every finite currently admitted InFlightStepRec path that changes D's roots. JF2 still has genuine continuation work: phase-legal activity not already contained in that initial closure, including outside-D progress and subsequently admitted work, must preserve the selected frozen operations/release closure for as long as they remain pending. $\text{K.CompareAdvanceLegal}$ remains the separate P0 preparation/R Issue legality condition, with the existing $\text{JointFreeze}/\text{CompareAdvanceLegal}$ consistency rule.

Element	Role
SDet	A flow qualification that fixes every history-affecting choice - model, capacities, elaboration, reset, stimulus/environment, witness, arbitration, concrete simulator scheduling, Core.PolicyS issue semantics, and epsilon normalization - and identifies one run artifact ρ_S^{det} . SDet alone does not prove completeness or response cleanliness.
K.InterruptionGate / K.DomResetScope / Reset / Terminal	Before the frozen-continuation argument, either certify candidate-relative K.DomResetScope/DomEpoch records and evaluate Reset first then Terminal, or provide one fixed conservative U1-U3 instance-level scope proving all required Continue branches. D2/D3 range over modeled process identities associated with source-level process-facing endpoints and the selected elaboration's qualified process-dependency/release-support interface, plus the consumed boundary-transfer identities; stateless couplers are not process identities. DirectFail remains the independently mapped RW2 cancelled-pending or DeclaredTerminal maximal-pending response-failure path. If the applicable route cannot be proved, use another A5-L discharge.
K.RespCov	Default-total finite-bound coverage of every static internal process-facing blocking site and relevant dynamic class, with an unambiguous trigger/response definition and a validated monitor, static bound, or unreachability disposition.
K.RespClean	No finite prefix of the complete selected run witnesses a non-waived certified-bound expiration, maximal-finite pending frontier, cancellation/reset failure, or end-of-test pending failure.
K.CompleteS	The selected run is maximal and fair. A terminating instance reaches its declared terminal state with every monitor resolved; a nonterminating instance requires an invariant, sound model-checking/lasso proof, or equivalent completeness evidence.
K.SerialRoutePremises	Authoritative deterministic serial-reduction applicability record consumed by K.2b. SR1 packages SDet, A1-A4/B5, and the constructive Core.FairPick form of Core.IssueFair/B2/B3; SR2 packages A6-A11, K.BF, ownership, and elaborated ReleaseOwnerSets/WaitOwners; SR3 packages K.CV, nonwithdrawal, Ke, K.Drain, and Core.CycleClosure; SR4 packages K.EnvMF and K.CompareAdvanceLegal; SR5 packages the reset-first K.ResetEpoch/K.TerminalDisposition schema, candidate-specific or uniform U1-U3; SR6 fixes all theorem-instance choices; SR7 packages NB-Align and applicable K.Low correspondence. When K.2b consumes a coupling-sensitive boundary, the bound elaboration/template evidence must also provide Q.1(l) handshake support and CouplerPersistence. In particular, Q.1(d)-(k) alone does not admit mutual-stall, shared-capacity, or arbitrated/non-monotone handshake topologies to K.PSF/CF-S; a template without the required persistence proof must use another A5-L route. The record does not include A5-L. K.RespCov/K.CompleteS are separate direct-K.2b inputs; A5-S supplies K.RespCov/K.CompleteS/K.RespClean for K.2c/K.1.
A5-S / CF-S	The complete response certificate over the SDet-selected run. For K.2c/K.1, A5-S supplies K.CompleteS + total K.RespCov + K.RespClean, bound to the same normalized theorem instance and trust boundary. The applicability/provenance conditions are referenced through K.SerialRoutePremises rather than duplicated here. This source-side response evidence, together with that applicability record and K.Low.A5 when lowering is used, is the route from K.2c to A5-L on the supported fragment.
K.CV / K.OpKey / K.CVPred / K.CV-WF / K.2b.P0-P1	K.CV is the authoritative design-level causal/value determinism and provenance premise. CV1 binds the theorem-instance OpKey universe and per-execution realization/source-order layer. CV2 is discharged by Definition K.CVPred/K.CV-WF: a common typed CVFact universe, finite predecessor sets, occurrence closure, deterministic fact evaluators, serial-route comparison-availability, explicit Core.16 reach-legality dispositions for recursively materialized non-transfer predecessors, and a well-founded relation/rank. K.CVPred/K.CV-WF define the certificate structure and do not assume K.CV as a prior premise. K.2b.P1 proves value/identity conditional on occurrence; K.2b.P0 proves keyed occurrence/reachability.
Core.CycleClosure / Core.FairPick / Core.FairCycleDovetail	K.2b.E constructs the frozen liberal continuation through typed Core.CycleResult successors. Core.CycleClosure C2/Core.CycleChoiceExt type the partial-cycle completions. Core.FairPick packages the existing Core.IssueFair and B2/B3 behavioral discharges as prefix-local selectors/starvation

Element	Role
	proofs; Core.FairCycleDovetail first completes the selected L1 segment through an actual L2 SettlePoint, then selects legal fair L3 work and completes the CycleResult. B5 remains a separate closure premise.
K.InFlightQual / Q.2 / J.InFlightPathSound	Model-indexed generalized-drain qualification. IQ1 binds the exact Q.2 action/effect schema and dangerous-direction effect soundness; IQ2 finite choice plus Core.InFlightDrainWF; IQ3 Core.InFlightCycleExt/full CycleClosure/equivalent opportunity; IQ4 J.InFlightPathSound with prefix-compositional profile closure and exact reconstructed successors; IQ5 precision/completeness/search class. G2 ModalQual binds one record for Sys_K and one for Sys_ref unless a named preservation theorem discharges a specific field.

LEMMA: K.2b - deterministic serial-source dominance with response-failure extraction

Assume the fixed theorem-instance choices, K.SerialRoutePremises, K.RespCov, and separate K.CompleteS evidence for the SDet-selected Policy-S run. Then any Core.LAdm-admissible Policy-L source KCut satisfying Dead_K^W forces that same selected Policy-S execution to produce a finite non-waived K.RespFail witness. K.SerialRoutePremises centralizes the blocking/lowering, K.CV provenance/comparison-availability, environment/frontier-freeze, Core.16 comparison-advance, constructive fairness, drain, identity, reset/terminal, fixed-choice, and NB-Align/K.Low applicability conditions. The strengthened proof carries both earlier serial endpoint transfers and lower-ranked K.CV predecessor facts needed by P0/R; predecessor materialization may not release the frozen component or synthesize missing prerequisite transfers. K.2b.E uses Core.FairPick/Core.FairCycleDovetail rather than assuming a pre-existing fair continuation. Failed non-blocking polls remain WC-selected return outcomes, not commits. The lemma does not claim equal traces, equal timing, or the same deadlock component.

COROLLARY: K.2c - the selected deterministic Policy-S execution suffices

If the selected Policy-S execution satisfies A5-S, the response package plus K.SerialRoutePremises rules out any liberal Dead_K^W counterexample on the selected Sys_K proof model, with that model's generalized K.InFlightQual bound. In G1 Sys_K=Sys_ref and the result is source A5-L directly. In G2, K.Low.ModalQual supplies the completed cross-model path/root correspondence and the separate Sys_ref K.InFlightQual; generalized K.Low.A5 transports A5L_Sys_K to the source A5-L consumed by K.1A. SDet still removes alternate Policy-S simulator histories; the q→K changes do not turn response monitoring into a trace-equivalence or timing theorem.

STATUS: q→K paper architecture closed for G1 and qualified G2; Lean mechanization, reusable checker implementation, and concrete template/flow qualification outstanding

The paper theorem stack now includes generalized in-flight drain, subset/heredity and relocation, typed synchronization, and the complete Stage-2 lowering stuttering correspondence. Remaining work is deliberately per-template/per-instance or mechanization work: qualify concrete Q.2 schemas beyond the worked artifacts; discharge Core.InFlightCycleExt/J.InFlightPathSound on A5-L routes that do not already carry suitable CycleClosure; provide reusable K.Low.ModalQual/MQ4 proofs for real guard/epoch lowering packages; and mechanize/check the serial-dominance and certificate-coverage interfaces. These are applicability/implementation obligations, not an unresolved q→K definition gap.

PRACTICAL WARNING: One run must be complete and flow-qualified

For a terminating design, the selected run must reach the declared maximal terminal state and resolve every covered monitor. For a nonterminating design, a finite trace is only evidence; an invariant, sound model-checking result, recurrence/lasso proof, or equivalent argument must establish completeness and continued response cleanliness. In either case, the flow must also establish SDet, route applicability, total finite-bound response coverage, and certificate validity; a clean trace alone is not the theorem premise.

8.6 Alternate direct discharges of A5-L

Route	When it can discharge A5-L
K.Str-1 - acyclic dependency graph	If the full static modeled-process dependency graph induced by the possible WaitOwners_E projections is acyclic, no reachable coarse WFG cycle can occur and therefore no K.Dead component can satisfy its connectivity clause.
K.Str-2 - ranked dependency	A well-founded rank strictly decreases along every realizable coarse WFG edge induced by WaitOwners_E. Therefore a WFG cycle would force an impossible descent, thereby excluding the internal cycle/connectivity required by K.Dead. This is a conservative sufficient route; K.Dead itself retains exact ReleaseOwnerSets_E closure.
K.Str-3 - credit/token cycle breaking	Every static dependency cycle contains an edge proved unrealizable at reachable cutpoints, often by occupancy, initial-token, or credit invariants.
CF-3 / direct invariant	An inductive abstraction proves the prohibited Policy-L cutpoint unreachable.
CF-4 / exploration	Sound exhaustive or bounded model checking over the selected transition system.
CF-5 / tool certificate	A tool-emitted certificate using knowledge of the scheduled control graph, storage, arbitration, joins, and automatic flush.

These are alternate proofs of exactly the same A5-L property. They cannot rescue a design for which an admissible serial-shaped Policy-L prefix already reaches Dead_K^W.

Route-dependent continuation, reset, and termination coverage. The alternate A5-L routes prove the same abstract premise but need not use the Policy-S complete-cycle machinery. The Policy-S route requires K.CompleteS and its selected Core.CycleResult semantics, the constructive Core.FairPick continuation discharge, and reset/terminal interruption coverage. That coverage may be candidate-specific through K.DomResetScope plus reset-first K.ResetEpoch/K.TerminalDisposition records, or one uniform instance-level U1-U3 sufficient condition. A reset or terminal event that cannot be classified by the selected valid route puts that candidate outside the serial reduction; simulation stopping does not make it safe. Under generalized drain, an alternate structural/invariant/exploration/tool A5-L route also binds the Sys_ref K.InFlightQual needed by the Dead_K predicate it is proving; it may discharge Core.InFlightCycleExt directly rather than carrying the full serial-route Core.CycleClosure. A direct proof of A5-L is not a substitute for this model-specific reconstructed-path qualification.

8.7 Standing assumptions A1-A11 in engineer terms

Assumption	Plain-language role
A1	The source and RTL satisfy the previously defined R-rules and B-rules at the selected boundaries.
A2	Finite internal epsilon depth / finite stutter when not externally stalled.
A3	Weak fairness: a continuously enabled action is eventually selected.
A4	Channel progress: persistent complementary requests eventually discharge full/empty/rendezvous blocking.

Assumption	Plain-language role
A5	For the user-facing Policy-S route, a complete bounded-response-clean selected serial run; K.1A itself consumes A5-L directly.
A6	Point-to-point source ownership plus elaborated dependency projection: each source-level logical channel has one Push-owning process and one Pop-owning process; shared source resources are explicitly arbitrated. At an elaborated boundary a stateless coupler may occupy one side without becoming a process. The qualified elaboration supplies the ReleaseOwnerSets_E/WaitOwners_E process projection, including soundness/completeness and no spurious self-dependency.
A7	B1 RTL determinism/witness selection plus the exact equal-KObs source/RTL correspondence and Core.LAdm qualification.
A8	Barrier participation is well formed; barrier mismatches are outside the internal message-passing deadlock theorem.
A9	Instance assumption referring back to the Core scalar endpoint/cardinality convention: at most one Push and one Pop commit per channel per cycle unless an explicit multi-lane/burst model is selected. It does not redefine the Core rule.
A10	Instance assumption referring back to the Core reset-empty buffered-channel convention: every selected buffered channel is logically empty at each applicable reset boundary unless preload is explicitly modeled. It does not create a competing reset-state definition.
A11	K ϵ : explicit normative WFG-inert epsilon condition. After the WFG-relevant request/attribution cutpoint is fixed, an epsilon-only normalization segment with no selected observable/request-changing event may not create, remove, redirect, or otherwise change the K-relevant frontier, enablement, release-support, or WFG structure.

THEOREM: K.1 - user-facing deterministic Policy-S preservation

Combine the uniform K.1A premises with the selected Policy-S route. Bind the applicable generalized-drain qualifications in addition to K.SerialRoutePremises and A5-S. In G1, Sys_K=Sys_ref and the same valid K.InFlightQual serves the serial and source theorem. In G2, the selected Sys_K run binds its own K.InFlightQual and K.Low.ModalQual binds a separate Sys_ref K.InFlightQual plus the global lowering stuttering/root correspondence; K.Low.A5 transports the derived A5L_Sys_K result to the source A5-L used by K.1A. The universal RTL layer remains Core.QSync-Post/QSync-KCutCanonical plus J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync and Core.KCutClosure_post. The conclusion is still absence of the defined non-waived reachable internal message-passing Dead_K cutpoint class, not “RTL never hangs.” $q \rightarrow K$ is paper-closed; concrete Q.2/path/lowering qualifications and mechanized checkers remain required.

8.8 What Appendix K does not cover

- Barrier mismatches or conditional SyncChannel participation remain outside the literal internal message-passing Dead_K theorem and are handled by A8/separate protocol verification. The Stage-2 SyncCarrier/SyncFire machinery exists only so a well-formed synchronization obstruction can be represented faithfully in in-flight/lowering correspondence; it does not add synchronization to MsgOf or the WFG.
- Waiting indefinitely for an external environment input, output readiness, starvation/perpetual under-supply, termination, EOF, cancellation, or reset behavior not represented by the defined closed internal WFG/Dead_K predicate. The persistent-deadlock conclusion is deliberately limited to that internal wait-cycle class.
- Independent nonterminal withdrawal, cancellation, retry, or reattribution of an already-issued blocking request; Definition Core.IC excludes those lifecycles from Sys_ref unless a separate operational/correspondence/progress extension is defined, and the RTL side must separately demonstrate Core.IC conformance.
- Cycle-accurate RTL response bounds; K.Resp evidence is a source-side discharge of A5-L, not an RTL timing contract.

- Designs whose correctness requires favorable eager/same-cycle issue when a serial-shaped admissible prefix deadlocks.

9. Appendix L - witness selection and snooping

PURPOSE

Conditionally realize a WC-compatible source witness for latency-sensitive epsilon choices while leaving the unique free-running RTL behavior untouched.

Engineer takeaway: Snooping selects one admissible source witness only when the required WC/L.Dir witness exists. It must not feed pre-HLS readiness back into RTL, relabel payloads, drop/reorder observations, or treat an unrealizable witness as a silent stall.

9.1 Why snooping is needed

A non-blocking arbiter, interrupt controller, or retry/timeout block may make a later observable choice based on which request arrived first or how many polls failed. HLS changes internal latency, so the raw pre-HLS and post-HLS simulations can select different epsilon outcomes even though both satisfy the broad scheduling contract. When those outcomes affect later Sigma-visible behavior, Appendix I needs a specific WC-compatible witness.

If NB-CF holds - failed non-blocking outcomes do not affect later observable control, values, frontier, or request attribution - the witness is trivial and snooping is unnecessary. Cadence-sensitive polling is different: it is a localized latency-sensitive protocol and must be isolated or explicitly snoop-aligned. Appendix L is conditional: if no admissible source execution can consume the RTL event sequence in ordinal order with matching kind and payload, WC/L.Dir is not discharged for that comparison.

DEFINITION: L.1 - witness selector / snooping

At each relevant epsilon-quiescent source witness-choice cutpoint, the selector returns the outcome chosen by the matched RTL prefix using only causally available history. This is intentionally a generic quiescent witness-selection interface, not a Core.KCut requirement: KCut adds a K-facing boundary-evaluation condition that an arbitrary snooped epsilon choice need not satisfy. The selector cannot depend on future RTL events or invent a retry/timeout outcome inconsistent with the isolated protocol.

LEMMA: L.2 - conditional snooping discharge of WC

If the snooped set covers every epsilon choice that can affect later observable behavior, the wrapper changes only epsilon-choice selection, selected outcomes are causal and RTL-consistent, successful and failed non-blocking calls retain the required Sigma/epsilon interpretation, cadence-sensitive blocks are isolated, and the selected ordinal event stream is realizable, then the selected source execution satisfies WC for Appendices I and J. This is a conditional witness-construction theorem, not a guarantee that every RTL event stream has a matching source witness at the chosen boundary.

9.2 One-directional architecture

CONDITION: L.Dir - one-directional alignment / RTL-side non-interference

The RTL_B execution is free-running under the fixed stimulus. The pre-HLS snoop sequence must consume the recorded RTL snoop sequence in ordinal order, with matching kind and payload, and no pre-HLS backpressure/readiness may stall or alter RTL. Benign cycle skew is permitted. Order, kind, or payload divergence means that the required WC witness does not exist at the chosen boundary; the run has no theorem-backed guarantee and the harness must abort and triage the failure rather than silently gating or hanging.

9.3 Wrapper realization requirements W1-W5

Requirement	Engineering meaning
W1 - recorded ordinal observation	Capture each RTL snooped event by dynamic ordinal and hold it until the pre-HLS side consumes the same ordinal. Do not rely on overlapping live assertion windows.
W2 - gate only commit enable	Gate only the mirror signal that enables the pre-HLS commit: observed valid for a pre-HLS Pop or observed ready for a pre-HLS Push. Never modify the source endpoint-owned request.
W3 - registered sampling	Register the RTL observation before it enters the source-side gate to avoid delta-cycle races; the extra source latency is part of witness selection.
W4 - compare and watchdog	At each source snoop commit, compare ordinal, kind, and payload against the recorded RTL event. Never substitute the RTL payload into the source, because that would hide value divergence. Detect missing, extra, reordered, or never-consumed observations. A watchdog must convert non-existence or failure to realize the witness into a reported result; if the watchdog bound is not proved sufficient, a bound-exceeded abort is an inconclusive witness-realization failure, not by itself a proved L.Dir violation.
W5 - independent coverage proof	The wrapper mechanism is sound only for the points it records. A separate analysis must prove that all epsilon choices capable of affecting later observables are covered.

IMPLEMENTATION PITFALL: Why a live AND is unsound

ANDing the current pre-HLS and current RTL handshake levels can drop an RTL event when the RTL assertion retires before the source arrives. It can also reorder or duplicate matching under multiple outstanding events. The normative rule is an AND against a recorded per-ordinal observation retained until source consumption.

9.4 Harness soundness and shared testbenches

LEMMA: L.5 - online realization of the witness selector

A co-simulation harness realizes the abstract selector only when a matching witness exists, the harness preserves the free-running RTL trace, implements W1-W5, satisfies L.Dir, and produces the same source-side witness choices assumed by the formal comparison. The formal theorem is about the fixed RTL trace and selected source witness; harness correctness, coverage, and successful witness realization are additional implementation obligations.

A shared reactive testbench becomes problematic once one model can lag significantly, because reactions may be driven by whichever model the testbench observes. The flow should replicate the stimulus/testbench per model, or structure a shared environment around a common causal transaction stream rather than raw cycle timing.

9.5 What snooping means formally

Snooping does not use RTL to change the source specification. The source model is intentionally under-specified with respect to permitted latency and epsilon choice. Core.SelectLatitude supplies the ordinary finite L1/L3 non-maximal-selection component of that latency freedom; WC/Appendix L handles witness-sensitive epsilon choices whose outcomes affect later observable behavior. When a WC/L.Dir witness exists, the RTL observation selects one admissible execution of that specification for comparison. If the event order, kind, or payload cannot be realized, the comparison is outside theorem scope and must be triaged among source-model ill-formedness, incomplete snooping coverage/harness behavior, concrete source-carrier nonconformance to Core.SelectLatitude/WC, and tool/RTL non-compliance.

9.6 Latency-robustness stress testing

Appendix L also recommends randomized stalls, varied channel latency/capacity, and adversarial weakly fair scheduling in the pre-HLS model. This is not a substitute for the formal premises, but it is a practical way to detect designs that accidentally rely on incidental ordering of causally independent events. A failure under such stress usually indicates that the source specification itself is outside the intended latency-insensitive model or that an explicit snoop/isolation boundary is missing.

10. How the appendices work together in a verification flow

PURPOSE

Translate the theorem architecture into a concrete signoff workflow.

Engineer takeaway: Safety-only use stops after the J.1/J.1.1-Safe package. Appendix K is added only when the claim includes the defined internal wait-cycle freedom, and it requires substantially more global evidence.

10.1 Step-by-step safety flow

Step	Deliverable
1. Fix the theorem instance	First screen the design patterns against Appendix F of the scheduling-rules document. Then select one single-global-clock theorem instance: Sys_ref = Sys_B or Sys_B^elab, explicit boundaries and capacities, and, where elaborated, the Core.ElabProj package (Π_{elab} , π_{Σ} , E, Π_{token} , E, signed A_E plus the declared handshake/dependency interface); observable alphabet; reset interpretation; fixed stimulus/environment; witness provenance; tool/RTL identities; and any mapped-memory logical reference/boundary/profile. Multi-clock designs require a separate semantic layer or per-domain verified boundaries.
2. Establish Core/G and QSync conformance	Check the Core/G instance semantics and qualifications - Core.8 E4 commit mapping; Core.9 operational Issue, where Issue(q) is the first cycle in which the calling process begins the process-owned request and is not a witness-selectable timestamp; Core.10's fixed bidirectionally complete operation/boundary-specific BoundaryReq mapping, which distinguishes the process-owned request from complementary channel/storage availability or coupler logic; Core.IC request nonwithdrawal/live-owner/back-to-back attribution, including Issue(q1)=t+1 when ownership passes immediately after q0 commits; signal anchors; Core.RegSeq; Core.ReachPrep where eager preparation is admitted; Core.SelectLatitude source-model conformance when a concrete executable reference carrier is used; Core.SyncFence; Core.MemFaithful/Core.MemSyncOrder and Core.MemProfile/J.Mem where triggered; Core.LLegal/Core.Phase/Core.Settle and the L2 frame; Core.17 SigmaMatch including the final-source-Write value rule at a shared E2 anchor, selected proof-internal RESET units, and R2C component/local-occurrence matching; R1-R5/E1-E5; channel/elaboration boundaries; and Core.QSync QS1-QS5 including QS3 realization/adequacy. On Sys_ref, Core.9/Core.10/Core.IC/Core.SyncFence/Core.SelectLatitude are operational/selection semantics; implementation or concrete executable carriers are evidence-backed where applicable. Bind the single global clock and fixed cycle-entry/environment choices.
3. Discharge WC	Use NB-CF, an Appendix L snooping/L.2 construction with a realized WC/L.Dir witness, direct witness construction, or a complete mixed per-site coverage argument.
4. Produce local evidence	Generate J.PCert from Appendix I and per-channel/atomic/environment/reset certificates plus typed footprints and local dependencies.

Step	Deliverable
5. Run J.GWR-Comp	On the normal route, invoke Core.QSync-Ref for source L2 settling, generate the D1-D4 semantic edges and D5 Horizon-stratification edges, validate D5-only commutation with J.HorizonOrderSafe, prove the graph well-founded with J.FoundationRank/J.DepRank, then build the phase-normal nonempty-Horizon finite-ideal chain and J.GWR/J.RegPhaseReach. If selected cycle-sensitive evidence requires a later source cycle entry before a later nonempty bucket, J.UnitExt uses Core.SelectLatitude to insert the finite invariant-preserving unit-empty CompleteCycle/SilentCycle bridge; no empty rank bucket is created and no B2/B3 fairness is invoked.
6. Apply Proposition J.0 and Theorem J.1	Use Proposition J.0 on the compatible source/RTL pair constructed by J.GWR-Comp (or supplied by the direct J.GWR route), then apply Theorem J.1 to obtain Post $\rightarrow$ Ref observable compatibility for the covered reachable RTL prefix over Reach_post/Reach_ref. Core.17 Σ Match supplies complete selected occurrence/value matching; Appendix G supplies E1-E5. In the restricted Ref $\rightarrow$ Post clause, J.RTLConsistentRefPrefix instead supplies the already-selected reachable matching RTL prefix with no unmatched selected observable unit and the required annotations, and J.0 composes that pair.
7. Transfer the safety property	Use route-sensitive J.TraceCertCov and Corollary J.1-Safe over the required RTL coverage domain, with the safety predicate satisfying J.IdealPrefixClosed. For the standard QSync route, discharge J.QSyncTraceInd (or project an already-proved J.QSyncCertInd through J.QSyncCertInd-Trace), then use J.TraceCertCov-QSyncCycle/J.1-Safe-QSyncCycle on CycleReach_post(I). Core.RTLCycleCarrier plus J.CycleIdealCover/J.1-Safe-QSync then extend that conclusion to all reachable finite outer observable prefixes without constructing a separate package for every intra-cycle ideal. This shortcut does not cover a property that fails J.IdealPrefixClosed, such as some cross-interface scoreboards; those claims require direct J.TraceCertCov on the relevant arbitrary-prefix domain. If one complete source run represents source verification, also prove J.RefProjCover, including latency-coverage when different Core.SelectLatitude-legal delays can change the canonical Σ_{DUT} history.

10.2 Additional liveness flow

Step	Deliverable
8. Establish the KCut observation domain	For source models in the standard scope, Core.QSync-Ref proves the finite unique L2 SettlePoint and Core.SettleIsKCut makes it the source K-facing state. For RTL_B, establish QSync including QS2a origin recoverability and invoke Core.QSync-Post/Core.QSync-KCutCanonical: every reachable post KCut is the unique same-cycle endpoint of its designated reachable K-origin and has a canonical NormPost witness before the next registered update. Core.KCutObserve preserves any persistent K-relevant blocker under K _e .

Step	Deliverable
9. Build the KObs bridge	Discharge J.HCanon, J.OfferReach, J.OfferConf, and J.KObsConf for the exact same source/RTL/witness package and KCut endpoints. J.KObs-Proc/Chan/Offer/WFG reconstruct the message-root state and exact ReleaseOwnerSets/WaitOwners. If synchronization is selected, also bind Core.SyncCarrierWF and use J.KObs-SyncCarrier/J.KObs-SyncInFlight. J.KObs-InFlight reconstructs the current Sys_ref FlightStateRec and Q.2 action/effect relation. Do not treat this as a future-progress theorem: KObs/current local reconstruction contains no historical Issue time, activation certificate, or future action-opportunity fact.
10. Qualify the source witness	First use theorem-derived J.RegPhaseReach plus K.DrainReach to prove K.RLAdmReach/Core.LAdm admissibility for the exact reached source witness; on the normal route K.DrainReach-Comp combines DR1-DR4 with the J/QSync construction facts. Separately bind generalized K.InFlightQual for the proof model on which Dead_K will be interpreted: exact Q.2 action/effect schema and Q.InFlightActionEffectSound, finite action choice/Core.InFlightDrainWF, Core.InFlightCycleExt or sufficient CycleClosure-equivalent opportunity, and J.InFlightPathSound with profile closure/exact successors. These are different layers: K.DrainReach qualifies the reached prefix; K.InFlightQual qualifies finite future already-admitted paths from the K-facing observation.
11. Certify the cutpoint	Assemble K.CertifiedCutpoint for the selected RTL KCut and one recorded reachable-prefix/NormPost pair, including the exact source KCut witness, J.RegPhaseReach, K.DrainReach/Core.SettleKBridge, source ownership/elaborated dependency projection, handshake/release-support template, waivers, and remaining K-facing assumptions. Under generalized drain, K.CertHdr also binds the Q.2 action/effect schema identity and the uniform Sys_ref K.InFlightQual used by K.1A. The per-cutpoint file need not duplicate an instance-level path-soundness proof.
12. Run and qualify the A5-L discharge	For the primary route, execute one complete instrumented Core.PolicyS run and establish K.SerialRoutePremises plus A5-S/K.Resp. Bind the generalized-drain schema for Sys_K. If no lowering is required (G1), Sys_K=Sys_ref and the serial route's Core.CycleClosure may discharge the continuation component of the same-model K.InFlightQual once action legality/profile closure are proved. If lowering is required (G2), additionally discharge K.Low.ModalQual MQ1-MQ6: exact lowering/local action schemas; finite LowAdminNormalize/rank; LowStateCorr and normalized RootCorr; MQ4 downward projection, upward finite lift, and finite-administrative-suffix root reflection; separate Sys_K/Sys_ref K.InFlightQual; and certificate/schema binding. Generalized K.Low.A5 then transports A5L_Sys_K to source A5-L. An alternate A5-L route may bypass the serial run but still supplies the Sys_ref K.InFlightQual appropriate to generalized Dead_K.

Step	Deliverable
13. Establish universal KCut closure and coverage	Apply K.1A directly to one selected certified KCut. For K.1A-Total or universal K.1, consume Core.KCutClosure_post and K.CertCov. In the preferred standard QSync route, QSync-Post/QSync-KCutCanonical derive the complete canonical KCut domain, while J.QSyncCertInd proves the full per-cutpoint correspondence package by base/one-cycle/exhaustive-successor induction. On the normal route Cl1 preserves applicability of the already-bound DR1-DR4 evidence, reconstructs the successor J/QSync facts, and re-runs K.DrainReach-Comp to supply successor QC5; it does not assume an arbitrary successor DrainReach package. J.CertExist-QSync then gives J.QSyncCertCov and K.CertCov-QSync derives K.CertCov. Alternate independently checked K.CertCov routes remain permitted. A single observed run or sampled KCut set is not package-totality evidence.

10.3 Recommended reading order in the scheduling-rules document

- Start with the front Abstract and Formal Guarantees at a Glance, then read Appendix F for the architect-facing design constraints. Use Appendix M for the formal conclusions, scope, and division of responsibilities. Continue with the unlettered Guide to the Formal Appendices, the Normative Formal Core for the shared semantic skeleton, and Appendix G for the R/E observable contract.
- In the Normative Formal Core, focus on Sys_ref, Core.8 Commit/CommitCycle, Core.ReachPrep, Core.IC, Core.RegSeq, Core.PolicyL/Core.LLegal/Core.SelectLatitude/Core.Phase/Core.LAdm, Core.PolicyS, Core.IssueFair, Core.Settle and its L2 frame, Core.QSync/Core.QSync-Ref/Core.QSync-Post including QS3 and RTL QS2a, Core.QSync-KCutCanonical, Core.CycleState/Core.CycleResult/Core.RTLCycleCarrier/Core.CycleClosure/Core.CycleChoiceExt, Core.FairPick/Core.FairCycleDovetail, Core.SyncFence, Core.MemFaithful/Core.MemSyncOrder/Core.MemProfile and J.Mem, Core.ElabProj ($\Pi_{\text{elab}}/\pi_{\Sigma,E}/\Pi_{\text{token},E/A_E}$), Core.11-Core.13 MirrorAvail_E/ChannelEnabled semantics, Core.WFG with ReleaseOwnerSets_E/WaitOwners_E, Core.16/Core.Drain/Core.SettleKBridge, Core.17 SigmaMatch, and Core.18/Core.18-e KObs. Read Core.SelectLatitude as finite-prefix source legality rather than fairness, and read the Core.IC/Core.SyncFence status note as a trust-boundary rule: source legality is definitional; corresponding RTL and concrete executable-source conformance are evidence-backed where applicable.
- Read Appendix I to understand exactly what is local and why deferred non-blocking holes are needed.
- Read Appendix J as the central safety construction; focus first on J.LocalGWR, the D1-D4/D5 distinction in J.DepJ, J.DepSound/J.HorizonOrderSafe/J.DepComplete/J.DepRank/J.DepAcyclic, finite canonical ideals, J.RegPhaseNF, J.UnitExt, J.GWR-Comp, Theorem J.1 and J.RTLConsistentRefPrefix, J.IdealPrefixClosed, J.TraceCertCov/J.RefProjCover/J.1-Safe, then the QSync safety layer J.CycleReach_post/J.QSyncTraceInd/J.TraceCertCov-QSyncCycle/J.1-Safe-QSyncCycle/Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync and the richer J.QSyncCertState/J.QSyncCertInd/J.CertExist-QSync package induction. In J.UnitExt/J.GWR, note that only nonempty Horizons participate in the rank/order machinery; when cycle-sensitive evidence requires a later source cycle entry, a finite invariant-preserving unit-empty CompleteCycle/SilentCycle bridge may precede the next nonempty bucket under Core.SelectLatitude, without importing B2/B3 or matching the raw RTL Horizon gap mechanically. For the K-facing bridge, read J.HCanon/J.OfferReach/J.OfferConf/J.KObsConf plus J.KObs-Proc/Chan/Offers/WFG; the latter reconstructs the settled handshake, ReleaseOwnerSets_E/WaitOwners_E, and Core.WFG from the common KObs record. On the normal route QC5 may cite K.DrainReach-Comp rather than carrying an independently invented drain-history certificate.
- Read Appendix L before K when witness-selected non-blocking or latency-sensitive interfaces are present.
- Read Appendix K for the release-aware Core.WFG/ReleaseOwnerSets_E/WaitOwners_E deadlock reconstruction, K.ReleaseSCCReduction, K.EnvTerminalAdequate/K.KObsDerived, K.KObs-Dead/K.KObs-Ext, K.DeadChar, K.DrainReach/K.DrainReach-Comp/K.RLAdmReach, the certified-cutpoint and K.CertCov interfaces including K.CertCov-QSync, A5-L, Definition K.CV with K.OpKey/K.CVPred/K.CV-WF, explicit K_e and NB-Align, K.Low/K.Low.A5, Definition K.SerialRoutePremises, K.2b.P0/P1, K.JointFreeze-Ord/K.JointFreeze, K.EnvMF/K.EnvMF-Uniform, K.CompareAdvanceLegal/K.CompareAdvanceLegal-Ord, K.DomResetScope, K.InterruptionGate with candidate-specific or uniform U1-U3 reset-first discharge, and the Core.FairPick complete-cycle Policy-S response workflow.

11. Assumptions and evidence matrix

PURPOSE

Summarize the named assumptions that most often determine whether a theorem instance is usable.

Engineer takeaway: The important audit question is not merely “is the assumption plausible?” but “what artifact establishes it for this exact source, RTL, capacity/elaboration, stimulus, witness, reset, and environment instance?”

Architect-facing entry point. Appendix F of the scheduling-rules document is the design-pattern entry point to this matrix. Poll-dependent control triggers WC/L.Dir or isolation evidence; a concrete executable source/reference carrier must preserve Core.SelectLatitude rather than impose hidden maximal progress; mapped-memory transformations hidden below the proof boundary trigger Core.MemFaithful, qualifying cross-process memory dependencies trigger the complete J.Mem audit, array/synchronization ordering independently triggers Core.MemSyncOrder, and transaction-changing or persistently backpressuring memories may require Core.MemProfile; eager source/RTL overlap triggers Core.ReachPrep coverage; ordinary blocking interfaces trigger the Core.IC implementation-conformance check even though Core.IC is constitutive on Sys_ref; coupling-sensitive Sys_B^elab boundaries trigger the stateless-handshake plus ReleaseOwnerSets/WaitOwners template qualification; standard K-facing QSync additionally triggers QS2a origin recoverability; and the Policy-S route adds K.CV comparison-availability, the applicable release-structure JointFreeze/CompareAdvanceLegal checks, K.EnvMF-Uniform where used, constructive Core.FairPick continuation evidence, and reset/terminal interruption coverage. Universal standard-QSync K claims additionally use J.QSyncCertInd, whose normal-route drain step re-applies K.DrainReach-Comp from bound instance evidence. Appendix M.7a remains the normative per-instance discharge ledger; M.7b is the standard four-bundle packaging view over that ledger.

How to read this matrix. The entries below are not a checklist of new manual tasks for every architect or design. The underlying engineering conditions represented by many entries were largely already present, explicitly or implicitly, in both HLS-generated and manual RTL flows; the formalism exposes them so a flow can determine applicability, assign ownership, and retain auditable evidence. Depending on the selected claim, an entry may be inapplicable, satisfied by a simple design restriction, derived from another checked result, discharged once through reusable tool, library, or template qualification, or established automatically for the particular implementation. The theorem-backed assurance layer can add explicit certificate, provenance, or checking work, but that additional assurance should not be confused with creating the underlying engineering condition.

Premise / evidence	Where it matters and typical discharge
B1 - RTL deterministic trace	Consumed by J/K/L selected-trace reasoning. Evidence: RTL/tool determinism proof, qualification, deterministic arbitration/environment setup, or a validated witness-selection setup. B1 is observable trace determinism, not the same thing as QSync same-cycle settling determinism.
B2 - weak fairness	Consumed only where a complete/fair execution or eventual boundary selection is required, notably later liveness/fair-continuation reasoning. B2 is weak fairness: a boundary action that remains continuously ChannelEnabled from some point onward must eventually be selected; merely intermittent enablement does not satisfy the antecedent. It is not a premise of either finite-prefix direction of Theorem J.1 and is not the source of ordinary finite deferral; Core.SelectLatitude supplies that legality. Discharge ownership is architectural rather than categorically “environmental”: it may lie with source/scheduler semantics, HLS-generated or hand-written RTL resource/memory-port/interface/channel arbitration/back-pressure, channel-library behavior, external masters, or reusable qualified arbiter classes bound to the concrete instance. Evidence: scheduler/arbiter fairness proof, assertions, or Appendix N/M.7a qualification.

Premise / evidence	Where it matters and typical discharge
B3 - channel progress	Consumed by I.2 and K. For Push at a full buffered channel, B3 guarantees a complementary Pop discharge that frees space; for Pop at empty, it guarantees a complementary Push discharge that supplies data. This is not by itself a conclusion that the original blocked operation commits. If that request remains pending and continuously enabled afterward, E4 plus B2 govern its eventual commit. Evidence may be a per-boundary progress certificate or the stated persistent two-endpoint-request premises.
B4 - finite epsilon-step depth	Consumed by Appendix I finite preparation and replay construction. D_P bounds consecutive P-owned epsilon-steps only while P remains internally advancing toward its next observable action and is not externally stalled. Stable waiting, peer delay, environmental delay, and back-pressure waiting are outside this charge. Evidence: finite source-control/pipeline/FSM progress plus the required no-infinite-epsilon-spin argument. B4 is not the QSync same-cycle settle theorem.
B5 / B6-Terminal	B5 supplies bounded global stabilization where invoked. A B6 idle suffix is permitted only after Core.TerminalIdle proves both a genuine B5/global fixed point and permanent no-future-enabling machine/environment behavior (Core.EnvTerminal supplies only the environment component). A currently quiet nonterminal cycle is Core.SilentCycle, not B6 stutter.
WC	Consumed by I and J; provenance may be NB-CF, L.2 snooping, direct witness construction, or mixed per-site coverage.
Core.RegSeq	Consumed by I/J/K. Evidence: sequential interface structure; request/data outputs are functions of registered state and operands fixed before L2; ordinary combinational derivation from those registers is allowed, but no same-cycle newly returned interface result may feed a dependent sequential output.
Core.QSync / QSync-Ref / QSync-Post	Normal J.LocalGWR → J.GWR-Comp uses source QSync-Ref to obtain finite L2 settling. Standard QSync safety uses J.QSyncTraceInd over RTL K-origins, then Core.RTLCycleCarrier/J.CycleIdealCover to extend J.IdealPrefixClosed safety from cycle-aligned histories to all finite outer observable prefixes. Corollary J.1 cutpoint transfer and Appendix K use post-side QSync; the standard K-facing RTL audit includes QS2a origin recoverability in addition to QS1-QS5/QS3 realization. Core.QSync-KCutCanonical then identifies the complete reachable post-KCut domain. Whole-source Sys_B ^{elab} qualification also needs checked template/design settling composition.
J.QSyncTraceInd / J.QSyncCertInd	QSyncTraceInd is the safety-only base/one-cycle/exhaustive-successor certificate over CycleReach_post and carries only the exact LocalGWR/J.GWR-Comp + J.HCanon state needed by safety. QSyncCertInd is the richer K-facing induction carrying OfferReach/OfferConf/KObsConf, theorem-derived RegPhaseReach, and QC5 K.DrainReach. On the standard normal route, CI1 preserves the bound DR1-DR4 K.DrainReach-Comp applicability, re-establishes the successor attribution/history/QSync settling facts, and re-applies the lemma to obtain QC5; no separate DR5 history certificate or arbitrary successor DrainReach assumption is required. J.QSyncCertInd-Trace projects the richer induction to the safety form. For either induction, one observed simulation path is insufficient unless independent determinism/exhaustiveness collapses the reachable successor set.

Premise / evidence	Where it matters and typical discharge
J.LocalGWR	Consumed by the normal J.GWR-Comp route. Evidence: complete local process/channel/atomic/env/reset records, typed footprints, dependencies, agreement, and coverage. The optional J.GWR-Direct route may replace the local construction for a selected prefix only with a directly validated J.GWR package and J.ValidatedConstructionOrder satisfying all V1-V6 obligations, including the phase-normal V6 condition; the direct route is not a shortcut around those checks.
B-faithfulness / Core.ElabProj	Consumed by J/K at every explicit abstract boundary. Evidence: back-annotation, structural RTL checks, interface monitors, refinement checks, or tool certificates. Elaborated instances additionally need the coordinated Core.ElabProj package: Π_{elab} channel fibers, outer trace $\pi_{\Sigma, E}$, signed state accounting A_E , finite $\text{ElabToken}(E)/\Pi_{\text{token}, E}$ projection and functional transfer attribution, the complete stateless handshake equations, and proof-internal process-dependency visibility. Q.1(b) checks A_E against independently reconstructed outer Sys_B history at ϵ -quiescent cycle boundaries; the resulting $0 \leq A_E \leq B(c)$ range is derived from outer E4 rather than a separate qualification item. Q.1(j) requires the settled handshake and state-instantiated ReleaseOwnerSets family to be reconstructible from E plus the allowed reconstructed roots and requests, with the same schema/signals validated by J.KObsConf against the implementation. Q.1(k) separately requires exact family-level support soundness/no-spurious-rescue and completeness/no-missed-rescue relative to the declared template semantics before WaitOwners_E is formed. A local coupler is not a process vertex or residual offer.
Q.1(l) handshake support / CouplerPersistence	Triggered when the K.2b/Policy-S serial route consumes a coupling-sensitive boundary action. The template identifies finite earlier channel/history support plus persistent source-attributed request roots sufficient to determine the target ChannelEnabled result, and proves CouplerPersistence over the fixed-DomEpoch base frozen continuation: every permitted whole action before the target commits preserves that support or directly preserves ChannelEnabled at the next applicable settled opportunity. Q.1(d)-(k) well-formedness does not by itself establish this for mutual-stall, shared-capacity, or arbitrated/non-monotone handshake topologies. If the selected template cannot supply the Q.1(l) proof, the candidate is outside K.PSF/CF-S and requires another A5-L route. Evidence may be a reusable template invariant/proof, such as the sole-producer join persistence argument, bound to the concrete E instance.
J.TraceCertCov	Consumed by J.1-Safe over a selected RTL-prefix domain D. Evidence is route-sensitive. On the normal per-prefix route, each prefix carries accepted J.LocalGWR/J.GWR-Comp plus source QSync-Ref and remaining Post $\rightarrow$ Ref premises. For the standard cycle-aligned QSync domain CycleReach_post, J.QSyncTraceInd and J.TraceCertCov-QSyncCycle derive the package coverage; Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync then extend J.IdealPrefixClosed safety to all reachable finite outer observable prefixes without adding per-prefix packages. A property that fails J.IdealPrefixClosed still requires direct J.TraceCertCov on its needed arbitrary-prefix domain. Safety coverage does not imply K.CertCov.

Premise / evidence	Where it matters and typical discharge
Core.ReachPrep / Core.16 / K.Drain / Core.SettleKBridge / Kε	ReachPrep and Core.16/K.Drain qualify the actual source execution: once a fully disabled frontier settles, no source-later replenishing work may launch/Issue and only finite already-admitted work may progress. K.DrainReach/Core.SettleKBridge connect that reached activation to the selected K-facing cutpoint, with Kε covering only WFG-inert normalization. This is not the generalized modal drain theorem. Generalized DrainClosedNow is separately computed from FlightStateRec/InFlightStepRec and requires K.InFlightQual/J.InFlightPathSound. On the normal J.GWR-Comp route, DR1-DR4 and K.DrainReach-Comp supply the path-indexed reached-prefix facts; Q.2/InFlightQual supply the future already-admitted path abstraction.
J.OfferReach / J.OfferConf / J.KObsConf	Consumed by Corollary J.1 and all K cutpoint reasoning. Evidence must bind the exact same source/RTL prefix, witness, E/w/H/O, and NormRef/NormPost endpoints. Core.18 fixes the common typed record/equality interface; J.KObsConf is the theorem-level proof that J's KObsPost_J representation equals the realized source KObs and that the bound E handshake/release-support schema reconstructs the same MirrorAvail, ReleaseOwnerSets_E/WaitOwners_E, and WFG facts.
K.CertCov	Consumed by K.1A-Total and universal K.1 together with Core.KCutClosure_post. In the preferred standard QSync route, QS2a + Core.QSync-KCutCanonical identify the complete KCut domain, J.QSyncCertInd proves the full per-cutpoint package by base/one-cycle/exhaustive-successor induction, J.CertExist-QSync gives J.QSyncCertCov, and K.CertCov-QSync derives K.CertCov. The normal route first establishes the independent DR1-DR4 evidence once for the bound instance; CI1 then re-applies K.DrainReach-Comp at each exact successor rather than requiring unrelated per-cutpoint drain certificates. Alternate total-extraction/refinement/exhaustive routes remain valid. One CF-0 package, sampled KCuts, or a clean simulation is insufficient.
A5-L	Consumed directly by K.1A. Evidence: Policy-S route through K.2c or a valid structural/direct/invariant/exploration/tool proof.
Selected Policy-S run / SDet / A5-S	Consumed by K.2b/K.2c/K.1. The user artifact is one complete instrumented serial-issue run with total finite-bound response monitoring. The flow separately establishes the authoritative K.SerialRoutePremises applicability record. For direct K.2b, K.RespCov and separate K.CompleteS are supplied alongside that record. For K.2c/K.1, A5-S supplies K.RespCov, K.CompleteS, and K.RespClean for the same SDet-selected run; K.Low.A5 is additionally required when lowering transports the result back to Sys_ref. Universal RTL coverage remains separate through the QSync/QSyncCertInd package-totality layer.

Premise / evidence	Where it matters and typical discharge
K.EnvMF	Consumed by the serial route. Under StandardEnv/B1-available, candidates satisfying all K.JointFreeze-Ord ordinary/single-release-support premises use that theorem; every other admitted candidate - including a single frontier with a non-singleton or multi-alternative ReleaseOwnerSets_E family and every explicit multi-frontier - needs K.JointFreeze JF1/JF2. Under a timed/reactive or otherwise nondefault environment, every relevant process must have exactly the selected internal frontier and exactly one singleton release support together with the remaining K.JointFreeze-Ord premises; environment-facing co-frontiers, second internal frontier items, and non-singleton/multi-alternative release supports are outside the serial route. K.EnvMF-Uniform is the candidate-independent sufficient discharge when these structural conditions hold at every reachable K-facing cutpoint. In the common ordinary primitive case Core.OrdFrontSingleton, Dead_K membership, and Core.WFG's {{Q}} fact supply the structural part.
K.CV / K.OpKey / K.CVPred / K.CV-WF	Definition K.CV remains the authoritative deterministic serial-reduction provenance premise. Supply CV1 OpKey realization/source-order evidence and the CV2 K.CVPred/K.CV-WF common fact universe, finite predecessor sets, occurrence closure, deterministic evaluators, comparison-availability, recursive Core.16 reach dispositions, and well-founded relation/rank; add the observation determinism/isolation clauses when cross-process observations beyond ordinary Pop values affect control/payload/operation choice. This provenance machinery is separate from the generalized in-flight action/path qualification.
NB-Align / K.SerialRoutePremises	Definition NB-Align is an explicit Appendix-K condition separate from K.CV. Every witness-selected non-NB-CF non-blocking site that can participate in a potential K wait cycle must either be Appendix-L aligned so its chosen outcome dependency is tied to an explicitly modeled message/synchronization obstruction and supplier that can be lowered through K.Low, or be proved structurally unable to participate in such a wait cycle. Definition K.SerialRoutePremises is the authoritative K.2b applicability record: SR1-SR7 package constructive fairness, K.BF/ownership/release projection, K.CV/nonwithdrawal/K ϵ /K.Drain/CycleClosure, EnvMF/CompareAdvanceLegal, reset/terminal gates, fixed theorem choices, and NB-Align/K.Low where triggered. K.RespCov/K.CompleteS remain separate direct-K.2b inputs; A5-S supplies the response package for K.2c/K.1.
K.DomResetScope / K.InterruptionGate / K.ResetEpoch / K.TerminalDisposition	Consumed by the Policy-S serial reduction. Candidate-specific records must validate D1-D4. D2/D3 range over modeled process identities associated with source-level process-facing endpoints and with the selected elaboration's qualified process-dependency/release-support interface, plus every process-owned endpoint operation and explicit-boundary transfer identity consumed by K.2b.P0/R/P; stateless couplers are not process identities. Evaluate Reset first, then Terminal only after Reset Continue. Alternatively, one conservative uniform U1-U3 instance-level scope may prove all required Continue branches at once. DirectFail remains the independently mapped RW2 cancellation or DeclaredTerminal maximal-pending failure path.
L.Dir and W1-W5	Consumed by the online snooping harness. Evidence: free-running RTL, ordinal recording/consumption, comparison/watchdog, noninterference, and complete choice-point coverage.

Premise / evidence	Where it matters and typical discharge
ClockScope - single global clock	Applies to the G-K theorem stack and directly discharges QS1 evidence. The selected theorem instance uses one global synchronous cycle relation; CDCs are excluded from that instance or represented through separately specified/verified crossing components. Appendix O is only an informative multi-clock sketch until a separate semantic layer is supplied.
Core.SyncFence / R3/E5 completion fence	Core.SyncFence is constitutive synchronization-completion legality on Sys_ref and a separate RTL conformance obligation. Every blocking q in pref_S(s) has a designated process-facing commit no later than sync s. Same-cycle q/s completion is allowed; on RTL, ordering “before interval advance” is annotation/certificate accounting rather than physical sub-cycle timing. Typical evidence: R3/E5 trace checks, control-state assertions, boundary mapping, or formal refinement.
Core.MemSyncOrder	Separate local scheduling/mapping qualification whenever an array/memory access is source-ordered with a selected synchronization call, whether storage is internal or external. Verify mapped storage commits before s no later than s and after s strictly later, including any cache/merge/split/reorder transformation evidence. A single merged storage commit may not represent accesses on opposite sides of the same synchronization boundary. It does not add memory events to Σ or become an Appendix I/J.1 replay premise; J.Mem is separately required for qualifying cross-process shared-memory value flow, and Core.MemFaithful is additionally required when transformed physical traffic is hidden below the selected proof boundary.
Core.KCutClosure_post / Core.KCutObserve	Consumed by K.1A-Total and universal K.1, separate from K.CertCov. In the standard user-facing scope, Core.KCutClosure_post is a theorem-derived interface: accepted RTL-side Core.QSync evidence invokes Core.QSync-Post, giving a finite same-cycle normalization to a unique PostKCut before the next registered update. $K\epsilon$ plus Core.KCutObserve preserves any persistent relevant request/frontier/enablement configuration into that KCut. The former arbitrary invariant/checked-normalization routes are not independent standard-scope discharges; a non-QSync model needs a separately qualified extension.
Core.9/Core.10/Core.IC - Issue/BoundaryReq lifecycle conformance	Core.9 makes Issue(q) the first cycle in which the calling process begins requesting q at the selected endpoint; an annotation may carry but may not choose that cycle. Core.10 makes BoundaryReq a fixed-mapping, operation- and boundary-specific process-owned request; raw valid/ready levels representing complementary channel/storage availability or coupler logic are not BoundaryReq merely because they are high. Core.IC is constitutive Sys_ref transition legality. On RTL_B, establish bidirectionally complete owner decoding; live-owner uniqueness; stable blocking request/payload persistence from Issue until process-facing Commit or affecting RW2 RESET; and back-to-back ownership: if q0 commits at t and the same process-owned request continues at t+1 as the next same-direction blocking q1, q1 owns it and Issue(q1)=t+1, with no ownerless interval. Genuine delay is only before q1 BoundaryReq begins. Evidence may be interface assertions/monitors, schedule/control-state evidence, request-attribution certificates, or formal refinement. K.BF consumes this persistence but does not own the conformance obligation.
K.BF - blocking-only serial fragment	Consumed by K.2b/K.2c/K.PSF/CF-S on the bounded-response serial route. In addition to citing established Core.IC/RW2 persistence, show every WFG-relevant wait is an ordinary blocking Push/Pop on a finite B-faithful point-to-point boundary, local computation cannot diverge before the process-facing frontier, and shared arbitration

Premise / evidence	Where it matters and typical discharge
	is explicit and deterministic or witness-fixed. This is a K-route applicability condition, not the definition of request persistence itself.
Core.CycleState / Core.CycleResult / Core.CycleClosure / Core.CycleChoiceExt / Core.FairPick	Required when K.2b.E constructs a complete/fair source continuation, not by finite-prefix J.1. Bind the model-indexed Core.CycleState entry state, full Core.CycleResult next-state semantics, and Core.CycleClosure C2 partial-cycle extension. Core.FairPick packages the existing Core.IssueFair/B2/B3 discharges as prefix-local issue/boundary selectors with starvation-freedom and fixed-scheduler/environment legality; Core.FairCycleDovetail uses two CycleChoiceExt completions around the actual L2 SettlePoint to produce each typed CycleResult. B5 remains separate.
Core.SelectLatitude	Finite-prefix source-model qualification used by Post -> Ref whenever an otherwise legal L1 issue or L3 boundary action may be postponed. The abstract Sys_ref discharges it definitionally; a concrete executable source/reference model used as the theorem carrier must prove it does not impose hidden maximal progress that excludes the accepted J construction. Nonselection is legal under both Policy L and Policy S, remains subordinate to Core.16 and all other legality, and is separate from Core.IssueFair/B2. The M.7a entry also warns that J.RefProjCover needs latency coverage only when different legal delays can change canonical Σ_{DUT} history.
Core.MemFaithful	Required when mapped-memory channels/signals or transformed physical memory activity are hidden below the selected J/K boundary. Bind the declared untransformed logical mapping-layer semantics as reference and prove the transformed realization preserves the selected Core.17/E1-E5 event identities, values, required order, and atomicity. Selected message/signal events cannot be selectively hidden. For K, hidden realization must also preserve the represented process-facing blocking/frontier/enablement/release-support/WFG facts; persistent backpressure therefore needs a K-faithful logical boundary or is outside the transformed K scope.
J.Mem	Required for every shared-memory dependency through which one process's write can affect another process's control flow, payload/value expressions, or the identity/order of later operations. Discharge at dependency level, not per physical RAM access, but prove completeness: every qualifying path is lowered to covered proof-visible operations (including a qualified logical memory boundary) or the separate memory-state extension, and no other qualifying unlowered memory-carried path remains.
Core.MemProfile	Used for a reusable logical memory boundary above transaction-changing cache/merge/split/reorder behavior, including transformed cross-process memory and persistently backpressuring memories whose raw transaction stream cannot be the E4/Core.17 boundary. The profile binds the logical boundary/alphabet and transformation set; discharges Core.MemFaithful, J.Mem, Core.MemSyncOrder, and the K WFG-inert/K-faithful disposition; enforces A6 explicit arbitration/point-to-point ownership, Appendix Q.1 where the exposed boundary is Sys_B^elab, and RW reset disposition; and binds the qualified profile to the instance through K.CertHdr. If a K-faithful transformation-invariant boundary cannot be supplied for a persistently backpressuring realization, disable the transformation or exclude that memory from K scope.
Q.2 / Core.InFlightDrainWF / Core.InFlightCycleExt / K.InFlightQual / J.InFlightPathSound	Required whenever generalized DrainClosedNow/Dead_K is used. Bind the exact model/profile Q.2 action/effect schema, Q.InFlightActionEffectSound (and Q.SyncInFlightActionEffectSound when selected), Q.InFlightChoiceFinite, finite decreasing Core.InFlightDrainWF, and either Core.InFlightCycleExt, full model-indexed Core.CycleClosure, or an equivalent checked opportunity theorem.

Premise / evidence	Where it matters and typical discharge
	J.InFlightPathSound must realize every reconstructed finite path with exact successors and prefix-compositional profile closure. Omitted genuine actions are conservative but reduce precision; fictitious actions/effects/opportunities are unsound because they can fabricate escape. Worked artifacts A-C are examples, not concrete-template qualifications.
Core.SyncCarrierWF / K.Low.ModalQual MQ1-MQ6	Triggered by profile G2 lowered generalized-drain claims. Core.SyncCarrierWF types synchronization independently of MsgOf/WFG, including exact participant/readiness/release-support reconstruction; SyncFire is the source transaction and lowering-only guard/epoch actions are separately typed. ModalQual binds exact lowering rule/action schemas, finite LowAdminNormalize/rank, LowStateCorr/normalized RootCorr, MQ4 bidirectional action projection/lifting plus finite-administrative-suffix root reflection, separate Sys_K and Sys_ref K.InFlightQual (or named preservation of specific fields), and K.CertHdr schema/checker/version binding. Sys_K CycleClosure does not silently establish Sys_ref continuation evidence.

11.1 A practical certificate boundary

A useful implementation is a versioned package with a common header binding every artifact to one theorem instance: source and RTL build IDs, selected Sys_ref/Sys_K and any concrete executable source/reference carrier, explicit boundaries/capacities/elaboration, and—when Sys_B^elab is used—the coordinated $\Pi_{\text{elab}}/\pi_{\Sigma,E}/\Pi_{\text{token},E}/\text{signed-A}_E$ Core.ElabProj data plus qualified stateless-handshake and ReleaseOwnerSets/WaitOwners template; stimulus/environment/reset policy; witness; Core.SelectLatitude carrier-conformance reference where applicable; Core.MemFaithful/Core.MemSyncOrder/J.Mem/Core.MemProfile references when triggered; Core.IC/Core.SyncFence conformance references; QSync evidence provenance; and the exact Core.18 KObs package. For standard universal QSync coverage it should also bind the QSyncCertInd base/step generator or proof, exhaustive-successor justification, and the DR1-DR4 evidence from which K.DrainReach-Comp discharges normal-route QC5. For Policy-S it should additionally bind the OpKey/CVFact universes, K.CV-WF plus comparison-availability/recursive Core.16 dispositions, Core.CycleState/Core.CycleResult/Core.CycleClosure and FairPick evidence, K.EnvMF/K.EnvMF-Uniform where applicable, JointFreeze/CompareAdvanceLegal references, either candidate-specific DomResetScope/gates or one uniform U1-U3 record, response-monitor configuration, run completeness, and waivers. The checker should reconstruct derived state and validate subordinate evidence rather than accepting a final Boolean claim. Appendix M.7b standardizes the logical grouping of this evidence without prescribing a physical file format.

SIGNOFF RULE: Do not mix witnesses or cutpoints

A J.OfferReach proof for one source state, a J.OfferConf proof for another RTL cycle, and a K.Drain proof for a different witness do not compose. The formal architecture repeatedly requires exact identity of the theorem instance, stimulus, elaboration, witness, H, O, source prefix, RTL prefix, and normalized cutpoints.

11.2 Standard QSync certificate profile

Appendix M.7b provides a normative packaging profile over the detailed M.7a discharge ledger for the standard normal-compositional QSync flow. The four names below are logical certificate interfaces only: they do not introduce new theorem predicates, merge or weaken obligations, make required conditions optional, or prescribe one physical file format. M.7a remains definitive. A bundle may reference theorem-derived facts instead of regenerating them, and every artifact used in one claim must map unambiguously to its M.7a obligations and share the same K.CertHdr/K.CertPkg identity/trust binding.

Logical bundle	What it owns in the architecture
----------------	----------------------------------

Logical bundle	What it owns in the architecture
1. Instance and QSync qualification	The theorem-instance identity and structural/scheduling/mapping qualifications: ClockScope; source/RTL QSync QS1-QS5 (including QS2a when K-facing); Core.RegSeq/ReachPrep/Core.SelectLatitude where triggered; triggered Core.IC/Core.SyncFence conformance; R1-R5/E1-E5; selected boundaries, capacities, B-faithfulness/Core.ElabProj; applicable Core.MemFaithful/Core.MemSyncOrder/J.Mem/Core.MemProfile; and reset, direct-input, and elaboration/template-composition qualification. For Sys_B^elab this includes $\Pi_{\text{elab}}/\pi_{\Sigma,E}$, finite ElabToken(E)/ $\Pi_{\text{token},E}$ attribution, signed A_E agreement with outer E4 history, the finite stateless handshake network, and qualified ReleaseOwnerSets_E/WaitOwners_E process-dependency projection, with couplers remaining non-process realization detail. The legal nonnegative A_E range is theorem-derived from that agreement plus outer E4 and is not an extra bundle item. It may cite QSync-Ref/QSync-Post/QSync-KCutCanonical after their inputs are checked. It does not itself establish J correspondence, KObs equality, K.CertCov, or A5-L.
2. J correspondence/refinement	J.LocalGWR/J.GWR-Comp or a permitted J.GWR-Direct package with independently checked V1-V6, WC/witness binding, and J.HCanon. Safety uses QSyncTraceInd/TraceCertCov-QSyncCycle and ideal closure as applicable; K-facing use additionally binds the exact OfferReach/OfferConf/KObsConf/RegPhaseReach/DrainReach package. Theorem J.1's restricted Ref $\rightarrow$ Post clause is selected-pair compatibility over an already supplied RTL-consistent source/RTL pair, not a constructive RTL-existence result.
3. K applicability/provenance	The qualifications that make K sound beyond J correspondence: K.BF/request persistence and ownership; exact ReleaseOwnerSets/WaitOwners projection; authoritative K.CV/OpKey/CVPred; independent K.Drain/K ϵ /no-replenishment evidence for the reached prefix; the generalized Q.2/K.InFlightQual action/path layer for every proof model on which Dead_K is interpreted; release-aware EnvMF/JointFreeze; CompareAdvanceLegal; source cycle/fairness evidence; reset-first interruption; and applicable NB-Align/waivers/lowering. For G2 this bundle includes Core.SyncCarrierWF and K.Low.ModalQual MQ1-MQ6 with both Sys_K and Sys_ref K.InFlightQual. K.SerialRoutePremises packages the serial-reduction subset of these fields but does not manufacture the source generalized-drain qualification or A5-L.
4. Policy-S response	For the primary bounded-response A5-L discharge: the SDet-selected Policy-S run and monitor semantics, K.CompleteS, total finite-bound K.RespCov, K.RespClean, response waivers, and CF-S/A5-S. This bundle references the applicability/provenance bundle rather than duplicating it. In G1 K.2c yields source A5-L directly. In G2 K.2c yields A5L_Sys_K and the K applicability bundle's completed K.Low.ModalQual/K.Low.A5 transport yields the source A5-L consumed by K.1A.

11.3 How the bundles compose by claim

Claim	Bundle composition
Standard QSync functional safety	Bundles 1 + the safety form of 2. J.QSyncTraceInd supplies cycle-aligned packages and Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync supply all-finite-prefix closure for J.IdealPrefixClosed properties. Bundles 3 and 4 are not safety premises. If the K-facing form of bundle 2 already exists, QSyncCertInd-Trace supplies the safety induction.

Claim	Bundle composition
One selected K.1A cutpoint	The applicable instance qualification, exact K-facing J/cutpoint package, Sys_ref generalized K.InFlightQual, and any valid source A5-L discharge. If that A5-L was produced through a G2 serial/lowering route, the upstream A5 certificate also binds K.Low.ModalQual/generalized K.Low.A5, but the K.1A cutpoint package itself remains a Sys_ref package. Universal QSync package induction/K.CertCov is not required merely for one already selected certified KCut.
Universal Theorem K.1 via Policy-S	All four bundles. Checked QSync structure provides the canonical KCut domain; J.QSyncCertInd -> J.CertExist-QSync -> K.CertCov-QSync provides package totality. Bundle 3 binds the generalized-drain action/path qualifications: one same-model record in G1, or Sys_K + Sys_ref records plus K.Low.ModalQual in G2. Bundle 4 and K.2b/K.2c discharge A5-L before K.1A-Total.
Universal K result with alternate A5-L route	Bundles 1-3 as triggered plus the alternate K.A5Cert route payload in place of bundle 4; the correspondence, KCut-domain, and K.CertCov obligations remain unchanged by choosing a different proof of A5-L.

This packaging is intentionally an API/user-interface simplification rather than a semantic simplification. A user-facing flow may show four top-level statuses with drill-down to M.7a, but an independent checker must still validate the referenced subordinate evidence and recompute theorem-derived or KObs-derived facts where required.

12. Common misunderstandings

Misreading	Correct interpretation
“Trace compatible” means cycle accurate.	No. It preserves selected labels, values, ordering, atomicity, channel semantics, and anchors. Independent events may move between cycles.
The source pass transfers by Ref -> Post.	For ordinary safety, the deductive path is Post -> Ref plus J.1-Safe: RTL adds no covered canonical safety behavior absent from Sys_ref. Ref -> Post is restricted to annotated RTL-consistent or witness-selected source prefixes.
Per-process replay certificates already imply a system replay.	No. Shared channel state, snapshots, atomic participants, resets, and field interference can conflict. J.GWR-Comp proves the merge.
The causal-dependency graph is the J induction order.	No. Appendix-G causal dependency explains information flow and may contain legal atomic cycles. J.DepSound applies only to D1-D4 semantic prerequisite edges; D5 is proof stratification and uses J.HorizonOrderSafe commutation evidence. J.FoundationRank/J.DepRank prove the generated J graph acyclic. Appendix K's K.CVPred/K.CV-WF is a third, separately well-founded provenance order and must not reuse the broad causal graph.
KObs is a normalized execution or full state.	No. Core.18 makes the cross-model cutpoint seam equality of the typed record <E,w,H,O>. The source record is defined by Core.18c; Appendix J constructs KObsPost_J and J.KObsConf proves typed equality for the exact selected package. Core.18d/e reconstruct K-facing facts from that record; historical Issue timing and arbitrary RTL/source microstate are not equated.
No-reverse scheduling proves deadlock freedom.	No. It prevents one deadlock-introduction mechanism. Timing overlap, finite capacity, pipeline drain, environment behavior, and serial-shaped source deadlocks require Appendix K.

Misreading	Correct interpretation
One clean Policy-S simulation is automatically a proof of A5-S.	No. The primary workflow uses one selected run, but the serial reduction also requires Definition K.SerialRoutePremises, which binds SDet, constructive fairness, K.BF/ownership/release projection, authoritative K.CV and its OpKey/K.CVPred/K.CV-WF certificate, nonwithdrawal/K ϵ /K.Drain/CycleClosure, EnvMF/JointFreeze, CompareAdvanceLegal, reset/terminal gates, fixed-instance choices, and NB-Align/K.Low where triggered. A5-S separately supplies total K.RespCov, K.CompleteS, and K.RespClean for K.2c/K.1. A universal RTL claim additionally needs the QSync/QSyncCertInd package-totality route or another valid total-coverage proof.
Core.LAdm adds a new pipeline-drain requirement beyond Appendix B.	No. Core.Phase/Core.LAdm formalize the already-required freeze/drain behavior. Core.16's 'launch' restriction is intentionally broader than message Issue, and K.Drain supplies the finite non-replenishing cleanup evidence; capacity alone does not prove finiteness. Core.SettleKBridge connects the L2 activation to the selected later drain-closed KCut.
The Policy-S reduction is already a certified theorem.	Not as a mechanized and flow-qualified theorem. The scheduling-rules document supplies explicit paper proofs of K.Low.C/K.Low.A5, K.2b.P0/P1/R/P/E/Q and their assembly, including common operation keys, well-founded provenance, typed cycle extension, and nested reset/terminal gates. Lean mechanization, simulator/lowering/QSync qualification, reusable certificate validation, and package-coverage realization remain outstanding.
Another A5-L route can rescue a serial deadlock.	No. If an admissible Policy-S-shaped Policy-L prefix reaches Dead_K^W, A5-L is false for the unchanged instance.
Snooping changes the specification to match RTL.	No. It selects one behavior already admitted by the under-specified source latency/epsilon semantics.
A live AND is enough for snooping.	No. The wrapper must record per-ordinal RTL events and retain them until source consumption; otherwise events can be dropped or reordered.
Tool generation automatically discharges the proof obligations.	No. The tool or flow must emit documentation, certificates, static checks, monitors, or formal evidence for each named obligation.
A synthesizable and functionally working design is necessarily within the theorem scope.	No. Hidden timing dependence, ambiguous signal anchors, unsupported shared-memory value flow, unmatched reset disposition, fall-through/bypass behavior, or coupled boundary structure can place a correct implementation outside the selected formal model. Appendix F is the architect-facing applicability screen.
A fall-through FIFO is just an ordinary FIFO with a particular capacity.	No. Same-cycle empty-to-consume or full-to-accept behavior changes the channel transition semantics. The bypass or fall-through path must be represented explicitly in Sys_ref and covered by the boundary/elaboration evidence.

Misreading	Correct interpretation
Dynamic reset is covered whenever reset is supported by the design.	No. Safety requires matched reset scope/state/request/in-flight disposition. The Policy-S route additionally needs a K.DomResetScope and the prefix-local K.ResetEpoch gate; RW5-disjoint resets can remain legal, but an intersecting reset must either be excluded by Continue or satisfy the independently mapped DirectFail cancellation conditions. Only after Reset Continue is K.TerminalDisposition considered.
Failed non-blocking polls are invisible, so they cannot affect safety.	Their failure is not a committed Sigma event, but the outcome or cadence may select later observable control, values, requests, payloads, or endpoints. Such sites require NB-CF, a valid WC/L.Dir witness, isolation, or an explicitly modeled timing-sensitive protocol.
Input FIFO depths in a joined pipeline are independent available slack.	Not when acceptance is coupled through a join, bundle, shared stage, stateless coupler, or epoch gate. B(c)/occupancy still describes the represented storage legality, but the actual settled handshake is determined by the declared Sys_B^elab network. For K-facing reasoning the same qualified template must also expose the modeled-process ReleaseOwnerSets_E/WaitOwners_E dependency projection; independent input capacities are therefore not automatically independent release slack.
K.Resp is an RTL latency guarantee.	No. It is source-side evidence used to discharge A5-L through serial dominance.
Any epsilon-quiescent state is a KCut.	No. Core.KCut additionally requires reachability and a settled explicit-boundary request/enableness fixed point. Source L2 SettlePoints satisfy this by Core.SettleIsKCUT; Core.QSync-Ref supplies their existence/uniqueness in the standard scope, while Core.QSync-Post derives the RTL KCut-closure interface.
K.CertCov covers every RTL prefix or every normalization witness.	No. K.CertCov is total only over reachable RTL KCuts. For each KCut it may exhibit one reachable-prefix/NormPost pair and certified package. In the standard QSync route, QS2a/Core.QSync-KCutCanonical identify the complete canonical KCut domain and J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync derive package totality. Arbitrary transient prefixes and arbitrary normalization witnesses need not carry packages.
QSync all-finite-prefix safety requires the KObs/drain package.	No. J.QSyncTraceInd deliberately carries only exact-prefix J.LocalGWR/J.GWR-Comp plus J.HCanon and excludes OfferReach/OfferConf/KObsConf/K.DrainReach/K.RLAdmReach. It first establishes cycle-aligned packages; Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync then extend J.IdealPrefixClosed safety to all reachable finite outer observable prefixes. If J.QSyncCertInd is already proved for K, it projects to the safety induction.
One observed QSync simulation proves universal K.CertCov.	No. J.QSyncCertInd requires CI1 for every reachable next-cycle successor and CI2 exhaustive successor coverage. A determinism theorem may collapse that set, but B1 alone does not justify ignoring unobserved internal successors.

Misreading	Correct interpretation
Delta/tau/idle are just three names for epsilon.	No. Delta is same-cycle settling; a SilentCycle is a real CompleteCycle that may advance registered/environment/monitor state; and a B6 idle tick is permitted only after Core.TerminalIdle proves permanent machine+environment no-future-enabling behavior. A present cycle with no selected commit is not automatically terminal stutter.
Core.QSync makes the source model deterministic.	No. QSync gives a unique settled result for fixed cycle-entry state, inputs, and explicit drives/choices, and QS3 requires the operational settling relation to be realized by that network. RTL QS2a is an origin-recoverability condition for reachable KCuts, not a global execution-determinism theorem. WC-sensitive outcomes, Core.SelectLatitude under Policy L or Policy S, arbitration, and reactive-environment choices may still admit multiple source executions; SDet is a separate selected Policy-S-run premise.
Core.KCutClosure_post is still an arbitrary per-instance normalization proof in the standard flow.	No. The downstream interface is unchanged, but the standard QSync scope derives it through Core.QSync-Post from audited structural facts. An alternative non-QSync settling semantics requires an explicitly defined and separately qualified theorem extension.
Same-cycle Core.SyncFence requires a physical RTL sub-cycle ordering.	No. The physical E5 requirement is cycle-granular: the predecessor process-facing commit occurs no later than the synchronization commit. The equal-cycle "accounted before interval advance" rule is the proof/certificate attribution convention used to maintain source-interval semantics.
K.CV can use the Appendix-G causal graph as its induction order.	No. K.CV is the authoritative design-level causal/value-determinism premise, and Definition K.CVPred/K.CV-WF supplies its separate fact-key provenance certificate with explicit predecessor occurrence closure, deterministic evaluators, comparison-availability, and a well-founded relation/rank. K.CVPred/K.CV-WF do not presuppose K.CV, but their relation also does not inherit the broad Appendix-G causal graph. Same-cycle rendezvous enablement and broad causal/temporal proximity do not become provenance edges automatically.
Core.16 'launch' means only endpoint Issue.	No. The 15 August clarification makes launch intentionally broader: any source-later preparation/reach or other work capable of replenishing the blocked component is covered by the no-new-later-work discipline.
"Prefix-closed" means ordinary prefix closure of one linear event log.	No. In the safety theorems it means J.IdealPrefixClosed: downward closure over finite ideals of the retained required-order relation. A cross-interface scoreboard may fail this condition even if an engineer informally calls it a safety check; use direct J.TraceCertCov on the required arbitrary-prefix domain or retain the needed dependency in the observation/order.
Core.IC and Core.SyncFence are trusted axioms that can simply be assumed for RTL.	No. They are constitutive legality/invariant clauses of Sys_ref. The corresponding RTL request lifecycle, attribution, process-facing completion, and synchronization-fence behavior must be established by implementation-side conformance evidence and recorded through the M.7a audit entries.
Commit is defined as valid-and-ready being high in every proof model.	No. Core.8 defines the abstract boundary commit as the legal E4 channel-commit transition. Simultaneous mapped valid/ready and payload sampling are the RTL_B realization/conformance of that abstract event. Commit(q) is the designated process-facing completion/release occurrence, and returned-result availability remains a later Core.Phase L4/Core.RegSeq matter.
Finite-prefix J.1 delays an enabled source action by assuming B2	No. Core.SelectLatitude makes finite nonselection of otherwise legal L1 issue and L3 boundary actions constitutive source legality under both policies.

Misreading	Correct interpretation
fairness.	Core.IssueFair and B2 are separate complete-execution requirements that prevent indefinite deferral; neither is imported into finite-prefix J.1.
A Core.SilentCycle used between J Horizon buckets leaves the full source state unchanged.	No. SilentCycle is a real CompleteCycle. Registered/internal state, environment state, reset bookkeeping, requests/witness state, and cycle-valued monitors may evolve. J.UnitExt must preserve or re-establish its induction invariant through the typed cycle transition; only the selected-unit ideal/CanonHist is unchanged when no selected J unit commits.
An empty RTL Horizon becomes an empty J.CanonicalRank bucket, or fixes the same number of source idle cycles.	No. Height/FR/DepRank/DepAcyclic rank actual construction actions in Act_L(τ_{post}), so empty Horizons create no rank elements. Any required source-cycle gap is a finite evidence-determined unit-empty CompleteCycle bridge under Core.SelectLatitude, and its length is not the raw RTL Horizon-index difference unless the selected cycle-sensitive evidence requires that alignment.
Mapped external RAM traffic may always be hidden from Σ once a higher boundary is chosen.	No. Hiding is an admissible use of the existing observation-boundary freedom only when Core.MemFaithful holds against the declared untransformed logical mapping semantics. J.Mem still requires complete representation of every qualifying cross-process memory dependency, and a persistently backpressuring K realization needs a transformation-invariant K-faithful logical boundary (normally Core.MemProfile) or the transaction-changing transformation is outside the covered K instance.
A high ready/valid level can be annotated as whichever BoundaryReq/Issue makes E3 work.	No. Core.9 Issue is the operational first-request cycle, not a witness-selected timestamp. Core.10 BoundaryReq is fixed-mapping, process-owned, operation- and boundary-specific. A raw high valid/ready level that represents complementary channel/storage availability or declared coupler logic is not BoundaryReq. Core.IC also requires one live owner for each asserted blocking BoundaryReq and forbids an ownerless back-to-back request interval.
If an operation or boundary action is enabled only intermittently and never selected, Core.IssueFair/B2 is violated.	No. These are weak-fairness conditions with continuous-enablement antecedents. Core.IssueFair applies when a reached operation remains IssueEnabled at every applicable L1 opportunity; B2 applies when an issued boundary action remains continuously ChannelEnabled. Intermittent enablement is outside those weak-fairness guarantees by design, not a violation. If source control never reaches the operation, that is a separate source-control case.
Core.DeclaredTerminal and Core.TerminalIdle are the same terminal condition.	No. Core.DeclaredTerminal is a modeled machine/test termination, abort, or end-of-test predicate; an external host watchdog is not enough unless modeled. Core.TerminalIdle is the stronger B6-eligible global fixed/no-future-enabling condition for machine plus environment. TerminalIdle can also hold for a nonterminating service model proved permanently idle after stimulus exhaustion.
Any persistent nonprogress involving a message channel is the Appendix-K closed internal deadlock.	No. K.Dead is a specific reachable drain-closed, release-aware internal wait-cycle predicate. A continuously enabled but unselected boundary action is a B2 fairness failure; lack of required complementary progress is a B3 progress failure; intermittent enablement/starvation can lie outside both weak-fairness antecedents; and environment-only or source-control stalls are also distinct. Appendix K excludes only its defined non-waived closed internal wait-cycle class under the stated premises.
DrainClosedNow means there is no	No. That is only the legacy primitive fast-path intuition. Generalized

Misreading	Correct interpretation
enabled non-frontier residual offer.	DrainClosedNowRec is the absence of any finite qualified InFlightStepRec path that changes WaitRoots_D. K.PrimitiveDrainExact gives equivalence with the residual-offer selector only under its restricted primitive profile.
The drain search should consider only actions owned by processes in D.	No. D scopes only the root family being compared. Core.InFlightStepRec deliberately has no D/ownership/relevance filter. An already-admitted action outside D must be explored if a finite path through it can change a D root; the two-stage/join artifacts are direct regressions against a D-scoped action relation.
Stage 2 puts synchronization into MsgOf, BoundaryReq, or the literal WFG.	No. Core.7b/Core.7c create a parallel synchronization carrier with SyncOf/SyncTxn/SyncReleaseOwnerSets. Literal message BoundaryReq/Front/WFG/Dead_K remain message-only. SyncFire and lowering gate actions participate only in the generalized in-flight/lowering proof infrastructure.
If the serial Sys_K model has Core.CycleClosure, the lowered source Sys_ref path theorem follows automatically.	No. Continuation/path premises are model-indexed. In G2, Sys_K CycleClosure can discharge only the Sys_K opportunity/path field unless a named preservation theorem is proved. K.Low.ModalQual explicitly binds a separate Sys_ref K.InFlightQual.
Equal KObs proves future in-flight progress.	No. KObs reconstructs current state and local Q.2 action/effect applicability only. Core.InFlightCycleExt/J.InFlightPathSound separately establish that reconstructed edges and finite paths have legal operational realizations. B2/B3 cannot bootstrap that existence because their eventuality is interpreted along an execution whose existence is already assumed.

13. Quick-reference index

PURPOSE	Provide a compact map from theorem/definition names to their role in the proof architecture. Engineer takeaway: Use this section as a navigation aid when reading the normative scheduling-rules document.
----------------	---

13.1 Normative Formal Core and Appendix G

Name	Role
Core.8 Commit / CommitCycle	Abstract boundary commit is the selected model's legal E4 channel-commit transition; RTL mapped valid/ready/payload is the implementation realization. Commit(q) is the designated process-facing completion/release occurrence and CommitCycle(q) its cycle index; returned-result availability remains separate under L4/Core.RegSeq.
Core.RegSeq	Registered next-cycle dependence rule: cycle-t sequential interface outputs derive from registered state/fixed operands; no same-cycle newly returned interface result feeds a dependent sequential output.

Name	Role
Core.Settle / delta-settling	Complete source L2 same-cycle settling relation and its fixed-point endpoint. The L2 frame freezes cycle index, registered state, selected cycle inputs, and stable sequential drives; Definition Core.Settle itself does not prove termination.
Core.QSync / QSync-Ref / QSync-Post	Qualified one-global-clock same-cycle model. QS1-QS5 make settling finite, deterministic, acyclic over explicit state boundaries, boundary-complete, and QS3-realized by the selected operational relation. For the standard RTL K-facing scope, QS2a recovers a unique same-cycle K-origin/frame for every reachable post KCut. QSync-Ref supplies source SettlePoint/KCut existence; QSync-Post supplies the canonical RTL endpoint/closure; QSync-KCutCanonical identifies the complete reachable KCut domain.
Core.ReachPrep / Core.9 / Core.10 / Core.IC / Core.PolicyL / Core.LLegal / Core.SelectLatitude	Overlap-aware source preparation/reach; operational Issue; process-owned BoundaryReq; constitutive blocking-request lifecycle/attribution; permissive issue policy/fixed-instance step legality; and finite-prefix non-maximal selection. Reach may precede an earlier blocking return only when dependency-independent and certified; it remains distinct from Issue. Core.9 Issue is the actual first request cycle. Core.10 BoundaryReq is operation-specific under a fixed bidirectionally complete mapping and excludes raw complementary availability/coupler levels. Core.IC supplies live-owner uniqueness, nonwithdrawal, and back-to-back ownership with Issue(q1)=t+1 when the continued process-owned request transfers to the next same-direction blocking operation after q0 commits at t. Core.SelectLatitude permits otherwise legal L1/L3 actions to remain unselected at a finite opportunity but cannot create an ownerless already-started BoundaryReq. Source semantics are definitional; concrete RTL/source carriers are separately qualified where applicable.
Core.Phase / Core.LAdm / Core.CycleState / Core.CycleResult / Core.RTLCycleCarrier / Core.CycleClosure / Core.FairPick	L0-L4 phase structure and liveness-admissible source prefixes/executions. Core.CycleState is the model-indexed cycle-entry theorem state; Core.CycleResult is the typed real-cycle outcome carrying the next state and selected units. Core.RTLCycleCarrier completes an already-entered legal RTL cycle to its successor K-origin; Core.CycleClosure supplies the source partial-cycle completion used for complete continuations; and Core.FairPick/Core.FairCycleDovetail provide constructive selector/dovetail evidence when K.2b.E builds the continuation. Core.CycleClosure C2 uses selected finite/non-maximal L1/L3 endpoints consistent with Core.SelectLatitude. Core.IssueFair/B2/B3 remain separate behavioral fairness; RTLCycleCarrier is not generic RTL progress.
Core.PolicyS	Conservative serial-blocking issue policy used by the primary instrumented liveness route. It inherits Core.SelectLatitude: an otherwise legal earlier issue need not be selected at the first finite opportunity. Core.IssueFair prevents indefinite omission on complete admissible executions, while SDet separately fixes the concrete selected Policy-S simulator/scheduler run used by the reduction.

Name	Role
Core.ElabProj / Core.WFG / ReleaseOwnerSets / Core.16 / Core.Drain / Core.SettleKBridge / generalized in-flight Core	Core.ElabProj coordinates explicit elaborated storage/token/accounting, settled handshake, and modeled-process release projection. Core.16/Core.Drain/Core.SettleKBridge govern the reached no-replenishment drain from an L2 blocked activation to later K-facing states. Generalized drain adds Core.InFlightStep (already-admitted complete action units), Core.InFlightDrainWF (finite decreasing admitted work), Core.InFlightCycleExt (restricted same-/cross-cycle opportunity), and Core.18 FlightStateRec/InFlightStepRec reconstruction. The reconstructed step relation has no candidate-D filter; D is introduced later only through K.WaitRoots. Capacity supports finiteness but is not the well-founded decrease or future-existence proof.
Core.OrdFrontSingleton	Ordinary non-elaborated source intervals have at most one pending blocking frontier item; genuine multi-frontiers require explicit Sys_B^elab partial-order structure.
Core.18 / Core.18c-e KObs / Core.9-Core.10 Issue & BoundaryReq	Core.18 defines cross-model correspondence as equality of one common typed KObs record <E,w,H,O>; Core.18c defines the source record and Core.18d/e reconstruction functions. J defines KObsPost_J and proves equality through J.KObsConf. Core.9/Core.10 and Appendix C now make Issue/BoundaryReq operational rather than witness-selectable: Issue(q) is the first process-owned request cycle, BoundaryReq is the fixed-mapping operation-specific process-owned request, and Core.IC supplies live-owner/nonwithdrawal/back-to-back attribution. Historical Issue/ReturnedAtCycle timestamps remain outside KObs fields.
Core.17 ΣMatch / R1-R5 / E1-E5 / causal dependency	Core.17 supplies canonical complete selected occurrence/value matching. Appendix G expands it with R1-R5/E1-E5. For sequential signal I/O, each anchor records at most one Read and at most one Write per (process, signal); if several source Writes share the same E2 anchor, the last executed source Write supplies the canonical value and earlier writes are epsilon-internal. R2C fixed-point units match by component identity/local emitted-occurrence ordinal rather than a global cycle bucket; explicitly selected proof-internal RESET units may be carried without adding RESET to Σ . The explanatory causal graph may be cyclic and is neither the J construction order nor the K.CVPred/K.CV-WF provenance order.
Core.KCut / Core.KCutClosure_post / Core.KCutObserve	K-facing settled/quiescent comparison domain; downstream RTL observation-domain interface; and persistent-blocker preservation into that domain. In the standard scope QSync-Post derives KCutClosure_post; K.CertCov remains separate package totality.
Core.SyncFence	Constitutive Sys_ref synchronization-completion legality with a separately checked RTL conformance counterpart: blocking operations in pref_S(s) process-facing commit no later than s; same-cycle completion is permitted and equal-cycle interval accounting is certificate semantics rather than physical sub-cycle timing.
Core.MemSyncOrder	Array/synchronization commit-order qualification at the schedule/mapping layer; separate from Appendix I/J replay and from J.Mem cross-process shared-memory lowering. Cache/merge/split/reorder must preserve the fence, and one merged storage commit cannot span opposite sides of one synchronization boundary.
Core.MemFaithful / J.Mem / Core.MemProfile	Mapped-memory observation/profile architecture. MemFaithful compares transformed physical realization with the declared untransformed logical mapping semantics at the selected boundary and preserves selected J behavior plus K blocking/dependency facts. J.Mem completely lowers every qualifying cross-process memory dependency or uses the separate memory-state extension. MemProfile packages a reusable transformation-invariant logical boundary, including

Name	Role
	MemFaithful/MemSyncOrder/J.Mem, K disposition, A6/Q.1 ownership/elaboration, and reset treatment.
Core.7b-Core.7e / SyncCarrierWF / SyncFire / EpochGateAdvance	Separate synchronization/lowering carrier architecture. SyncOf/SyncTxn/participants/pending-ready-blocked/SyncReleaseOwnerSets type current source synchronization without widening MsgOf or message BoundaryReq. SyncFire is the real already-pending/ready source synchronization transaction. EpochGateAdvance is lowering-only explicit Sys_K gate progress erased by K.Low projection. Core.SyncInFlightOpportunityQual specializes InFlightCycleExt for these constructors.

13.2 Appendix I

Name	Role
I.A0	RTL E3 message-issue discipline assumption.
I.ReplayObs	Logical replay checkpoint determined by committed history, stimulus, and witness; excludes raw preparation and current request state.
I.CutpointOfferObs	Operational current-offer slice carrying Pending_P, Front_P, BoundaryReq, attribution, boundary, direction, epoch, and stable Push payload facts.
I.ReplayObs-Congruence	Immediate same-unit replay closure plus paired congruence of the exact selected normalization paths.
I.DR / I.DR'	Deterministic equality of logical replay checkpoints at selected normalized and canonical immediate post-unit cutpoints.
I.DeferredNBHole	Carried unresolved shared non-blocking decision with NoDependentUse.
I.3	Finite process-local preparation script.
I.4	Construction of the J.PCert records.

13.3 Appendix J

Name	Role
J.LocalGWR	Complete local certificate package; no global replay assumption.
J.DepJ / J.DepSound / J.HorizonOrderSafe / DepComplete / DepRank / DepAcyclic	Generate D1-D4 semantic prerequisites and D5 Horizon proof-stratification edges; prove semantic soundness only for D1-D4, validate D5-only pairs by commutation, complete all noncommuting dependencies, and use the graph-independent foundation rank to prove acyclicity.
Finite ideals / J.ValidatedConstructionOrder	Finite canonical ideals/down-closed required-order prefixes used by the J induction; tie ordering among commuting units has no semantic significance. J.RefProjCover uses the same finite-ideal notion for one-run source coverage.
J.RegPhaseNF	Normalize same-Horizon actions to registered L1/L2/L3 phases. It consumes a finite L2-settle existence fact; Core.QSync-Ref supplies it uniformly on the normal compositional route, while a direct J.GWR route may carry per-Horizon evidence.

Name	Role
J.NBDecision	Decide each deferred non-blocking call from the common settled snapshot.
J.UnitExt	Before the next nonempty bucket, take any finite evidence-determined invariant-preserving unit-empty CompleteCycle/SilentCycle bridge required by cycle-sensitive source/environment/reset evidence under Core.SelectLatitude; then extend the complete Horizon bucket. No synthetic rank bucket or B2/B3 premise is introduced.
J.GWR-Comp	Normal local-to-global witness construction from J.LocalGWR. The alternate J.GWR-Direct route must independently validate the stable J.GWR interface and all J.ValidatedConstructionOrder V1-V6 obligations; V6 requires the within-Horizon Core.LAdm/J.RegPhaseNF phase normal form. The direct route does not weaken the order/phase/cutpoint checks.
J.1	System-level observable compatibility. Normal signoff is Post -> Ref existence from J.GWR-Comp or a directly validated J.GWR package. The restricted Ref -> Post clause is selected-pair compatibility only: J.RTLConsistentRefPrefix already supplies a selected reachable RTL prefix plus compatible annotations; the theorem does not construct an RTL execution from an arbitrary source run.
J.0	Compose an already selected compatible pair. On normal Post -> Ref, J.GWR-Comp supplies that pair; on restricted Ref -> Post, J.RTLConsistentRefPrefix supplies it.
J.TraceCertCov / J.RefProjCover	Safety-side RTL-prefix certificate coverage and optional one-complete-source-run coverage. J.TraceCertCov is route-sensitive; for CycleReach_post the standard QSync route derives it from J.QSyncTraceInd. Core.RTLCycleCarrier/J.CycleIdealCover can then extend J.IdealPrefixClosed safety to non-cycle-aligned finite ideals without per-prefix packages, while properties that fail ideal-prefix closure still need direct broader-domain J.TraceCertCov. J.RefProjCover uses finite canonical ideals; Core.SelectLatitude-legal delay matters to that one-run coverage only when it changes canonical Σ_{DUT} history, not for pure unit-empty delay already hidden by the canonical quotient. Neither predicate implies K.CertCov.
J.IdealPrefixClosed / J.1-Safe / J.1-Safe-QSync	J.IdealPrefixClosed defines safety closure under finite required-order ideals, not just suffix deletion from a linear trace. J.1-Safe transfers such a J.HCanon-invariant property through Post -> Ref under route-sensitive J.TraceCertCov. J.1-Safe-QSyncCycle is the cycle-aligned standard-QSync instantiation; J.CycleIdealCover plus J.1-Safe-QSync extend it to every reachable finite RTL prefix. One-run source use additionally requires J.RefProjCover.
J.CycleReach_post / J.QSyncTraceInd / J.CycleIdealCover	Cycle-aligned RTL safety domain and base/one-cycle/exhaustive-successor package induction carrying only LocalGWR/J.GWR-Comp plus J.HCanon; K-only Offer/KObs/drain facts are excluded. J.QSyncCertInd-Trace can supply it. Core.RTLCycleCarrier provides the within-already-entered-cycle execution carrier, and J.CycleIdealCover maps every reachable finite outer canonical history to a finite ideal of a reachable cycle-aligned history.

Name	Role
J.QSyncCertState / J.QSyncCertInd / J.CertExist-QSync	Richer canonical-cutpoint package state and anti-circular base/one-cycle/exhaustive-successor induction. Carries J.HCanon/OfferReach/OfferConf/KObsConf, theorem-derived J.RegPhaseReach, and QC5 K.DrainReach. On the normal route QC5 may cite K.DrainReach-Comp from the bound DR1-DR4 instance evidence; CI1 rebuilds the successor attribution/history/QSync facts and re-applies the lemma rather than assuming arbitrary successor DrainReach. J.CertExist-QSync lifts the induction to total J-side package existence; Appendix K's K.CertCov-QSync binds those tuples into K.CertCov.
Core.18 / J.HCanon / OfferReach / OfferConf / KObsConf / KObsPost_J	Core fixes the common KObs type/equality seam. J proves common canonical history, source H/O realizability, complete RTL own-side request-to-offer attribution, constructs KObsPost_J, and proves its typed equality with KObs_E,w for the exact selected cutpoint package. The bound E template supplies deterministic settled handshake and ReleaseOwnerSets_E reconstruction; stateless mirror signals are not extra residual offers.
Corollary J.1	Export equal KObs plus the J.KObs-Proc/Chan/Offers/WFG reconstruction facts, including settled handshake, ReleaseOwnerSets_E/WaitOwners_E, and Core.WFG. It makes no K deadlock, waiver, environment-terminal, or diagnostic conclusion.
J.KObs-SyncCarrier / J.KObs-SyncInFlight / J.KObs-InFlight	Source-side current-state reconstruction from the common KObs. These lemmas recover typed synchronization carrier/readiness and local SyncFire effects, then the selected Sys_ref FlightStateRec/InFlightStepRec action relation. They do not supply future action opportunities.
J.InFlightPathSound	Model-indexed operational realization of every finite reconstructed in-flight path under Q.2 effect soundness plus Core.InFlightCycleExt/CycleClosure-equivalent opportunity and prefix-compositional profile closure. This is the level-3 bridge that makes generalized DrainClosedNow an operationally sound abstraction.

13.4 Appendix K

Name	Role
K.Dead / K.DeadW / K.ReleaseSCCReduction	Literal message-rooted reachable generalized-drain-closed internal wait-cycle predicates. K.Dead uses exact ReleaseOwnerSets_E closure plus coarse-WFG connectivity and generalized DrainClosedNowRec. K.ReleaseSCCReduction now relies on K.DrainClosedHeredity: process-indexed WaitRoots restriction gives InFlightEscape monotonicity and therefore drain-closure inheritance to a terminal SCC. Synchronization is not added to the literal WFG.
K.O / K.DeadChar	K.O reconstructs the frontier from KObs-derived inputs. K.DeadChar is the source-side/lowered-KCut disabled-channel characterization used by K.Dead and cites Core.12/Core.13; it is not an arbitrary RTL microstate theorem.
K.KObs-Dead / K.KObs-Ext	K-owned factoring and equal-KObs transfer of release-aware deadlock, waiver, environment-terminal, and diagnostic predicates. Equality includes the reconstructed ReleaseOwnerSets_E/WaitOwners_E and handshake facts; Core.WFG remains the authoritative coarse graph definition.

Name	Role
K.DrainReach / K.DrainReach-Comp / K.RLAdmReach	K.DrainReach supplies nonwithdrawal, explicit $K\epsilon$, Core.Drain/K.Drain, and Core.SettleKBridge for the exact source witness; K.RLAdmReach conjoins it with theorem-derived/direct J.RegPhaseReach to prove Core.LAdm admissibility. On the standard normal compositional route K.DrainReach-Comp derives the exact K.DrainReach field from bound DR1-DR4 instance evidence plus theorem-derived J.GWR-Comp/I.A0 attribution, J.OfferReach/J.HCanon history, and QSync-Ref settling facts. There is no independent DR5 history/attribution certificate.
K.CertifiedCutpoint	Complete package for one reachable RTL KCut plus one recorded reachable-prefix/NormPost pair ending at that KCut.
K.CertCov	Package totality over every reachable RTL KCut. In the standard QSync route, QS2a/Core.QSync-KCutCanonical identify the canonical domain and J.QSyncCertInd $\rightarrow$ J.CertExist-QSync $\rightarrow$ K.CertCov-QSync derive total certified packages. Alternate independently checked total-extraction routes remain valid.
SDet	Flow qualification fixing one fully specified deterministic Core.PolicyS simulator/run; separate from run completeness and response cleanliness.
A5-L	No admissible Policy-L prefix reaches a source KCut satisfying Dead_ K^W . Drain-closedness is part of Dead_ K , not an extra A5-L cutpoint restriction.
K.CV / K.OpKey / K.CVPred/K.CV-WF / K.CompareAdvanceLegal	Serial-route provenance and comparison-advance architecture. K.CV remains authoritative for design-level causal/value determinism; OpKey/CVPred/CV-WF implement occurrence/provenance and comparison-availability; CompareAdvanceLegal bridges Core.16 launch/Issue legality. The generalized in-flight/lowering architecture is a separate qualification layer rather than part of K.CV.
$K\epsilon$ / NB-Align / K.SerialRoutePremises	$K\epsilon$ is the explicit WFG-inert epsilon normalization premise. NB-Align is the separate witness-selected non-blocking condition: a relevant non-NB-CF site is Appendix-L aligned to an explicit obstruction/supplier lowerable through K.Low or is structurally excluded from potential K wait cycles. K.SerialRoutePremises is the authoritative SR1-SR7 applicability record for K.2b, packaging the fixed-instance constructive fairness, blocking/provenance/drain/environment/comparison/reset/lowering conditions without including A5-L or the separate response package.
K.JointFreeze-Ord / K.JointFreeze / K.EnvMF / K.EnvMF-Uniform	Frozen-component no-release/environment applicability. K.JointFreeze-Ord is theorem-derived for the qualifying ordinary or comparability-certified single-frontier/single-singleton-release-support case. Every other admitted StandardEnv candidate - including a single frontier with a multi-owner or multi-alternative ReleaseOwnerSets_E family - requires K.JointFreeze, which may be packaged uniformly for all such candidates. Timed/reactive or otherwise nondefault environments admit only the selected internal frontier plus the K.JointFreeze-Ord release-support shape; K.EnvMF-Uniform is the instance-level sufficient discharge when those conditions hold at every reachable K-facing cutpoint.
K.InterruptionGate / K.DomResetScope / K.ResetEpoch / K.TerminalDisposition	Shared reset-first interruption scheme. Either validate candidate-specific K.DomResetScope/DomEpoch + K.ResetEpoch/K.TerminalDisposition Continue/DirectFail branches, or one uniform U1-U3 instance-level record proving all required Continue branches. The dominance scope contains the modeled process identities associated with source endpoints and the selected elaboration process-dependency/release-support interface plus the consumed boundary identities; stateless couplers are not process members.

Name	Role
K.RespCov / RespClean / CompleteS / Core.CycleClosure / Core.FairPick	Total finite-bound response evidence over one complete selected run, plus typed cycle qualification and the constructive FairPick/FairCycleDovetail evidence used to build the hypothetical fair Policy-L continuation. B2/B3 are the underlying behavioral boundary-progress properties; B5 remains separate.
K.2b / K.2c	Paper-level serial-dominance and A5-S -> A5-L reduction. K.2b consumes K.SerialRoutePremises plus K.RespCov and separate K.CompleteS evidence for the SDet-selected run. K.2c uses A5-S to supply K.RespCov/K.CompleteS/K.RespClean and combines that response package with K.SerialRoutePremises; K.Low.A5 transports the result to Sys_ref when lowering is used. The record centralizes authoritative K.CV/common keys/provenance, release-structure-appropriate JointFreeze/EnvMF, CompareAdvanceLegal, FairPick/FairCycleDovetail, K.Drain/K.DrainReach-Comp applicability, fixed-instance choices, NB-Align/K.Low, and candidate-specific or uniform reset-first interruption discharge. Lean mechanization and concrete flow qualification remain pending.
K.1A / K.1A-Total / K.1	K.1A is the per-certified-cutpoint theorem and consumes A5-L but not the serial-route premises. K.1A-Total adds Core.KCutClosure_post and K.CertCov. In the standard QSync route those universal-domain facts are derived through QSync-Post/QSync-KCutCanonical and J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync. K.1 is the user-facing composition that first obtains A5-L from SDet/A5-S/K.2b/K.2c and then applies the universal layer.
K.WaitRoots / K.InFlightEscape / K.DrainClosedHeredity / K.InFlightRelocate	Generalized drain core. WaitRoots_D is a process-indexed family; InFlightEscape is any finite qualified path changing it; DrainClosedNow is negated escape. The D-independent step relation gives subset heredity. InFlightRelocate realizes an available escape, stops at the first K-facing root change, and re-evaluates all candidate sets without assuming SCC/candidate monotonicity.
K.InFlightQual / Q.2 / K.PrimitiveDrainExact	Model-indexed action/path qualification and primitive compatibility. Q.2 declares current-state action/effect schemas; K.InFlightQual binds effect soundness, finite choice/rank, opportunity/path realization, and precision/search class. K.PrimitiveDrainExact is the restricted theorem under which the old enabled residual-offer fast path equals generalized drain closure.
K.Low.ModalQual / LowAdminNormalize / LowWaitRoots / InFlightPathCorr / DeadRootBridge	Qualified G2 lowering transport. LowAdminNormalize removes finite lowering-only residue; LowWaitRoots is an extended source proof view, not a new user deadlock predicate. ModalQual MQ1-MQ6 binds local cases, normalization/root correspondence, bidirectional path simulation/lifting with finite-suffix root reflection, separate Sys_K/Sys_ref InFlightQual, and certificate versions. InFlightEscapeCorr + DeadRootBridge yield ordinary DrainClosedNow correspondence for literal source Dead_K candidates and generalized K.Low.A5.

13.5 Appendix L

Name	Role
------	------

Name	Role
L.1	Causal witness selector for epsilon-modeled source choices at generic epsilon-quiescent source choice points. It is intentionally not restricted to Core.KCut, whose additional boundary-evaluation condition is K-specific.
L.2	Conditional coverage/causality/noninterference theorem under which a realizable snoop-selected witness proves WC.
L.3	NB-CF identity witness; no snooping required for irrelevant failed polls.
L.4	Cadence-sensitive polling requires isolation/snoop alignment.
L.Dir	Ordinal-preserving source consumption of a free-running RTL snoop stream; benign cycle skew is allowed, while order/kind/payload divergence means witness non-existence and no theorem-backed guarantee.
L.5	Online harness realization of an existing abstract witness selector, with W1-W5, L.Dir, coverage, and clean failure reporting as separate obligations.
W1-W5	Recorded ordinal observation, source-side commit gating, registered sampling, comparison/watchdog with proved-versus-inconclusive timeout treatment, and separate coverage proof.

13.6 Appendix M evidence packaging

Name	Role
M.7a discharge ledger	Definitive per-instance trigger/discharge/evidence record. The 31 August additions make generalized drain and lowering explicit: Q.2/K.InFlightQual/J.InFlightPathSound are model-indexed application obligations; profile G2 has a dedicated K.Low.ModalQual/MQ1-MQ6 entry whose MQ4 projection/lifting/root-reflection clause is a universal qualification premise, not a result manufactured by InFlightPathCorr. The existing scheduling, QSync, memory, reset, serial, and response entries remain independently triggered.
M.7b standard QSync certificate profile	Four-bundle packaging overlay over M.7a. Standard safety uses bundles 1-2. One-cutpoint K.1A adds the applicable Sys_ref K.InFlightQual and A5-L route. Universal Policy-S K uses all four bundles; G1 binds one source/serial generalized-drain profile, while G2 additionally binds the lowering ModalQual package and both model-indexed InFlightQual records. The bundle names remain packaging interfaces, not new theorem predicates.
K.CertHdr / K.CertPkg binding	Common theorem-instance/version/trust boundary. Generalized-drain packages bind the exact Q.2 action/effect schema, model/profile identity, K.InFlightQual and search precision class; G2 additionally binds Core.SyncCarrierWF/SyncFire/EpochGateAdvance identities and K.Low.ModalQual/LowAdminNormalize/LowWaitRoots/InFlightPathCorr checker schemas plus both model-indexed K.InFlightQual records. Old offer-only or earlier-version lowered certificates do not silently establish the current generalized theorem.

13.7 Architecture summary

SUMMARY

Appendix F remains the architect-facing applicability screen. The Core/G/I/J safety architecture is unchanged in shape: the Core fixes the qualified operational model and observable contract, I supplies process-local replay/preparation, and J constructs reachable source witnesses and typed KObs equality. The 31 August $q \rightarrow K$ work strengthens the $J \rightarrow K$ seam without expanding KObs: Core.18 FlightStateRec/InFlightStepRec and J.KObs-InFlight reconstruct the current finite already-admitted action system; Q.2 gives exact action/effect schemas; Core.InFlightCycleExt/J.InFlightPathSound separately prove operational realization. K.WaitRoots/InFlightEscape generalize DrainClosedNow from an offer-only snapshot to finite reachability, with D scoping roots rather than actions; DrainClosedHeredity repairs SCC reduction and InFlightRelocate handles changing candidate structure. Stage 2 types synchronization separately through Core.7b-Core.7e, keeping MsgOf/BoundaryReq/WFG/Dead_K message-rooted, and adds the global K.Low modal stuttering correspondence through LowAdminNormalize, LowWaitRoots, ModalQual, InFlightPathCorr, InFlightEscapeCorr, DeadRootBridge, and generalized K.Low.A5. The deterministic Policy-S response/serial-dominance route, QSync universal package-totality route, and Appendix-L witness-selection role remain intact around this stronger drain/lowering interface. $q \rightarrow K$ is closed at the paper rules-architecture level; remaining work is mechanization plus concrete Q.2, InFlightCycleExt/path, and ModalQual/MQ4 qualification for actual templates and flows. Appendix M.7a remains the definitive per-instance discharge ledger and M.7b the packaging view. The scheduling-rules document remains normative if any simplified explanation in this guide differs.

Source note: theorem and definition names follow the Catapult HLS User View Scheduling Rules dated 31 August 2026